

Comprehensive Transmission Cost Calculator – Full Documentation

Contents

1	Master Methodology Map	15
1.1	Introduction	15
1.2	Visual Map of the Method	15
1.3	How the Method Fits Together	15
1.3.1	Preprocessing	15
1.3.2	Capital and revenue	17
1.3.3	Risk	17
1.3.4	Benefits	17
1.3.5	Operations, losses, and emissions	17
1.3.6	Benefit–cost ratio	17
1.4	Concepts at a Glance	17
2	CTCC Orchestrator Documentation	19
2.1	Introduction	19
2.2	Method Overview	19
2.2.1	Purpose	19
2.2.2	Calculation Approach	20
2.2.3	Inputs and Outputs	20
2.2.4	Integration with Other Scripts	20
2.3	Key Functions	21
2.3.1	<code>run_script(script_name, quiet=False)</code>	21
2.3.2	<code>build_bcr_data_from_json(json_results)</code>	21
2.3.3	<code>aggregate_json_outputs(scenario_id, output_dir="outputs")</code>	22
2.3.4	<code>write_final_json_output(agggregator, bcr_results, scenario_id, output_dir, csv_equivalent, summary_override)</code>	22
2.3.5	<code>main()</code>	23
2.4	Example	23
3	CTCC YAML Configuration Guide	24
3.1	Introduction	24
3.1.1	Purpose	24
3.1.2	How to Use This Guide	24
3.1.3	Project Categories	25
3.1.4	YAML File Inventory	25
3.2	Core Project Configuration	25

3.2.1	Purpose	25
3.2.2	Parameter Choices	25
3.2.3	Example Configuration	29
3.3	Physical Project Details	29
3.3.1	Purpose	29
3.3.2	Parameter Choices	30
3.3.3	Total Line Length	31
3.3.4	Example Configuration	31
3.4	Financing Parameters	31
3.4.1	Purpose	32
3.4.2	Parameter Choices	32
3.4.3	Example Configuration	33
3.5	Insurance Configuration	34
3.5.1	Purpose	34
3.5.2	Parameter Choices	34
3.5.3	Example Configuration	35
3.6	Delay Costs	35
3.6.1	Purpose	35
3.6.2	Parameter Choices	35
3.6.3	Total Annual Delay Cost	35
3.6.4	Example Configuration	36
3.7	Wildfire Risk Parameters	36
3.7.1	Purpose	36
3.7.2	Parameter Choices	36
3.7.3	Example Configuration	37
3.8	Outage Risk Parameters	38
3.8.1	Purpose	38
3.8.2	Parameter Choices	38
3.8.3	Example Configuration	40
3.9	Environmental Mitigation	41
3.9.1	Purpose	41
3.9.2	Parameter Choices	41
3.9.3	Example Configuration	42
3.10	Right-of-Way Configuration	43
3.10.1	Purpose	43
3.10.2	Parameter Choices	43
3.10.3	Zone Definitions	44
3.10.4	Example Configuration	44
3.11	O&M Vegetation Management	45
3.11.1	Purpose	45
3.11.2	Parameter Choices	45
3.11.3	Example Configuration	46
3.12	Category-Specific O&M Costs	46
3.12.1	Conductor O&M	46
3.12.2	Structure O&M	46

3.12.3	Converter O&M	47
3.13	Emissions Reductions	47
3.13.1	Purpose	47
3.13.2	Parameter Choices	47
3.13.3	Example Configuration	48
3.14	Congestion Reductions	49
3.14.1	Purpose	49
3.14.2	Parameter Choices	49
3.14.3	Example Configuration	50
3.15	Curtailment Reductions	51
3.15.1	Purpose	51
3.15.2	Parameter Choices	51
3.15.3	Example Configuration	52
3.16	Cost Timing Patterns	52
3.16.1	Purpose	52
3.16.2	Parameter Choices	52
3.16.3	Cost Categories	52
3.16.4	Example Configuration	54
3.17	Category-Specific Technical Data	54
3.17.1	ROW Widths	54
3.17.2	Circuit and Resistance Details	54
3.18	Special Considerations	55
3.18.1	Project Category Construction	55
3.18.2	Reconductoring vs. Greenfield	55
3.18.3	AC vs. DC	56
3.18.4	Construction Type Effects	56
3.18.5	Terrain Distribution	57
3.19	Summary	57
4	Path Config Documentation	58
4.1	Introduction	58
4.2	Method Overview	58
4.2.1	Purpose	58
4.2.2	Calculation Approach	58
4.2.3	Inputs and Outputs	59
4.2.4	Integration with Other Scripts	59
4.3	Key Functions	59
4.3.1	Constants	59
4.4	Example	60
5	Smart Loaders Documentation	61
5.1	Introduction	61
5.2	Method Overview	61
5.2.1	Purpose	61
5.2.2	Calculation Approach	61

5.2.3	Inputs and Outputs	62
5.2.4	Integration with Other Scripts	62
5.3	Key Functions	62
5.3.1	Module-level logic (conditional import)	62
5.3.2	<code>get_input_mode()</code>	62
5.3.3	<code>get_project_data_raw()</code>	62
5.3.4	<code>get_financing_data_raw()</code>	62
5.3.5	<code>get_physical_data_raw()</code>	63
5.3.6	<code>load_terrain_miles()</code>	63
5.3.7	Re-exported <code>load_*</code> (see yaml/json loader docs)	63
5.4	Example	63
6	JSON Loaders Documentation	64
6.1	Introduction	64
6.2	Method Overview	64
6.2.1	Purpose	64
6.2.2	Calculation Approach	64
6.2.3	Inputs and Outputs	65
6.2.4	Integration with Other Scripts	65
6.3	Key Functions	65
6.3.1	<code>JSONDataSource</code>	65
6.3.2	<code>set_json_data(combined_json)</code>	66
6.3.3	<code>load_project_technical_details()</code>	66
6.3.4	<code>load_financing_details()</code>	66
6.3.5	<code>load_congestion_curtailment_reductions()</code>	66
6.3.6	<code>load_primary_bcr_config()</code>	67
6.3.7	Other <code>load_*</code> functions	67
6.4	Example	67
7	CSV Output Manager Documentation	68
7.1	Introduction	68
7.2	Method Overview	68
7.2.1	Purpose	68
7.2.2	Architecture	68
7.2.3	Integration with Other Scripts	69
7.3	Key Functions	69
7.3.1	<code>__init__(output_dir, scenario_id)</code>	69
7.3.2	<code>load_technical_details()</code>	69
7.3.3	<code>write_batch_summary()</code>	70
7.3.4	<code>write_module_csv(module_name, detail_rows, summary_row, append)</code>	70
7.3.5	Module-Specific Add Methods	71
7.3.6	<code>add_congestion_curtailment(results)</code>	72
7.3.7	<code>add_bcr_metrics(bcr_results)</code>	72
7.4	Example	72

7.5	Key Implementation Details	74
7.5.1	Scenario ID Management	74
7.5.2	Row Update vs. Append	74
7.5.3	Technical Parameters Prepend	74
7.5.4	Fieldname Management	74
7.5.5	Detail vs. Summary Rows	74
7.5.6	Path Resolution	75
7.5.7	Error Handling	75
8	JSON Output Manager Documentation	76
8.1	Introduction	76
8.2	Method Overview	76
8.2.1	Purpose	76
8.2.2	Calculation Approach	77
8.2.3	Inputs and Outputs	77
8.2.4	Integration with Other Scripts	77
8.3	Key Functions	77
8.3.1	<code>JSONOutputManager.__init__(scenario_id=None)</code>	77
8.3.2	<code>load_technical_details()</code>	78
8.3.3	<code>add_*</code> () methods	78
8.3.4	<code>calculate_summary()</code>	78
8.3.5	<code>get_json_results()</code>	78
8.3.6	<code>save_to_file(output_dir=None, module_name=None)</code>	79
8.3.7	<code>load_from_file(json_file)</code>	79
8.4	Example	79
9	Smart Output Documentation	81
9.1	Introduction	81
9.2	Method Overview	81
9.2.1	Purpose	81
9.2.2	Calculation Approach	81
9.2.3	Inputs and Outputs	82
9.2.4	Integration with Other Scripts	82
9.3	Key Functions	82
9.3.1	<code>SmartOutputManager</code> (exported as <code>CTCCOutputManager</code>)	82
9.3.2	<code>__init__()</code>	82
9.3.3	<code>write_batch_summary()</code>	83
9.3.4	<code>__getattr__(name)</code>	83
9.4	Example	83
10	Weighted Miles Documentation	84
10.1	Introduction	84
10.2	Method Overview	84
10.2.1	Purpose	84
10.2.2	Calculation Approach	84

10.2.3	Inputs and Outputs	85
10.2.4	Integration with Other Scripts	85
10.3	Key Functions	85
10.3.1	<code>calculate_weighted_miles()</code>	85
10.4	Example	86
11	Build Costs Calculation Documentation	87
11.1	Introduction	87
11.2	Method Overview	87
11.2.1	Purpose	87
11.2.2	Calculation Approach	88
11.2.3	Inputs and Outputs	88
11.2.4	Integration with Other Scripts	89
11.3	Key Functions	89
11.3.1	<code>load_costs(category, total_miles, number_of_converters, contingencies, reconductoring)</code>	89
11.3.2	<code>main()</code>	90
11.4	Example	91
11.5	Key Implementation Details	92
11.5.1	Reconductoring Handling	92
11.5.2	AFUDC Calculation	92
11.5.3	Present Value of Build Costs	92
11.5.4	Weighted Miles	92
12	Right-of-Way Costs Calculation Documentation	93
12.1	Introduction	93
12.2	ROW Agreement Types	94
12.2.1	Permanent easement (new)	94
12.2.2	Lease / license (existing)	94
12.2.3	Fee simple purchase	94
12.2.4	Federal / hybrid	94
12.3	Mutual Exclusivity and Defaults	95
12.4	Configuring YAMLS for ROW Costs	95
12.4.1	New greenfield (permanent easement)	95
12.4.2	Existing corridor (lease / license)	95
12.4.3	Fee simple purchase	96
12.4.4	Federal / hybrid (upfront + annual)	96
12.4.5	Explicit override of inferred type	96
12.4.6	Quick reference	96
12.5	Method Overview	96
12.5.1	Purpose	96
12.5.2	Acres and Zone-Level Potential Costs	97
12.5.3	Inputs and Outputs	97
12.6	AFUDC, Capital vs. Operational	98
12.7	Key Functions	99

12.7.1	<code>calculate_zone_costs(row_width_feet)</code>	99
12.7.2	<code>main()</code>	99
12.8	Present Value Timing	100
12.9	Acres Calculation	100
12.10	Integration	100
13	Environmental Mitigation Costs Calculation Documentation	101
13.1	Introduction	101
13.2	Method Overview	101
13.2.1	Purpose	101
13.2.2	Calculation Approach	102
13.2.3	Inputs and Outputs	102
13.2.4	Integration with Other Scripts	103
13.3	Key Functions	103
13.3.1	<code>calculate_environmental_mitigation_costs(em_yaml, category, terrain_miles_dict, row_width_feet, reconductoring)</code>	103
13.3.2	<code>main()</code>	105
13.4	Example	105
13.5	Key Implementation Details	106
13.5.1	Uplift Factor	106
13.5.2	Credit Ratios	106
13.5.3	Habitat Terrain Classification	107
13.5.4	Present Value Timing	107
13.5.5	AFUDC Treatment	107
13.5.6	Reconductoring Projects	107
14	Insurance Costs Calculation Documentation	108
14.1	Introduction	108
14.2	Method Overview	108
14.2.1	Purpose	108
14.2.2	Calculation Approach	109
14.2.3	Inputs and Outputs	109
14.2.4	Integration with Other Scripts	110
14.3	Key Functions	110
14.3.1	<code>calculate_insurance_costs(insurance_yaml, conductor_cost_with_contingencies, structure_cost_with_contingencies, converter_cost_with_contingencies, construction_type, project_lifetime)</code>	110
14.3.2	<code>calculate_wildfire_liability_premium(insurance_yaml, project_lifetime)</code>	111
14.3.3	<code>main()</code>	112
14.4	Example	112
14.5	Key Implementation Details	113
14.5.1	Insurable Components	113
14.5.2	Premium Rate by Construction Type	113

14.5.3	Payment Timing	113
14.5.4	Discount rate and perspective	114
14.5.5	Wildfire Liability	114
14.5.6	AFUDC Treatment	114
14.5.7	Dependency on Build Costs	114
15	Delay Costs Calculation Documentation	115
15.1	Introduction	115
15.2	Method Overview	115
15.2.1	Purpose	115
15.2.2	Calculation Approach	115
15.2.3	Inputs and Outputs	116
15.2.4	Integration with Other Scripts	116
15.3	Key Functions	116
15.3.1	<code>load_project_technical_details()</code>	116
15.3.2	<code>main()</code>	117
15.4	Example	118
15.5	Key Implementation Details	118
15.5.1	Delay Cost Components	118
15.5.2	AFUDC Treatment	119
15.5.3	Present Value Timing	119
15.5.4	Simplified Structure	119
16	Wildfire Costs Calculation Documentation	120
16.1	Introduction	120
16.2	Method Overview	120
16.2.1	Purpose	120
16.2.2	Calculation Approach	120
16.2.3	Inputs and Outputs	121
16.2.4	Integration with Other Scripts	122
16.3	Key Functions	122
16.3.1	<code>get_discount_rate_from_config (from financial_utils)</code>	122
16.3.2	<code>calculate_wildfire_costs(wildfire_yaml, construction_type, terrain_miles, project_lifetime, discount_rate, delay_years=0, construction_years=0)</code>	122
16.3.3	<code>main()</code>	123
16.4	Example	124
16.5	Key Implementation Details	125
16.5.1	Payment timing (COD)	125
16.5.2	Severity definition (uninsured residual)	125
16.5.3	Discount rate and perspective	125
16.5.4	Probabilistic Risk Approach	125
16.5.5	Construction Type Multipliers	125
16.5.6	Growing Annuity Formula	126
16.5.7	Edge Cases	126

16.5.8	AFUDC Treatment	126
16.5.9	Warning for High Growth Rates	126
17	Outage Costs Calculation Documentation	127
17.1	Introduction	127
17.2	Method Overview	127
17.2.1	Purpose	127
17.2.2	Calculation Approach	127
17.2.3	Inputs and Outputs	128
17.2.4	Integration with Other Scripts	129
17.3	Key Functions	129
17.3.1	<code>get_discount_rate_from_config</code> (from <code>financial_utils</code>)	129
17.3.2	<code>calculate_voll_cost_piecewise</code> (<code>duration_hours</code> , <code>mw_lost</code> , <code>tiers</code>)	129
17.3.3	<code>calculate_outage_costs</code> (<code>outage_yaml</code> , <code>construction_type</code> , <code>terrain_miles</code> , <code>capacity_mw</code> , <code>project_lifetime</code> , <code>discount_rate</code>)	130
17.3.4	<code>main()</code>	131
17.4	Example	131
17.5	Key Implementation Details	133
17.5.1	Piecewise Value of Lost Load (VoLL)	133
17.5.2	Multiplicative Duration Model	133
17.5.3	Capacity at Risk Factor	133
17.5.4	Growing Annuity Formula	133
17.5.5	AFUDC Treatment	133
18	Congestion and Curtailment Benefits Calculation Documentation	134
18.1	Introduction	134
18.2	Method Overview	134
18.2.1	Purpose	134
18.2.2	Calculation Approach	135
18.2.3	Inputs and Outputs	135
18.2.4	Integration with Other Scripts	136
18.3	Key Functions	136
18.3.1	<code>allocate_curtailment_then_congestion</code> (<code>DC_eff_mw</code> , ...)	136
18.3.2	<code>calculate_congestion_reduction_costs</code> (...)	137
18.3.3	<code>main()</code>	138
18.4	Example	138
18.5	Key Implementation Details	140
18.5.1	Double-Counting Avoidance	140
18.5.2	Saturation Factors	140
18.5.3	Residual Exceedance	140
18.5.4	Flow Factor	140
18.5.5	Near-Binding Hours	140
18.5.6	Delay and Construction Period Costs	141

19 Benefit of Delivered Energy	142
19.1 Introduction	142
19.2 Method	142
19.2.1 Variables	142
19.2.2 Equations	143
19.3 Implementation	143
20 Energy Losses Calculation Documentation	144
20.1 Introduction	144
20.2 Method Overview	144
20.2.1 Purpose	144
20.2.2 Calculation Approach	145
20.2.3 Inputs and Outputs	145
20.2.4 Integration with Other Scripts	146
20.3 Key Functions	146
20.3.1 <code>get_total_energy_losses()</code>	146
20.3.2 <code>print_results(...)</code>	147
20.3.3 <code>main()</code>	147
20.4 Example	147
20.5 Key Implementation Details	148
20.5.1 AC vs. DC Calculations	148
20.5.2 Full Load Adjustment	148
20.5.3 Converter Losses	148
20.5.4 Resistance Values	148
20.5.5 Multiple Circuits/Poles	149
20.5.6 Reconductoring: Old Voltage, New Resistance	149
20.5.7 Standalone vs. Library Use	149
21 Emissions Costs Calculation Documentation	150
21.1 Introduction	150
21.2 Method Overview	150
21.2.1 Purpose	150
21.2.2 Calculation Approach	150
21.2.3 Inputs and Outputs	151
21.2.4 Integration with Other Scripts	152
21.3 Key Functions	152
21.3.1 Loss data from <code>energy_losses.get_total_energy_losses()</code>	152
21.3.2 <code>calculate_energy_mix_by_year(energy_source_mix_details, project_lifetime)</code>	152
21.3.3 <code>calculate_emissions_by_year(tec, energy_mix, emission_intensities)</code>	153
21.3.4 <code>calculate_lifetime_emissions(...)</code>	154
21.3.5 <code>main()</code>	155
21.4 Example	155
21.5 Key Implementation Details	156

21.5.1	Compensation Percentage	156
21.5.2	Energy Mix Evolution	156
21.5.3	Emission Intensities	156
21.5.4	Societal Costs	157
21.5.5	Present Value Timing	157
21.5.6	Year-by-Year Calculation	157
22	Line Loss Costs/Benefits Calculation Documentation	158
22.1	Introduction	158
22.2	Method Overview	158
22.2.1	Purpose	158
22.2.2	Calculation Approach	158
22.2.3	Inputs and Outputs	159
22.2.4	Integration with Other Scripts	159
22.3	Key Functions	160
22.3.1	<code>load_project_details()</code>	160
22.3.2	<code>calculate_configuration_losses(...)</code>	160
22.3.3	<code>main()</code>	161
22.4	Example	161
22.5	Key Implementation Details	163
22.5.1	Greenfield vs. Reconductoring	163
22.5.2	Comparison Methods for Reconductoring	163
22.5.3	Counterfactual Baseline	163
22.5.4	Present Value Timing	163
22.5.5	AFUDC Treatment	163
23	Operations and Maintenance Costs Documentation	164
23.1	Introduction	164
23.2	Method Overview	164
23.2.1	Purpose	164
23.2.2	Calculation Approach	164
23.2.3	Inputs and Outputs	165
23.2.4	Integration with Other Scripts	165
23.3	Key Functions	166
23.3.1	<code>load_vegetation_management_om_costs(construction_type)</code>	166
23.3.2	<code>load_conductor_om_costs(construction_type, ac_dc, capacity_mw, conductor_type, converter_type)</code>	166
23.3.3	<code>load_converter_om_costs(construction_type, ac_dc, capacity_mw, conductor_type, converter_type)</code>	167
23.3.4	<code>load_structure_om_costs(construction_type, forested_miles, ...)</code>	167
23.3.5	<code>main()</code>	168
23.4	Example	168
23.5	Key Implementation Details	169
23.5.1	Structure O&M for Overhead Projects	169

23.5.2	Vegetation Management	169
23.5.3	Converter O&M	170
23.5.4	Present Value Timing	170
23.5.5	Component-Level Tracking	170
24	Revenue Documentation	171
24.1	Introduction	171
24.2	Method Overview	171
24.2.1	Purpose	171
24.2.2	Calculation Approach (Option A: Real Stream, Real WACC)	172
24.2.3	Inputs and Outputs	173
24.2.4	Integration with Other Scripts	173
24.3	Key Functions	173
24.3.1	<code>load_rate_based_revenue_parameters()</code>	173
24.3.2	<code>load_project_technical_details()</code>	174
24.3.3	<code>get_rate_base()</code>	174
24.3.4	<code>main()</code>	174
24.4	Example	174
25	Benefit-Cost Ratio Calculator Documentation	176
25.1	Introduction	176
25.2	Method Overview	176
25.2.1	Purpose	176
25.2.2	Calculation Approach	176
25.2.3	Inputs and Outputs	177
25.2.4	Integration with Other Scripts	177
25.3	Key Functions	178
25.3.1	<code>load_scenario_data(scenario_id, output_dir)</code>	178
25.3.2	<code>calculate_benefits(data)</code>	178
25.3.3	<code>calculate_costs(data)</code>	179
25.3.4	<code>calculate_bcr_metrics(benefits, costs)</code>	179
25.3.5	<code>calculate_and_display_bcr(scenario_id, output_dir)</code>	180
25.3.6	<code>print_bcr_summary(benefits, costs, bcr_metrics, data)</code>	181
25.4	Example	181
25.5	Key Implementation Details	182
25.5.1	Line Loss Treatment	182
25.5.2	Multiple BCR Perspectives	182
25.5.3	Net Benefits	183
25.5.4	Division by Zero Protection	183
25.5.5	Data Loading	183
26	BCR (Benefit-Cost Ratio) Cheat Sheet	184
26.1	BCR Metrics (order matches Benefit-Cost Ratio Metrics tables)	184
26.1.1	<code>bcr_system</code>	184
26.1.2	<code>bcr_utility</code>	185

26.1.3	bcr_ratepayer	185
26.1.4	bcr_capital	185
26.1.5	bcr_capital_and_delay	185
26.1.6	bcr_primary	185
26.1.7	bcr_excluding_wildfire_risk_and_outage_risk	185
26.1.8	bcr_excluding_emissions	186
26.1.9	bcr_excluding_linelosses	186
26.1.10	bcr_excluding_emissions_and_linelosses	186
26.1.11	bcr_excluding_emissions_and_wildfire_risk_and_outage_risk	186
26.1.12	bcr_excluding_linelosses_and_wildfire_risk_and_outage_risk	186
26.1.13	bcr_excluding_emissions_and_linelosses_and_wildfire_risk_and_outage_risk	186
26.1.14	bcr_excluding_wildfire_risk	186
26.1.15	bcr_excluding_emissions_and_wildfire_risk	186
26.1.16	bcr_excluding_linelosses_and_wildfire_risk	186
26.1.17	bcr_excluding_emissions_and_linelosses_and_wildfire_risk	187
26.1.18	bcr_excluding_outage_risk	187
26.1.19	bcr_excluding_emissions_and_outage_risk	187
26.1.20	bcr_excluding_linelosses_and_outage_risk	187
26.1.21	bcr_excluding_emissions_and_linelosses_and_outage_risk	187
26.2	Net Benefits	187
26.2.1	System/Societal Net Benefits	187
26.2.2	Stakeholder Net Benefits	188
26.2.3	Partial Exclusion Net Benefits (16 Combinations)	188
26.3	Quick Decision Guide	189
26.4	Important Notes	189
26.5	Example: Scenario 1 (Rural Overhead AC 657MW)	189
27	Utility Modules Documentation	191
27.1	Introduction	191
27.2	yaml_loaders.py	191
27.2.1	Method Overview	191
27.2.2	Key Functions	191
27.2.3	Example	193
27.3	financial_utils.py	193
27.3.1	Method Overview	193
27.3.2	Key Functions	194
27.3.3	Example	197
27.4	calculation_utils.py	198
27.4.1	Method Overview	198
27.4.2	Key Functions	198
27.4.3	Example	201
27.5	weighted_miles.py	201
27.5.1	Method Overview	201
27.5.2	Key Functions	202
27.5.3	Example	202

27.6	Integration Examples	202
27.6.1	Combined Usage	202
27.7	Key Implementation Details	203
27.7.1	Path Resolution	203
27.7.2	Fisher Equation	203
27.7.3	Full Load Adjustment	204
27.7.4	AFUDC Timing	204
27.7.5	Terrain Multipliers	204
27.8	constants.py	204
27.8.1	Method Overview	204
27.8.2	Key Constants	204
28	Master Code Map	205
28.1	Introduction	205
28.2	Execution Flow Diagram	205
28.3	Every Step (Numbered)	205
28.4	Script Inclusion Table	206

Chapter 1

Master Methodology Map

1.1 Introduction

This chapter describes the CTCC method at a conceptual level: what costs and benefits are included, how they depend on one another, and how they combine into a benefit–cost comparison. The aim is a single map of the *method*—no implementation details, only the ideas that define the analysis.

1.2 Visual Map of the Method

Figure 1.1 shows how the main concepts of the method fit together. Two kinds of preprocessing support the rest: **terrain-adjusted distance** (distance scaled by terrain difficulty) and **energy losses** (physical losses on the line). Terrain-adjusted distance feeds into construction and insurance costs; physical energy losses feed into line loss costs and emissions. Capital costs (construction, right-of-way, environmental mitigation) are followed by the revenue requirement (the rate-based transfer from ratepayers to the utility). Risk costs—insurance, delay, wildfire, and outage—come next. Benefits are congestion reduction, curtailment reduction, and delivered energy. Operations and maintenance costs, then emissions and line loss costs, complete the cost side. All costs and benefits are combined into a **benefit–cost ratio** (BCR), which can be computed under several perspectives (e.g. including or excluding certain risk or emissions components).

1.3 How the Method Fits Together

1.3.1 Preprocessing

The method uses two preparatory concepts. **Terrain-adjusted distance** is the line length scaled by terrain difficulty (e.g. mountain, river crossing). It is used to scale construction and insurance costs that are expressed per unit distance. **Energy losses (physical)** are the losses on the line and, for DC, in converters. They are computed from line and converter parameters and feed into the economic cost of losses and into emissions attributable to compensating generation.

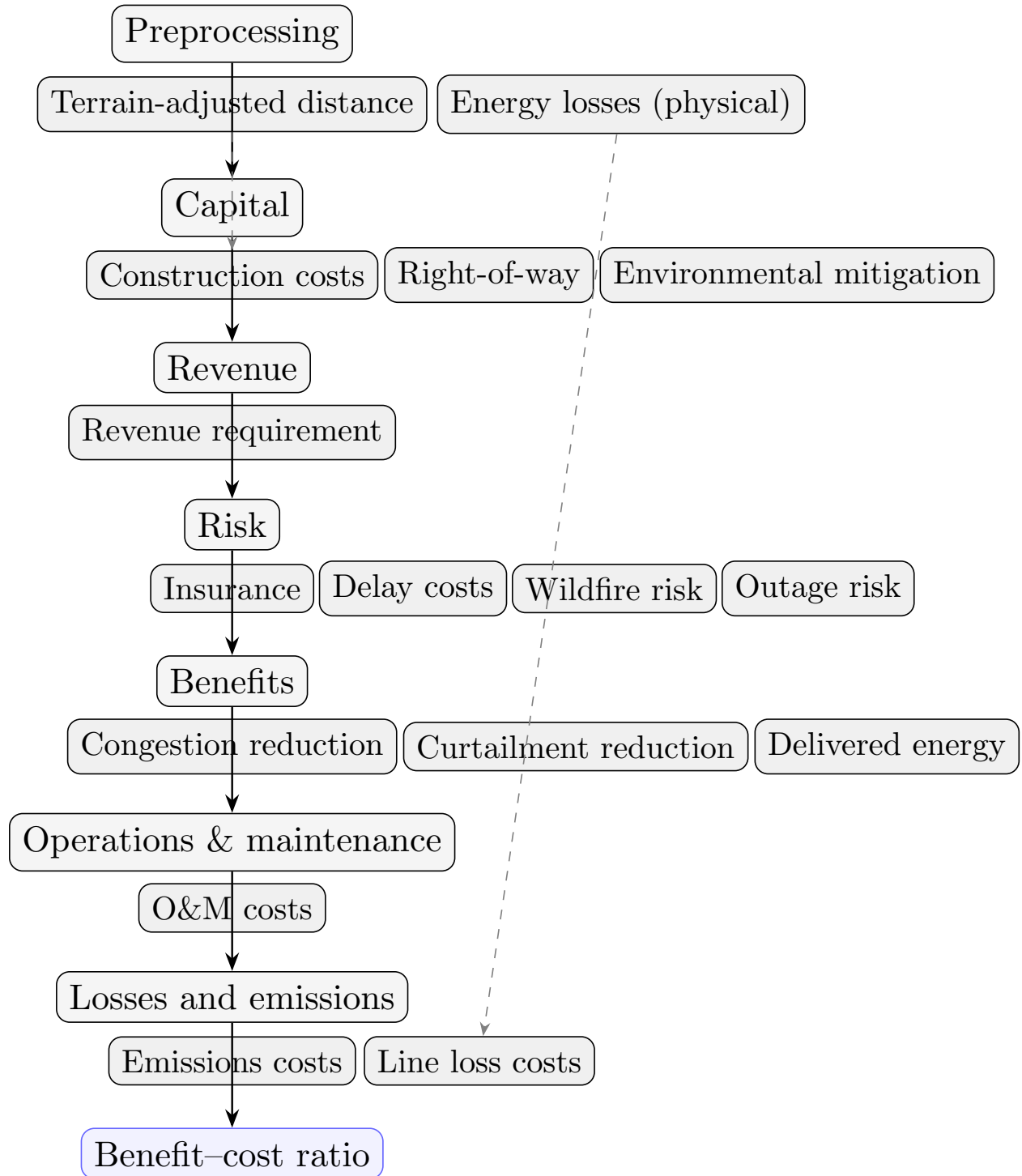


Figure 1.1: Conceptual flow of the CTCC method. Solid arrows show the main sequence; dashed arrows show key dependencies (terrain-adjusted distance into construction costs; physical energy losses into line loss costs).

1.3.2 Capital and revenue

Capital includes construction costs (conductors, structures, converters, adjusted for terrain and contingency), right-of-way costs, and environmental mitigation (temporary impacts and, for new build, wetland and habitat credits). These are combined with financing and timing to produce nominal, regulatory (AFUDC), and present-value measures. The **revenue requirement** is the rate-based revenue needed to recover capital and operating costs; it is a transfer between ratepayers and the utility, not a net social benefit, and is included only in stakeholder-specific benefit–cost views.

1.3.3 Risk

Risk costs include insurance (property, liability), construction and delay costs, wildfire risk (and liability where applicable), and outage risk. Each can be included or excluded depending on the perspective (e.g. societal vs. utility).

1.3.4 Benefits

Benefits are congestion reduction (value of reduced congestion from the project), curtailment reduction (value of reduced curtailment of generation), and the benefit of delivered energy (value of energy the line enables to be delivered each year, valued at an electricity price). Congestion and curtailment benefits can be scaled by conservative haircuts for societal BCRs. Revenue is not counted as a benefit in system-wide or societal perspectives.

1.3.5 Operations, losses, and emissions

Operations and maintenance costs cover vegetation management, conductor and structure O&M, and converter O&M. **Losses and emissions** include the economic cost of line losses (and converter losses for DC) and the cost of emissions from generation that compensates for those losses. Physical energy losses feed into both line loss costs and emissions.

1.3.6 Benefit–cost ratio

All cost categories (capital, operational, energy/emissions, risk, delay) and benefits (congestion reduction, curtailment reduction, and delivered energy) are combined into one or more **benefit–cost ratios**. Different BCR perspectives include or exclude specific cost or benefit components (e.g. wildfire, outage, emissions, line losses) so that analysts can compare full societal, risk-adjusted, or emissions-excluded views.

1.4 Concepts at a Glance

Table [1.1](#) summarizes each concept and its role in the method.

Concept	Role	Brief description
Terrain-adjusted distance	Preprocessing	Distance scaled by terrain difficulty; used to scale construction and insurance.
Energy losses (physical)	Preprocessing	Line and converter losses; feed line loss costs and emissions.
Construction costs	Cost	Capital cost of conductors, structures, converters (terrain- and contingency-adjusted).
Right-of-way	Cost	Cost of land and easements.
Environmental mitigation	Cost	Temporary impacts; wetland/habitat credits for new build.
Revenue requirement	Revenue	Rate-based revenue; transfer, not net benefit.
Insurance	Cost	Property and liability insurance.
Delay costs	Cost	Cost of construction delays.
Wildfire risk	Cost	Expected cost of wildfire risk (and liability if applicable).
Outage risk	Cost	Expected cost of outage risk.
Congestion reduction	Benefit	Value of reduced congestion.
Curtailment reduction	Benefit	Value of reduced curtailment.
Benefit of delivered energy	Benefit	Value of energy the line enables to be delivered (throughput).
O&M costs	Cost	Operations and maintenance (vegetation, conductors, structures, converters).
Emissions costs	Cost	Cost of emissions from loss-compensating generation.
Line loss costs	Cost	Economic cost of line (and converter) losses.
Benefit-cost ratio	Summary	Ratio of benefits to costs under chosen perspective.

Table 1.1: Concepts in the CTCC method and their role.

Chapter 2

CTCC Orchestrator Documentation

2.1 Introduction

The `ctcc.py` script is the CLI entry point for the Comprehensive Transmission Cost Calculator (CTCC). It orchestrates all cost and benefit calculation modules by running them as subprocesses in a fixed order, sets environment variables for input/output mode (YAML/JSON, CSV/JSON) and optional flags, and aggregates results. In CSV mode it updates `batch_summary.csv` and runs the BCR calculator; in JSON mode it aggregates per-script JSON outputs, computes BCR from the aggregated data, and writes a single `ctcc_results_<id>.json`.

2.2 Method Overview

2.2.1 Purpose

1. Parse command-line arguments (e.g. `--no_emissions`, `--capital_only`, `--simple`).
2. Load Primary BCR config from YAML or JSON (based on `CTCC_INPUT_MODE`) and override args with config flags where present.
3. Build `BCRConfig` from args and set env vars (e.g. `CTCC_NO_CONGESTION`, `CTCC_NO_WF_LIABILITY`).
4. Generate or use existing `CTCC_SCENARIO_ID`.
5. Build the list of scripts to run (prerequisites first, then capital costs, then optional modules based on flags; `--capital_only` runs only `weighted_miles`, `build_costs`, `row_costs`, `environmental_mitigation`).
6. Run each script as a subprocess from `scripts/` with the current environment; track successes and failures.
7. If output mode is CSV: run `calculate_and_display_bcr` on `batch_summary.csv`, then add BCR metrics to `batch_summary.csv` via `CSVOutputManager`.

8. If output mode is JSON: `aggregate_json_outputs` loads and merges `json_output_<id>*.json` files, `build_bcr_data_from_json` extracts a flat dict from aggregated results, BCR is computed via `calculate_benefits/calculate_costs/calculate_bcr_metrics`, then `write_final_json_output` writes `ctcc_results_<id>.json` with BCR and summary.
9. Exit with code 1 if any script failed.

2.2.2 Calculation Approach

Orchestration only; no cost/benefit formulas are implemented in `ctcc.py`. Calculations live in the scripts in `scripts/`. Sequencing and mode handling (env vars, script list, BCR aggregation) are the only logic here.

2.2.3 Inputs and Outputs

Inputs:

- Command-line flags: `--norisk`, `--simple`, `--no_wf_liability`, `--no_emissions`, `--no_linelosses`, `--capital_only`, `--no_oandm`, `--no_insurance`, `--no_delay_costs`, `--no_wildfire`, `--no_outages`, `--no_congestion`, `--no_curtailment`.
- Environment variables: `CTCC_INPUT_MODE`, `CTCC_OUTPUT_MODE`, `CTCC_JSON_DATA_FILE`, `CTCC_SCENARIO_ID` (optional; auto-generated if unset).
- Primary BCR config: loaded from YAML or JSON (via `load_primary_bcr_config`) when available; overrides the above flags for which modules run.

Outputs:

- CSV mode: `outputs/batch_summary.csv` (and other CSVs written by scripts), updated with BCR metrics; BCR printed to stdout.
- JSON mode: `outputs/ctcc_results_<scenario_id>.json` containing aggregated costs, benefits, technical parameters, summary, and BCR; intermediate `json_output_<id>*.json` files are removed after merge.
- Exit code 0 if all scripts succeeded, 1 if any failed.

2.2.4 Integration with Other Scripts

`ctcc.py` runs (as subprocesses) and/or imports:

- `scripts/weighted_miles.py`, `build_costs.py`, `row_costs.py`,
`environmental_mitigation.py`, `revenue.py`, `insurance_costs.py`,
`delay_costs.py`, `wildfire_costs.py`, `outage_costs.py`,
`congestion_curtailment_reduction.py`, `energy_losses.py`, `oandm.py`,
`emissions.py`, `line_loss_costs.py` (subset depending on flags).

- `scripts/bcr_calculator.py` — `calculate_and_display_bcr`, `calculate_benefits`, `calculate_costs`, `calculate_bcr_metrics`.
- `scripts/csv_output_manager.py` — `CSVOutputManager`, `BATCH_SUMMARY_FIELDS`.
- `scripts/json_output_manager.py` — `JSONOutputManager` (used in `aggregate_json_outputs`).
- `scripts/json_loaders.py` or `scripts/yaml_loaders.py` — `load_primary_bcr_config` (depending on `CTCC_INPUT_MODE`).

2.3 Key Functions

2.3.1 `run_script(script_name, quiet=False)`

Runs a single Python script from the `scripts/` directory as a subprocess with the current environment.

Parameters:

- `script_name` (str): Name of the script (e.g. `weighted_miles.py`).
- `quiet` (bool): If True, suppress success/failure messages.

Returns: True if the script exited with code 0, False otherwise.

Key logic:

```

1 env = os.environ.copy()
2 result = subprocess.run(
3     [sys.executable, script_name],
4     capture_output=True,
5     text=True,
6     cwd="scripts",
7     env=env,
8 )
9 return result.returncode == 0

```

2.3.2 `build_bcr_data_from_json(json_results)`

Builds a flat dictionary suitable for BCR calculations from the aggregated JSON results (costs and benefits by category).

Parameters: `json_results` (dict): Aggregated results from `JSONOutputManager.get_json_results()`.

Returns: Dict with keys such as `congestion_benefit_pv`, `build_cost_pv`, `delay_cost_pv`, etc., used by `calculate_benefits` and `calculate_costs`.

Key logic:

```

1 costs = json_results.get("costs", {}) or {}
2 benefits = json_results.get("benefits", {}) or {}
3 congestion = benefits.get("congestion_curtailment", {}) or {}
4 build = costs.get("build", {}) or {}
5 # ... extract PV and nominal fields from each category
6 return {
7     "congestion_benefit_pv": congestion.get("congestion_benefit_pv",
8         0) or 0,
9     "build_cost_pv": build.get("total_pv", 0) or 0,
10    # ... all BCR inputs
11}

```

2.3.3 aggregate_json_outputs(scenario_id, output_dir="outputs")

Finds all `json_output-<scenario_id>*.json` files in `output_dir`, loads each into a `JSONOutputManager` aggregator, merges results, and deletes the intermediate files.

Parameters:

- `scenario_id` (str): Scenario ID used in filenames.
- `output_dir` (str): Directory containing the JSON output files.

Returns: The `JSONOutputManager` instance with merged results (call `.get_json_results()` for the dict).

Key logic:

```

1 pattern = os.path.join(output_dir, f"json_output-{{scenario_id}}-*.
2   json")
3 json_files = glob.glob(pattern)
4 aggregator = JSONOutputManager(scenario_id=scenario_id)
5 for json_file in json_files:
6     aggregator.load_from_file(json_file)
7     os.unlink(json_file) # remove intermediate file
8 return aggregator

```

2.3.4 write_final_json_output(aggregator, bcr_results, scenario_id, output_dir, csv_equivalent, summary_override)

Adds BCR metrics to the aggregator, computes summary, and writes `ctcc_results-<scenario_id>.json`.

Key logic:

```

1 if bcr_results:
2     aggregator.add_bcr_metrics(bcr_results)
3 aggregator.calculate_summary()

```

```

4 results = aggregator.get_json_results()
5 if csv_equivalent is not None:
6     results["csv_equivalent"] = csv_equivalent
7 if summary_override is not None:
8     results["summary"] = summary_override
9 output_file = os.path.join(output_dir, f"ctcc_results_{scenario_id}.
    json")
10 with open(output_file, "w") as f:
11     json.dump(results, f, indent=2)
12 return output_file

```

2.3.5 main()

Parses arguments, loads BCR config, sets env vars, builds script list, runs each script via `run_script`, then either (CSV) runs `calculate_and_display_bcr` and updates `batch_summary.csv` or (JSON) aggregates JSON outputs, computes BCR from aggregated data, and calls `write_final_json_output`. Exits with 1 if any script failed.

2.4 Example

Default run (YAML input, CSV output, all modules):

```
1 python ctcc.py
```

JSON input and JSON output with scenario ID `test`:

```

1 export CTCC_INPUT_MODE=json
2 export CTCC_OUTPUT_MODE=json
3 export CTCC_JSON_DATA_FILE=server/json/final_combined.json
4 export CTCC_SCENARIO_ID=test
5 python ctcc.py

```

Capital-only run (weighted miles, build, ROW, environmental mitigation):

```
1 python ctcc.py --capital_only
```

Simple output (only BCR analysis printed):

```
1 python ctcc.py --simple
```

Chapter 3

CTCC YAML Configuration Guide

3.1 Introduction

This document serves as a comprehensive guide to all configuration choices available in the Comprehensive Transmission Cost Calculator (CTCC). The CTCC uses YAML (YAML Ain't Markup Language) configuration files to define project parameters, financial settings, risk parameters, and operational characteristics. The same structure is used when running in JSON input mode: `smart_loaders` dispatches to `yaml_loaders` or `json_loaders` by `CTCC_INPUT_MODE`; parameter names and nesting match this guide.

3.1.1 Purpose

This guide explains:

- What each YAML file configures
- All available parameters and their meanings
- Valid values and typical ranges
- How parameters interact with each other
- How choices affect cost calculations

3.1.2 How to Use This Guide

This guide is organized by YAML file, following the numerical naming convention used in the CTCC. Each section covers:

1. **Purpose:** What the file configures
2. **Parameter Choices:** All configurable parameters
3. **Valid Values:** Ranges, options, or typical values
4. **Meaning:** What each parameter affects

5. **Interactions:** How parameters relate to other files
6. **Examples:** Typical configurations

3.1.3 Project Categories

Many parameters reference *project categories*, which are constructed from core technical details using `calculation_utils.build_category_string` in the format:

```
"{construction_type}/{ac_dc}/{capacity_mw}MW/{conductor_type}/{converter_type}"
```

For example: "Overhead/AC/394MW/Advanced Aluminum/NA" or "Overhead/AC/500MW/Advanced Aluminum Conductor/NA".

For AC projects, `converter_type` is always "NA". For DC projects, it is typically "LCC Converter" or "VSC Converter". Category lookups are used in build costs (10), circuit/resistance (21), O&M conductors/converters (13, 15), and ROW widths (20).

3.1.4 YAML File Inventory

The CTCC `yamls/` directory includes: 01_project_technical_details.yaml, 02_project_physical_details.yaml, 03_financing.yaml, 04_insurance.yaml, 05_delays.yaml, 06_wildfire_costs.yaml, 07_outage_costs.yaml, 09_environmental_mitigation.yaml, 10_project_category_build_costs.yaml, 11_project_row_details.yaml, 12_project_om_vegetation_management.yaml, 13_category_om_conductors.yaml, 14_category_om_structures.yaml, 15_category_om_converters.yaml, 16_emissions_reductions.yaml, 17_congestion_curtailment_reductions.yaml, 19_cost_timing_patterns.yaml, 20_project_category_row_widths.yaml, 21_project_category_circuit_and_resistance_detail.yaml, 22_primary_bcr_config.yaml.

3.2 Core Project Configuration

File: 01_project_technical_details.yaml

3.2.1 Purpose

This file defines the core technical specifications that identify the transmission project. These parameters are used to construct the project category identifier and are included as the first columns in all CSV outputs for scenario comparison.

3.2.2 Parameter Choices

Project Identification

- **name** (string): Project identifier or name

- **Valid Values:** Any string (typically descriptive name)
- **Meaning:** Used for identification in outputs
- **Example:** "Rural Corridor 500MW Overhead"

Construction Type

- **construction_type** (string): Installation method
 - **Valid Values:**
 - * "Overhead" - Traditional overhead transmission lines
 - * "Underground Direct-Buried" - Direct-buried underground cables
 - * "Underground Tunnel" - Underground tunnel installations
 - * "Subsea" - Submarine transmission cables
 - **Meaning:** Determines build costs, ROW requirements, O&M costs, wild-fire/outage risks, and environmental mitigation costs
 - **Interactions:** Affects parameters in files 06 (wildfire), 07 (outage), 09 (environmental), 10-15 (category costs), 20-21 (category technical data)

Current Type

- **ac_dc** (string): Alternating or direct current
 - **Valid Values:** "AC" or "DC"
 - **Meaning:**
 - * "AC": Standard alternating current transmission
 - * "DC": Direct current transmission (requires converters)
 - **Interactions:** If "DC", must specify **converter_type** and **number_of_converters**

Capacity

- **capacity_mw** (float): Rated capacity in megawatts
 - **Valid Values:** Positive numbers (typical: 140, 329, 394, 460, 500, 657, 1000, 1500 MW)
 - **Meaning:** Determines project category, affects build costs, line losses, and benefit calculations
 - **Interactions:** Must match a valid capacity in category files (10, 13-15, 20-21)

Conductor Type

- **conductor_type** (string): Conductor technology
 - **Valid Values:**
 - * "Standard Aluminum Conductor" - Traditional ACSR conductors
 - * "Advanced Aluminum Conductor" - Advanced composite core conductors (ACCC)
 - **Meaning:** Affects conductor costs, resistance (line losses), and O&M costs
 - **Interactions:** Used in project category construction; affects files 10 (build costs), 13 (conductor O&M), 21 (resistance)

Converter Configuration (DC Only)

- **converter_type** (string): Converter technology (DC projects only)
 - **Valid Values:**
 - * "LCC Converter" - Line Commutated Converter
 - * "VSC Converter" - Voltage Source Converter
 - **Meaning:** Determines converter costs and O&M
 - **Note:** For AC projects, this parameter is set to "NA" automatically
- **number_of_converters** (integer): Number of converter stations
 - **Valid Values:** 1 or 2
 - **Meaning:** Typically 2 (one at each end) for point-to-point DC lines
 - **Interactions:** Multiplies converter costs from category files

Line Utilization

- **line_utilization** (float): Average capacity factor
 - **Valid Values:** Range [0, 1] (typical: 0.5 to 0.9)
 - **Meaning:** Fraction of rated capacity used on average; affects line loss calculations
 - **Example:** 0.70 = 70% average utilization

Electricity Price

- **baseline_electricity_price_per_mwh** (float): Base electricity price
 - **Valid Values:** Positive numbers (typical: 30-100 \$/MWh)
 - **Meaning:** Used for line loss cost/benefit calculations and congestion/curtailment benefit valuations
 - **Interactions:** Affects files 16 (emissions), 17 (congestion), 18 (curtailment), line loss calculations

Social Discount Rate

- **social_discount_rate** (float): Public sector discount rate
 - **Valid Values:** Typically 0.02 to 0.07 (2% to 7%)
 - **Meaning:** Used for present value calculations in public benefit-cost analysis
 - **Reference:** U.S. Office of Management and Budget, Circular A-4 (2003, reaffirmed 2023) recommends 3% for intergenerational projects
 - **Interactions:** Used in files 06 (wildfire), 07 (outage), and all present value calculations

Reconductoring Configuration

- **reconductoring** (boolean): Whether this is a reconductoring project
 - **Valid Values:** True or False
 - **Meaning:**
 - * True: Upgrading existing line (lower build costs, line loss benefits)
 - * False: New greenfield project (full build costs, line loss costs)
 - **Interactions:** If True, must specify old line parameters below
- **old_capacity_mw** (float): Original line capacity (reconductoring only)
 - **Valid Values:** Positive numbers, typically less than **capacity_mw**
 - **Meaning:** Original line capacity before upgrade
- **old_conductor_type** (string): Original conductor type (reconductoring only)
 - **Valid Values:** "Standard Aluminum Conductor" or "Advanced Aluminum Conductor"
 - **Meaning:** Used to calculate line loss benefits (reduction in losses)
- **old_ac_dc** (string): Original current type (reconductoring only)
 - **Valid Values:** "AC" or "DC"
 - **Meaning:** Used for benefit calculations

Timeline Parameters

- **construction_years** (float): Years to complete construction
 - **Valid Values:** Positive numbers (typical: 1-5 years)
 - **Meaning:** Duration of construction period; affects cost timing and AFUDC calculations
- **delay_years** (float): Years of delay before construction

- **Valid Values:** Non-negative numbers (typical: 0-5 years)
- **Meaning:** Permitting/regulatory delay period; affects delay costs, ROW holding costs, and AFUDC eligibility
- **project_lifetime** (integer): Operational lifetime in years
 - **Valid Values:** Positive integers (typical: 40-60 years)
 - **Meaning:** Duration of operational period; affects present value calculations for all recurring costs and benefits

3.2.3 Example Configuration

```

1 project:
2   name: "Rural Corridor 500MW"
3   construction_type: "Overhead"
4   ac_dc: "AC"
5   number_of_converters: 2
6   capacity_mw: 500
7   conductor_type: "Advanced Aluminum Conductor"
8   converter_type: "NA"
9   line_utilization: 0.70
10  baseline_electricity_price_per_mwh: 50.0
11  social_discount_rate: 0.03
12  reconductoring: False
13  old_capacity_mw: 0
14  old_conductor_type: "Standard Aluminum Conductor"
15  old_ac_dc: "AC"
16
17 timeline:
18   construction_years: 2
19   delay_years: 1.5
20   project_lifetime: 50

```

3.3 Physical Project Details

File: 02_project_physical_details.yaml

3.3.1 Purpose

This file defines the physical route characteristics, including terrain distribution and terrain-specific cost multipliers. The terrain distribution determines how many miles of the line pass through each terrain type, which affects build costs, O&M costs, wildfire risks, outage risks, and environmental mitigation costs.

3.3.2 Parameter Choices

Terrain Distribution

- **terrain_miles** (dictionary): Miles in each terrain type
 - **Valid Terrain Types:**
 - * **forested** - Forested areas
 - * **scrubbed_flat** - Scrubland/flat terrain
 - * **wetland** - Wetlands
 - * **farmland** - Agricultural land
 - * **desert_barren** - Desert/barren land
 - * **urban** - Urban areas
 - * **rolling_hills** - Rolling hills
 - * **mountain** - Mountainous terrain
 - * **subsea** - Subsea (for subsea projects)
 - **Valid Values:** Non-negative numbers (miles)
 - **Meaning:** The sum of all terrain miles equals total line length
 - **Interactions:** Affects files 06 (wildfire ignition rates), 07 (outage rates/durations), 09 (environmental mitigation), 12 (vegetation management), 14 (structure O&M)

Terrain Multipliers

- **terrain_multipliers** (dictionary): Cost multipliers by terrain
 - **Valid Values:** Positive numbers (typical: 1.0 to 2.5)
 - **Meaning:** Multiplier applied to base build costs for each terrain type
 - * **forested:** 2.25 (highest cost - difficult access)
 - * **scrubbed_flat:** 1.0 (baseline)
 - * **wetland:** 1.2 (moderate increase)
 - * **farmland:** 1.0 (baseline)
 - * **desert_barren:** 1.05 (slight increase)
 - * **urban:** 1.59 (higher - urban constraints)
 - * **rolling_hills:** 1.4 (moderate increase)
 - * **mountain:** 1.75 (high - difficult terrain)
 - * **subsea:** 1.0 (baseline for subsea)
 - **Interactions:** Applied to build costs in file 10

3.3.3 Total Line Length

The total line length is automatically calculated as the sum of all `terrain_miles` values. This total is used throughout the CTCC for:

- Build cost calculations (variable costs per mile)
- ROW cost calculations (acres = miles × ROW width)
- O&M cost calculations (per mile per year)
- Risk calculations (per mile rates)

3.3.4 Example Configuration

```
1 terrain:
2   terrain_miles:
3     forested: 25
4     scrubbed_flat: 35
5     wetland: 5
6     farmland: 20
7     desert_barren: 0
8     urban: 0
9     rolling_hills: 15
10    mountain: 0
11    subsea: 0
12
13   terrain_multipliers:
14     forested: 2.25
15     scrubbed_flat: 1.0
16     wetland: 1.2
17     farmland: 1.0
18     desert_barren: 1.05
19     urban: 1.59
20     rolling_hills: 1.4
21     mountain: 1.75
22     subsea: 1.0
```

Total Line Length: 100 miles

3.4 Financing Parameters

File: 03_financing.yaml

3.4.1 Purpose

This file defines financial parameters including inflation, discount rates, contingencies, capital structure, and AFUDC (Allowance for Funds Used During Construction) configuration. These parameters affect present value calculations, cost escalation, and regulatory accounting.

3.4.2 Parameter Choices

Inflation and Base Year

- **inflation_rate** (float): Annual inflation rate
 - **Valid Values:** Typically 0.02 to 0.05 (2% to 5%)
 - **Meaning:** Used to escalate costs from base year to actual year
- **base_year** (integer): Reference year for costs
 - **Valid Values:** Year (e.g., 2025)
 - **Meaning:** All costs are expressed in base year dollars

Discount Rates

- **wacc_nominal** (float): Weighted Average Cost of Capital (nominal)
 - **Valid Values:** Typically 0.06 to 0.12 (6% to 12%)
 - **Meaning:** Private sector discount rate; used for AFUDC calculations if capital structure not specified
 - **Note:** Real WACC = $(1 + \text{nominal WACC}) / (1 + \text{inflation}) - 1$
- **social_discount_rate** (float): Public sector discount rate
 - **Valid Values:** Typically 0.02 to 0.07 (2% to 7%)
 - **Meaning:** For public benefit-cost analysis; can differ from value in file 01 (this one takes precedence for financial calculations)
 - **Reference:** OMB Circular A-4 recommends 3% for intergenerational projects

Contingencies

- **contingencies** (dictionary): Contingency percentages
 - **conductor_contingency** (float): Range [0, 1] (typical: 0.10 = 10%)
 - **structure_contingency** (float): Range [0, 1] (typical: 0.10 = 10%)
 - **converter_contingency** (float): Range [0, 1] (typical: 0.10 = 10%)
 - **Meaning:** Applied to base build costs to account for uncertainty

Capital Structure (Optional)

- **capital_structure** (dictionary, optional): Equity/debt breakdown
 - **equity_percent** (float): Range [0, 1] (typical: 0.50 = 50%)
 - **debt_percent** (float): Range [0, 1] (must sum to 1.0 with equity)
 - **cost_of_equity** (float): Return on equity (typical: 0.10 = 10%)
 - **cost_of_debt** (float): Interest rate on debt (typical: 0.05 = 5%)
 - **Meaning**: If specified, used to calculate AFUDC rate; otherwise WACC used as proxy
 - **Note**: $WACC = \text{equity_percent} \times \text{cost_of_equity} + \text{debt_percent} \times \text{cost_of_debt}$

AFUDC Configuration

- **afudc** (dictionary): AFUDC settings
 - **apply_afudc** (boolean): Enable/disable AFUDC calculations
 - * **Valid Values**: true or false
 - * **Meaning**: Toggles AFUDC capitalization on/off
 - **delay_period_active_work** (boolean): AFUDC during delay period
 - * **Valid Values**: true or false
 - * **Meaning**:
 - true: AFUDC accrues if active work continues (permitting, design, CWIP-eligible costs)
 - false: AFUDC suspended during delay (conservative default per FERC)

3.4.3 Example Configuration

```
1 financial:
2   inflation_rate: 0.03
3   base_year: 2025
4   wacc_nominal: 0.08
5   social_discount_rate: 0.03
6
7   contingencies:
8     conductor_contingency: 0.10
9     structure_contingency: 0.10
10    converter_contingency: 0.10
11
12   capital_structure:
13     equity_percent: 0.50
14     debt_percent: 0.50
15     cost_of_equity: 0.10
```

```

16     cost_of_debt: 0.05
17
18     afudc:
19         apply_afudc: true
20         delay_period_active_work: false

```

3.5 Insurance Configuration

File: 04_insurance.yaml

3.5.1 Purpose

This file configures operational insurance costs for transmission line assets. Insurance premiums are calculated as a percentage of insurable asset value (conductors, structures, converters).

3.5.2 Parameter Choices

Default Premium Rate

- **premium_rate** (float): Annual insurance premium rate
 - **Valid Values:** Typically 0.001 to 0.005 (0.1% to 0.5%)
 - **Meaning:** Applied to insurable asset value to calculate annual premium
 - **Example:** 0.002 = 0.2% annual premium

Insurable Components

- **insurable_components** (dictionary): Which assets are insured
 - **conductors** (boolean): Include conductor costs
 - **structures** (boolean): Include structure costs
 - **converters** (boolean): Include converter costs
 - **Meaning:** Determines total insurable value

Construction Type-Specific Premiums (Optional)

- **premium_by_construction_type** (dictionary, optional): Premium rates by construction type
 - **overhead** (float): Premium for overhead lines (typical: 0.002 = 0.2%)
 - **underground** (float): Premium for underground lines (typical: 0.0015 = 0.15%)
 - **subsea** (float): Premium for subsea lines (typical: 0.0025 = 0.25%)
 - **Meaning:** If specified, overrides default **premium_rate** for that construction type

3.5.3 Example Configuration

```
1 insurance:
2   premium_rate: 0.002
3
4   insurable_components:
5     conductors: true
6     structures: true
7     converters: true
8
9   premium_by_construction_type:
10    overhead: 0.002
11    underground: 0.0015
12    subsea: 0.0025
```

3.6 Delay Costs

File: 05_delays.yaml

3.6.1 Purpose

This file defines annual costs incurred during project delays (permitting, regulatory, legal processes). These costs occur during the `delay_years` period defined in file 01.

3.6.2 Parameter Choices

All parameters are annual costs in dollars per year:

- **legal** (float): Legal expenses (\$/year)
- **admin** (float): Administrative costs (\$/year)
- **labor** (float): Labor costs (\$/year)
- **material_and_equipment** (float): Material/equipment costs (\$/year)
- **regulatory** (float): Regulatory compliance costs (\$/year)
- **public_relations** (float): Public relations costs (\$/year)
- **project_management** (float): Project management costs (\$/year)
- **miscellaneous** (float): Other delay-related costs (\$/year)

3.6.3 Total Annual Delay Cost

The total annual delay cost is the sum of all components. Total delay cost = annual cost $\times$ `delay_years` (from file 01).

3.6.4 Example Configuration

```
1 annual_delay_costs:
2   legal: 200000
3   admin: 100000
4   labor: 1000000
5   material_and_equipment: 800000
6   regulatory: 300000
7   public_relations: 50000
8   project_management: 400000
9   miscellaneous: 100000
```

Total Annual Delay Cost: \$2,950,000/year

3.7 Wildfire Risk Parameters

File: 06_wildfire_costs.yaml

3.7.1 Purpose

This file configures wildfire risk parameters, including event severity, ignition rates by terrain and construction type, and discount rate selection. Wildfire costs are calculated as expected annual loss = ignition rate $\times$ severity per event.

3.7.2 Parameter Choices

Event Severity

- **severity_per_event** (float): Mean **uninsured** loss per wildfire event
 - **Valid Values:** Large numbers (typical: \$1B to \$10B)
 - **Meaning:** Expected residual loss not covered by liability insurance—e.g. deductibles, amounts above the liability limit, exclusions, or societal/external costs. Wildfire liability insurance (file 04) covers the insured portion; severity here should exclude that to avoid double-counting.
 - **Example:** 5000000000 = \$5 billion uninsured loss per event

Risk Growth

- **risk_growth_rate** (float, optional): Annual increase in ignition probability
 - **Valid Values:** Typically 0.0 to 0.05 (0% to 5%)
 - **Meaning:** Accounts for increasing wildfire risk over time (climate change, etc.)
 - **Default:** 0.02 = 2% annual increase

Discount Rate Configuration

- **discount_rate_source** (string): Source of discount rate
 - **Valid Values:**
 - * "social" - Use social discount rate from file 03 (societal perspective)
 - * "wacc_real" - Use real WACC from file 03 (utility perspective)

Base Ignition Rates by Terrain

- **ignition_rates_by_terrain** (dictionary): Base ignition rates (events per mile per year)
 - **Valid Values:** Small positive numbers (typical: 0.00001 to 0.0001)
 - **Meaning:** Baseline ignition rate for overhead lines in each terrain
 - **Typical Values:**
 - * forested: 0.0001 (highest risk)
 - * mountain: 0.0001
 - * scrubbed_flat: 0.00005
 - * rolling_hills: 0.00006
 - * desert_barren: 0.00008
 - * farmland: 0.00003
 - * urban: 0.00002
 - * wetland: 0.00001 (lowest risk)
 - * subsea: 0.0 (no risk)

Ignition Rate Multipliers by Construction Type

- **ignition_rate_multiplier** (dictionary): Multipliers by construction type
 - **overhead** (float): 1.0 (baseline)
 - **underground** (float): 0.02 (98% reduction)
 - **subsea** (float): 0.0 (no risk)
 - **Meaning:** Effective rate = base terrain rate $\times$ multiplier

3.7.3 Example Configuration

```
1 wildfire:
2   severity_per_event: 5000000000
3   risk_growth_rate: 0.02
4   discount_rate_source: "social"
5
6   ignition_rates_by_terrain:
7     forested: 0.0001
```

```

8   scrubbed_flat: 0.00005
9   wetland: 0.00001
10  farmland: 0.00003
11  desert_barren: 0.00008
12  urban: 0.00002
13  rolling_hills: 0.00006
14  mountain: 0.0001
15  subsea: 0.0
16
17  ignition_rate_multiplier:
18    overhead: 1.0
19    underground: 0.02
20    subsea: 0.0

```

3.8 Outage Risk Parameters

File: 07_outage_costs.yaml

3.8.1 Purpose

This file configures outage risk parameters, including outage rates by construction type and terrain, outage durations, value of lost load (VoLL), and capacity at risk factors. Outage costs are calculated as expected annual cost = outage rate $\times$ duration $\times$ capacity at risk $\times$ VoLL.

3.8.2 Parameter Choices

Risk Growth

- **risk_growth_rate** (float): Annual increase in outage rates
 - **Valid Values:** Typically 0.0 (no growth)
 - **Meaning:** Accounts for increasing outage risk over time

Discount Rate

- **discount_rate_type** (string): Discount rate source
 - **Valid Values:** "social" or "wacc_real"

Capacity at Risk

- **capacity_at_risk_factor** (float): Fraction of capacity at risk during outage
 - **Valid Values:** Range [0, 1] (typical: 0.5 to 1.0)
 - **Meaning:**

- * 1.0 = Radial line (100% of capacity lost during outage)
- * >1.0 = Meshed/redundant system (partial capacity loss)

Value of Lost Load (VoLL)

- **value_of_lost_load** (dictionary): Tiered VoLL by outage duration
 - **tiers** (list): List of tier definitions
 - * **max_hours** (float): Maximum hours for this tier (use `.inf` for infinity)
 - * **value_per_mwh** (float): VoLL in \$/MWh for this tier
 - **Typical Tiers:**
 - * Tier 1: 0-4 hours, \$5,000/MWh (short outages)
 - * Tier 2: 4-24 hours, \$10,000/MWh (medium outages)
 - * Tier 3: 24+ hours, \$15,000/MWh (long outages)

Outage Duration by Terrain

- **outage_duration_by_terrain** (dictionary): Hours per outage event by terrain
 - **Valid Values:** Positive numbers (hours)
 - **Typical Values:**
 - * **urban:** 2 hours (fastest repair)
 - * **farmland:** 3 hours
 - * **scrubbed_flat:** 4 hours
 - * **desert_barren:** 5 hours
 - * **wetland:** 6 hours
 - * **rolling_hills:** 6 hours
 - * **forested:** 8 hours
 - * **mountain:** 12 hours
 - * **subsea:** 240 hours (10 days - very slow repair)

Duration Multipliers by Construction Type

- **outage_duration_multiplier** (dictionary): Multipliers by construction type
 - **overhead** (float): 1.0 (baseline)
 - **underground** (float): 2.0 (longer repair times)
 - **subsea** (float): 3.0 (very long repair times)

Outage Rates

- **outage_rates** (dictionary): Outages per mile per year by construction type and terrain
 - **Structure:** `outage_rates[construction_type][terrain]`
 - **Valid Values:** Positive numbers (typical: 0.001 to 0.1 outages/mile/year)
 - **Meaning:** Direct specification of outage frequency (no $f \times p$ split)
 - **Note:** Must specify for all construction types and all terrain types

3.8.3 Example Configuration

```
1 outage:
2   risk_growth_rate: 0.0
3   discount_rate_type: "social"
4   capacity_at_risk_factor: 1.0
5
6   value_of_lost_load:
7     tiers:
8       - max_hours: 4
9         value_per_mwh: 5000
10      - max_hours: 24
11        value_per_mwh: 10000
12      - max_hours: .inf
13        value_per_mwh: 15000
14
15   outage_duration_by_terrain:
16     forested: 8
17     scrubbed_flat: 4
18     wetland: 6
19     farmland: 3
20     desert_barren: 5
21     urban: 2
22     rolling_hills: 6
23     mountain: 12
24     subsea: 240
25
26   outage_duration_multiplier:
27     overhead: 1.0
28     underground: 2.0
29     subsea: 3.0
30
31   outage_rates:
32     overhead:
33       forested: 0.04
34       scrubbed_flat: 0.021
35       # ... (all terrain types)
```

```

36     underground:
37         forested: 0.005
38         # ... (all terrain types)
39     subsea:
40         # All terrains: 0.00475

```

3.9 Environmental Mitigation

File: 09_environmental_mitigation.yaml

3.9.1 Purpose

This file configures environmental mitigation costs, including base restoration costs per acre by construction type and terrain, credit purchase costs (off-site mitigation), and mitigation ratios (acres required per acre impacted).

3.9.2 Parameter Choices

Mitigation Uplift Factor

- **mitigation_uplift_factor** (float): Width multiplier for mitigation area
 - **Valid Values:** Typically 1.0 to 1.5
 - **Meaning:** Accounts for construction width beyond ROW width (TCE - Temporary Construction Easement)
 - **Example:** 1.25 = mitigation area is 25% larger than ROW area

Base Mitigation Costs

- **base_mitigation_cost_per_acre** (dictionary): Restoration costs by construction type and terrain
 - **Structure:** `base_mitigation_cost_per_acre[construction_type][terrain]`
 - **Valid Values:** Positive numbers (\$/acre)
 - **Construction Types:**
 - * overhead
 - * underground_direct_buried
 - * underground_tunnel
 - * subsea
 - **Typical Values:** \$800 to \$10,000 per acre
 - **Special Values:**
 - * `linear_corridor_default`: Default for unknown terrain (tunnel)

- * `shaft_site_default`: Default for pad/disturbed sites (tunnel)
- * `seabed_corridor_default`: 0 for subsea (passive recovery)
- * `landfall_default`: Onshore landfall costs (subsea)

Credit Purchase Costs

- **credit_cost_per_acre** (dictionary): Off-site mitigation credit costs
 - **wetlands** (dictionary): Wetland credit costs by subtype
 - * `palustrine`: \$50,000/acre
 - * `estuarine`: \$100,000/acre
 - * `other`: \$25,000/acre
 - **habitat** (dictionary): Habitat credit costs. May include per-terrain keys and a default fallback for any terrain not listed.
 - * `default`: \$30,000/acre (fallback)
 - * `forested`, `scrubbed_flat`, `desert_barren`, `rolling_hills`, `mountain`: optional \$/acre per habitat terrain

Mitigation Ratios

- **credit_ratios** (dictionary): Acres required per acre impacted
 - **wetlands** (dictionary):
 - * `palustrine`: 1.5 (1.5 acres required per acre impacted)
 - * `estuarine`: 2.5
 - * `other`: 1.0
 - **habitat** (dictionary): Per-terrain keys and a `default` fallback.
 - * `default`: 1.0 (fallback)
 - * `forested`, `scrubbed_flat`, `desert_barren`, `rolling_hills`, `mountain`: optional ratio per habitat terrain

3.9.3 Example Configuration

```

1 environmental_mitigation:
2   mitigation_uplift_factor: 1.25
3
4   base_mitigation_cost_per_acre:
5     overhead:
6       forested: 5000
7       scrubbed_flat: 1000
8       wetland: 5000
9       farmland: 800
10    # ... (all terrain types)

```

```

11     underground_direct_buried:
12         # ... (all terrain types)
13     underground_tunnel:
14         linear_corridor_default: 1500
15         shaft_site_default: 1000
16     subsea:
17         seabed_corridor_default: 0
18         landfall_default: 1000
19
20     credit_cost_per_acre:
21         wetlands:
22             palustrine: 50000
23             estuarine: 100000
24             other: 25000
25         habitat:
26             default: 30000
27             forested: 30000
28             mountain: 30000
29     # ... (scrubbed_flat, desert_barren, rolling_hills optional)
30
31     credit_ratios:
32         wetlands:
33             palustrine: 1.5
34             estuarine: 2.5
35             other: 1.0
36         habitat:
37             default: 1.0
38             forested: 1.0
39             mountain: 1.0
40     # ... (per-terrain optional)

```

3.10 Right-of-Way Configuration

File: 11_project_row_details.yaml

3.10.1 Purpose

This file configures right-of-way (ROW) acquisition and management costs using a zone-based approach. Each zone represents a segment of the line with different land values and rental costs.

3.10.2 Parameter Choices

Zone Configuration

The file defines up to 15 zones (zone_1 through zone_15), each with:

- **miles** (float): Miles of line in this zone
 - **Valid Values:** Non-negative numbers
 - **Meaning:** Sum of all zone miles should equal total line length
- **acquisition_cost** (float): One-time acquisition cost per acre
 - **Valid Values:** Positive numbers (\$/acre)
 - **Meaning:** Cost to acquire ROW land
 - **Typical Values:** \$5,000 to \$20,000 per acre
- **rent_cost** (float): Annual rental cost per acre
 - **Valid Values:** Positive numbers (\$/acre/year)
 - **Meaning:** Annual rent if ROW is leased rather than owned
 - **Typical Values:** \$9 to \$31,116 per acre/year (varies significantly by zone)
- **hold_cost** (float): Annual holding cost per acre during delay period
 - **Valid Values:** Positive numbers (\$/acre)
 - **Meaning:** Cost to hold/secure ROW during permitting delay
 - **Typical Values:** Often set to $0.1 \times \text{rent_cost}$ per year
 - **Note:** Applied during `delay_years` period

3.10.3 Zone Definitions

Zones typically represent increasing land values:

- **Zones 1-3:** Rural/low-value land (\$9-\$35/acre/year rent)
- **Zones 4-6:** Moderate-value land (\$53-\$106/acre/year rent)
- **Zones 7-9:** High-value land (\$148-\$507/acre/year rent)
- **Zones 10-15:** Very high-value land (\$1,556-\$31,116/acre/year rent)

3.10.4 Example Configuration

```

1 right_of_way:
2   zone_1:
3     miles: 25
4     acquisition_cost: 13569
5     rent_cost: 9
6     hold_cost: 9
7   zone_2:
8     miles: 25

```

```

9     acquisition_cost: 13569
10     rent_cost: 18
11     hold_cost: 18
12 zone_3:
13     miles: 30
14     acquisition_cost: 13569
15     rent_cost: 35
16     hold_cost: 35
17 # ... (zones 4-15, with miles: 0 if not used)

```

3.11 O&M Vegetation Management

File: 12_project_om_vegetation_management.yaml

3.11.1 Purpose

This file configures operations and maintenance costs for vegetation management, which applies only to overhead lines. Underground and subsea lines have zero vegetation management costs.

3.11.2 Parameter Choices

- **vegetation_management_om_costs** (dictionary): Costs by construction type and terrain
 - **Structure:** `vegetation_management_om_costs[construction_type][terrain]`
 - **Valid Values:** Non-negative numbers (\$/mile/year)
 - **Construction Types:**
 - * Overhead
 - * Underground Direct Buried
 - * Underground Tunnel
 - * Subsea
 - **Typical Values for Overhead:**
 - * `scrubbed_flat`: \$10,000/mile/year (highest - dense vegetation)
 - * `forested`: \$8,000/mile/year
 - * `mountain`: \$9,000/mile/year
 - * `wetland`: \$7,000/mile/year
 - * `rolling_hills`: \$3,000/mile/year
 - * `urban`: \$5,000/mile/year
 - * `farmland`: \$500/mile/year
 - * `desert_barren`: \$300/mile/year (lowest)
 - **Note:** All non-overhead construction types have 0 cost

3.11.3 Example Configuration

```
1 vegetation_management_om_costs:
2   Overhead:
3     forested: 8000
4     scrubbed_flat: 10000
5     wetland: 7000
6     farmland: 500
7     desert_barren: 300
8     urban: 5000
9     rolling_hills: 3000
10    mountain: 9000
11    subsea: 0
12   Underground Direct Buried:
13     # All terrains: 0
14   Underground Tunnel:
15     # All terrains: 0
16   Subsea:
17     # All terrains: 0
```

3.12 Category-Specific O&M Costs

3.12.1 Conductor O&M

File: 13_category_om_conductors.yaml

This file defines O&M costs for conductors organized by project category. Each category entry includes:

- **variable_cost_per_mile_year** (float): Annual O&M cost per mile (\$/mile/year)
- **Typical Values:** \$500 to \$2,000 per mile per year
- **Note:** Costs vary by capacity and conductor type

3.12.2 Structure O&M

File: 14_category_om_structures.yaml

This file defines O&M costs for transmission structures. Unlike other category files, structure O&M is organized by installation method only (not by individual categories):

- **Overhead:** Structure density and cost per structure
 - **structures_per_mile_[terrain]** (float): Structures per mile by terrain type
 - **cost_per_structure_per_year** (float): Annual maintenance cost per structure (\$/structure/year)
- **Underground/Subsea:** Variable cost per mile per year (typically 0)

3.12.3 Converter O&M

File: 15_category_om_converters.yaml

This file defines O&M costs for converters (DC projects only), organized by project category. Each category entry includes:

- **converter_om_cost_per_mile_year** (float): Annual O&M cost per mile (\$/mile/year)
- **Note:** Only applies to DC projects; AC projects have "NA" converter type

3.13 Emissions Reductions

File: 16_emissions_reductions.yaml

3.13.1 Purpose

This file configures environmental benefits from emissions reductions associated with line losses. When transmission lines lose energy, that energy must be compensated by additional generation, which has associated emissions. This file defines the energy source mix, emission intensities, and societal costs of pollutants.

3.13.2 Parameter Choices

Compensation Percentage

- **compensation_percent** (float): Fraction of losses compensated by additional generation
 - **Valid Values:** Range [0, 1] (typical: 0.9 = 90%)
 - **Meaning:** Not all losses require compensation (some are absorbed by system)

Energy Source Mix

- **energy_source_mix** (dictionary): Generation mix percentages and growth rates
 - **Structure:** For each energy source (coal, oil, natural_gas, solar, wind, hydro, nuclear, other):
 - * **percentage** (float): Initial percentage in mix (will be normalized)
 - * **rate_of_change** (float): Annual growth/decay rate (typical: -0.05 to 0.05)
 - **Note:** Percentages are normalized to sum to 100%
 - **Typical Values:**
 - * Coal: 10%, rate: -0.05 (declining)
 - * Natural Gas: 38%, rate: 0.0 (stable)
 - * Wind: 26%, rate: 0.04 (growing)
 - * Solar: 10%, rate: 0.05 (growing)

- * Hydro: 5%, rate: 0.0 (stable)
- * Nuclear: 5%, rate: 0.0 (stable)

Emission Intensities

- **emission_intensities** (dictionary): Pollutant emissions per MWh by energy source
 - **co2_intensity_kg_per_mwh** (dictionary): CO2 emissions (kg/MWh)
 - * Coal: 820 kg/MWh
 - * Oil: 650 kg/MWh
 - * Natural Gas: 450 kg/MWh
 - * Solar: 48 kg/MWh
 - * Wind: 12 kg/MWh
 - * Hydro: 4 kg/MWh
 - * Nuclear: 12 kg/MWh
 - **sox_intensity_kg_per_mwh** (dictionary): SOx emissions (kg/MWh)
 - **nox_intensity_kg_per_mwh** (dictionary): NOx emissions (kg/MWh)

Societal Costs

- **societal_costs_per_kg** (dictionary): Social cost of pollutants
 - **co2_cost_per_kg** (float): Social cost of carbon (\$/kg CO2)
 - * **Typical:** \$0.051/kg = \$51/tonne CO2
 - **sox_cost_per_kg** (float): Social cost of SOx (\$/kg SOx)
 - * **Typical:** \$6.2/kg
 - **nox_cost_per_kg** (float): Social cost of NOx (\$/kg NOx)
 - * **Typical:** \$5.0/kg

3.13.3 Example Configuration

```

1 emissions_reductions:
2   compensation_percent: 0.9
3
4   energy_source_mix:
5     coal:
6       percentage: 10
7       rate_of_change: -0.05
8     natural_gas:
9       percentage: 38
10      rate_of_change: 0.0
11     wind:
12      percentage: 26

```

```

13     rate_of_change: 0.04
14     solar:
15         percentage: 10
16         rate_of_change: 0.05
17     # ... (other sources)
18
19     emission_intensities:
20         co2_intensity_kg_per_mwh:
21             coal: 820
22             natural_gas: 450
23             wind: 12
24         # ... (other sources)
25         # ... (sox, nox)
26
27     societal_costs_per_kg:
28         co2_cost_per_kg: 0.051
29         sox_cost_per_kg: 6.2
30         nox_cost_per_kg: 5.0

```

3.14 Congestion Reductions

File: 17_congestion_reductions.yaml

3.14.1 Purpose

This file configures cost savings from reduced transmission congestion. Congestion occurs when transmission lines are at capacity and cannot deliver all desired power, leading to price differences between regions. This file defines both greenfield (new line) and reconductoring congestion reduction parameters.

3.14.2 Parameter Choices

Greenfield Congestion Reductions

- **greenfield_congestion_reductions** (dictionary):
 - **constraints** (dictionary):
 - * **flow_factor** (float): Deliverability factor [0, 1]
 - **Valid Values:** Range [0, 1] (typical: 0.6)
 - **Meaning:** Fraction of line capacity that effectively relieves the constraint
 - **Formula:** $\Delta C_{eff} = S \times \text{capacity}$
 - * **binding_hours** (integer): Hours per year line is at capacity
 - **Valid Values:** Non-negative integers (typical: 500-2000 hours/year)
 - **Meaning:** Number of hours when the constraint is binding

- * **average_exceedance** (float): Average MW exceedance during binding hours
 - **Valid Values:** Non-negative numbers (MW)
 - **Meaning:** Average amount by which demand exceeds capacity during binding hours
 - **Formula:** Baseline congestion energy = $H_b \times X$ MWh/year
- * **near_binding_hours** (integer): Hours per year line is near capacity
 - **Valid Values:** Non-negative integers
 - **Meaning:** Hours when constraint is nearly binding
- * **near_average_exceedance** (float): Average MW exceedance during near-binding hours
 - **Valid Values:** Non-negative numbers (MW)
 - **Meaning:** Relief during near-binding hours is capped by this value; $E_{near} = \text{near_binding_hours} \times \min(\Delta C_{eff}, \text{near_average_exceedance})$ (MWh/year), consistent with binding hours.
- * **saturation_factor** (float): System saturation factor
 - **Valid Values:** Non-negative numbers (typical: 0.0 to 0.3)
 - **Meaning:** 0.0 = Radial line (full benefit); ≥ 0.0 = Meshed system (diminishing returns).
- **costs** (dictionary):
 - * **average_congestion_price** (float): Average congestion price (\$/MWh)
 - **Valid Values:** Positive numbers (typical: 10-50 \$/MWh)
 - **Meaning:** Price difference between constrained and unconstrained regions

Reconductoring Congestion Reductions

- **reconductoring_congestion_reductions** (dictionary): Similar structure to greenfield, but:
 - **Meaning:** $\Delta C_{eff} = \Delta C_{eff,new} - \Delta C_{eff,old}$ (incremental benefit)
 - **hot_hour_weights** (float): Fraction of hours at or near Maximum Operating Temperature (MOT)
 - * **Valid Values:** Range [0, 1]
 - * **Meaning:** Accounts for thermal constraints in reconductoring

3.14.3 Example Configuration

```

1 greenfield_congestion_reductions:
2   constraints:
3     flow_factor: 0.6
4     binding_hours: 1200
5     average_exceedance: 150
6     near_binding_hours: 800

```

```

7     near_average_exceedance: 0
8     saturation_factor: 0.2
9     costs:
10        average_congestion_price: 20
11
12 reconductoring_congestion_reductions:
13     constraints:
14        binding_hours: 0
15        average_exceedance: 0
16        near_binding_hours: 0
17        near_average_exceedance: 0
18        saturation_factor: 0
19        hot_hour_weights: 0

```

3.15 Curtailment Reductions

File: 18_curtailment_reductions.yaml

3.15.1 Purpose

This file configures cost savings from reduced renewable energy curtailment. Curtailment occurs when renewable generators must reduce output because transmission capacity is insufficient to deliver power to demand centers.

3.15.2 Parameter Choices

- **curtailment_hours_total** (integer): Hours per year with curtailment
 - **Valid Values:** Non-negative integers (set to 0 if not applicable)
 - **Meaning:** Total hours per year when curtailment occurs on the exporting side of the constraint
- **average_curtailment_mw** (float): Average curtailed MW during curtailment hours
 - **Valid Values:** Non-negative numbers (MW)
 - **Meaning:** Average amount of renewable generation curtailed
 - **Note:** Leave at 0 if using total MWh instead
- **average_curtailment_price** (float): Average curtailment price (\$/MWh)
 - **Valid Values:** Positive numbers (typical: 20-40 \$/MWh)
 - **Meaning:** PPA proxy or avoided cost of curtailed renewable energy
- **curtailment_saturation_factor** (float): Saturation factor [0, 1]
 - **Valid Values:** Range [0, 1] (typical: 0.1)
 - **Meaning:** Accounts for diminishing returns in meshed systems

3.15.3 Example Configuration

```
1 curtailment_reductions:
2   curtailment_hours_total: 300
3   average_curtailment_mw: 60
4   average_curtailment_price: 25
5   curtailment_saturation_factor: 0.1
```

3.16 Cost Timing Patterns

File: 19_cost_timing_patterns.yaml

3.16.1 Purpose

This file defines when costs are incurred during the project lifecycle for AFUDC (Allowance for Funds Used During Construction) capitalization. Per FERC Uniform System of Accounts, AFUDC applies when (i) capital expenditure is incurred AND (ii) activities are in progress.

3.16.2 Parameter Choices

For each cost category, the file specifies:

- **during_delay** (float): Percentage of cost incurred during delay period
 - **Valid Values:** Range [0, 1]
- **during_construction** (float): Percentage of cost incurred during construction period
 - **Valid Values:** Range [0, 1]
 - **Note:** Should sum to 1.0 with **during_delay**
- **afudc_eligible** (boolean): Whether cost qualifies for AFUDC
 - **Valid Values:** true or false
 - **Meaning:**
 - * **true:** Capitalized to CWIP (Account 107)
 - * **false:** Operating expense, not capitalized

3.16.3 Cost Categories

- **build_costs:** Conductor, structure, converter costs
 - **during_delay:** 0.0 (typically no cost during delay)
 - **during_construction:** 1.0 (spread uniformly over construction)

- `afudc_eligible`: true
- **row_acquisition**: ROW acquisition costs
 - `during_delay`: 0.80 (acquisition happens before construction)
 - `during_construction`: 0.20 (final acquisitions during early construction)
 - `afudc_eligible`: true
- **row_holding**: ROW holding costs during delay
 - `during_delay`: 1.0 (only during delay period)
 - `during_construction`: 0.0
 - `afudc_eligible`: false (operating expense)
- **row_rent**: ROW rent costs during operation
 - `during_delay`: 0.0
 - `during_construction`: 0.0
 - `afudc_eligible`: false (operating expense)
- **environmental_mitigation_base**: Base restoration costs
 - `during_delay`: 0.0
 - `during_construction`: 1.0
 - `afudc_eligible`: true
- **environmental_mitigation_credits**: Credit purchases
 - `during_delay`: 0.20 (some credits purchased upfront)
 - `during_construction`: 0.80
 - `afudc_eligible`: true
- **delay_costs**: Regulatory/legal expenses
 - `during_delay`: 1.0
 - `during_construction`: 0.0
 - `afudc_eligible`: false (typically expensed)
- **operations_and_maintenance**: O&M costs
 - `during_delay`: 0.0
 - `during_construction`: 0.0 (O&M begins after COD)
 - `afudc_eligible`: false

Constraint: For any cost type with `afudc_eligible`: true, `during_delay` and `during_construction` must sum to 1.0.

3.16.4 Example Configuration

```
1 cost_timing_patterns:
2   build_costs:
3     during_delay: 0.0
4     during_construction: 1.0
5     afudc_eligible: true
6
7   row_acquisition:
8     during_delay: 0.80
9     during_construction: 0.20
10    afudc_eligible: true
11
12   row_holding:
13     during_delay: 1.0
14     during_construction: 0.0
15     afudc_eligible: false
16
17   # ... (other cost categories)
```

3.17 Category-Specific Technical Data

3.17.1 ROW Widths

File: 20_project_category_row_widths.yaml

This file defines right-of-way width requirements for all project categories. Each category entry includes:

- **row_width_feet** (float): Required ROW width in feet
 - **Typical Values:** 80 to 200 feet (varies by capacity and construction type)

3.17.2 Circuit and Resistance Details

File: 21_project_category_circuit_and_resistance_detail.yaml

This file defines electrical and physical circuit specifications for all project categories. Each category entry includes:

- **conductor_specification** (string): Specific conductor model (e.g., "ACSR 477 kmil Hawk")
- **voltage_kv** (float): Operating voltage in kilovolts
- **conductors_per_phase** (integer): Number of conductors per phase
- **number_of_phases** (integer): Number of phases (typically 3 for AC)

- **number_of_circuits_poles** (integer): Number of circuits or poles
- **AC_75_resistance** (float): AC resistance at 75°C (ohms per mile)
- **DC_20_resistance** (float): DC resistance at 20°C (ohms per mile)
- **Meaning:** Used for line loss calculations in `energy_losses.py` and `line_loss_costs.py`

3.18 Special Considerations

3.18.1 Project Category Construction

Project categories are automatically constructed from core technical details (file 01) using the format:

```
"{construction_type}/{ac_dc}/{capacity_mw}MW/{conductor_type}/{converter_type}"
```

The CTCC uses this category to look up:

- Build costs (file 10)
- ROW widths (file 20)
- Circuit specifications (file 21)
- O&M costs (files 13-15)

3.18.2 Reconductoring vs. Greenfield

- **Greenfield:** New line construction
 - Full build costs
 - Line losses are a *cost* (new losses created)
 - Full congestion/curtailment benefits
- **Reconductoring:** Upgrading existing line
 - Lower build costs (conductor replacement only)
 - Line losses are a *benefit* (reduced losses vs. old line)
 - Incremental congestion/curtailment benefits
 - Must specify old line parameters in file 01

3.18.3 AC vs. DC

- **AC Projects:**

- `converter_type` = "NA"
- `number_of_converters` = 0 (not used)
- No converter costs or O&M

- **DC Projects:**

- Must specify `converter_type` ("LCC Converter" or "VSC Converter")
- Must specify `number_of_converters` (typically 2)
- Converter costs and O&M apply

3.18.4 Construction Type Effects

Each construction type affects multiple cost categories:

- **Overhead:**

- Moderate build costs
- High wildfire risk
- Moderate outage risk
- Vegetation management required
- Structure O&M required

- **Underground Direct-Buried:**

- High build costs
- Low wildfire risk (98% reduction)
- Lower outage rates but longer repair times
- No vegetation management
- Higher environmental mitigation (wetland impacts)

- **Underground Tunnel:**

- Very high build costs
- Low wildfire risk
- Lower outage rates but very long repair times
- No vegetation management
- Minimal surface disruption (major advantage)

- **Subsea:**

- Very high build costs
- No wildfire risk
- Low outage rates but extremely long repair times (240 hours)
- No vegetation management
- Specialized construction and maintenance

3.18.5 Terrain Distribution

The terrain distribution (file 02) affects:

- Build costs (via terrain multipliers)
- Wildfire ignition rates (file 06)
- Outage rates and durations (file 07)
- Environmental mitigation costs (file 09)
- Vegetation management costs (file 12)
- Structure density and O&M (file 14)

3.19 Summary

This guide has covered all 21 YAML configuration files used in the CTCC. Key takeaways:

1. **File 01** defines core project identity and is used to construct project categories
2. **File 02** defines physical route characteristics affecting multiple cost categories
3. **Files 03-09** define financial, risk, and environmental parameters
4. **Files 10-15, 20-21** contain category-specific data (build costs, O&M, technical specs)
5. **Files 16-18** define benefit parameters (emissions, congestion, curtailment)
6. **File 19** defines cost timing for AFUDC calculations
7. **File 11** defines ROW costs using a zone-based approach

All parameters work together to produce comprehensive cost and benefit calculations. Users should ensure consistency across files (e.g., construction type in file 01 matches category selections in files 10-15, 20-21).

Chapter 4

Path Config Documentation

4.1 Introduction

The `path_config.py` module provides centralized path configuration for CTCC scripts. It defines absolute paths based on the location of `path_config.py` (inside `scripts/`) so that YAML config and output directories are resolved consistently regardless of the current working directory. There are no environment overrides; paths are derived from `Path(__file__).parent.absolute()`.

4.2 Method Overview

4.2.1 Purpose

1. Define `SCRIPT_DIR` as the directory containing `path_config.py` (i.e. `scripts/`).
2. Define `PROJECT_ROOT` as the parent of `SCRIPT_DIR` (the CTCC project root).
3. Define `YAMLS_DIR = PROJECT_ROOT / "yamls"` (YAML configuration files).
4. Define `OUTPUTS_DIR = PROJECT_ROOT / "outputs"` (CSV and JSON output files).
5. Define `SCRIPTS_DIR = SCRIPT_DIR` (same as `scripts/`).
6. Export `YAMLS_DIR`, `OUTPUTS_DIR`, `PROJECT_ROOT`, `SCRIPTS_DIR` via `__all__`.

4.2.2 Calculation Approach

Paths only; no cost/benefit logic. All paths are `pathlib.Path` instances. Base directory is the script location, not the current working directory, so running `python scripts/build_costs.py` from the project root or from elsewhere yields the same `YAMLS_DIR` and `OUTPUTS_DIR` relative to the project.

4.2.3 Inputs and Outputs

Inputs:

- None (no env vars or function arguments); paths are computed from `__file__`.

Outputs:

- Module-level constants: `YAMLS_DIR`, `OUTPUTS_DIR`, `PROJECT_ROOT`, `SCRIPTS_DIR`. Used by `yaml_loaders`, `weighted_miles`, `build_costs`, `row_costs`, `insurance_costs`, `environmental_mitigation`, `oandm`, `energy_losses`, `bcr_calculator`, `csv_output_manager`, `smart_loaders`, `revenue`.

4.2.4 Integration with Other Scripts

- `yaml_loaders.py` — Uses `YAMLS_DIR` for all YAML file paths (e.g. `YAMLS_DIR / "01_project_technical_details.yaml"`).
- `weighted_miles.py` — `YAMLS_DIR / "02_project_physical_details.yaml"`.
- `build_costs.py`, `row_costs.py`, `insurance_costs.py`, `environmental_mitigation.py`, `oandm.py`, `energy_losses.py` — `YAMLS_DIR` for config YAMLs.
- `bcr_calculator.py` — `OUTPUTS_DIR` for `batch_summary.csv`, `YAMLS_DIR` for BCR config YAML.
- `csv_output_manager.py` — `OUTPUTS_DIR` (default output dir), `YAMLS_DIR` where used.
- `smart_loaders.py` — `YAMLS_DIR` when in YAML mode for `get_project_data_raw`, `get_financing_data_raw`, `get_physical_data_raw`.
- `revenue.py` — `OUTPUTS_DIR` for `batch_summary.csv` and JSON output paths.

4.3 Key Functions

There are no callable functions; only module-level constants are defined.

4.3.1 Constants

Key logic (path resolution):

```
1 SCRIPT_DIR = Path(__file__).parent.absolute()
2 PROJECT_ROOT = SCRIPT_DIR.parent
3 YAMLS_DIR = PROJECT_ROOT / "yamls"
4 OUTPUTS_DIR = PROJECT_ROOT / "outputs"
5 SCRIPTS_DIR = SCRIPT_DIR
6 __all__ = ["YAMLS_DIR", "OUTPUTS_DIR", "PROJECT_ROOT", "SCRIPTS_DIR"]
```

- `SCRIPT_DIR` — Absolute path to `scripts/` (directory containing `path_config.py`).
- `PROJECT_ROOT` — Absolute path to the CTCC project root (parent of `scripts/`).
- `YAMLS_DIR` — `PROJECT_ROOT / "yamls"`; used for all YAML config file paths.
- `OUTPUTS_DIR` — `PROJECT_ROOT / "outputs"`; used for `batch_summary.csv`, `json_output*.json`, `ctcc_results*.json`.
- `SCRIPTS_DIR` — Same as `SCRIPT_DIR`; alias for the scripts directory.

4.4 Example

Typical usage in a script:

```

1 from path_config import YAMLS_DIR, OUTPUTS_DIR
2 with open(YAMLS_DIR / "02_project_physical_details.yaml", "r") as f:
3     data = yaml.safe_load(f)
4 # ...
5 output_path = OUTPUTS_DIR / "batch_summary.csv"
```

If `path_config.py` is at `/path/to/CTCC/scripts/path_config.py`, then `PROJECT_ROOT` is `/path/to/CTCC`, `YAMLS_DIR` is `/path/to/CTCC/yamls`, and `OUTPUTS_DIR` is `/path/to/CTCC/outputs`, regardless of `os.getcwd()`.

Chapter 5

Smart Loaders Documentation

5.1 Introduction

The `smart_loaders.py` module provides input abstraction for CTCC: it chooses `yaml_loaders` or `json_loaders` at import time based on the `CTCC_INPUT_MODE` environment variable, then re-exports all `load_*` and related symbols from the chosen backend. Calculation scripts import from `smart_loaders` `import load_project_technical_details, load_financing_details, etc.`, and get the same API regardless of whether input is YAML or JSON; no code changes are needed when switching mode.

5.2 Method Overview

5.2.1 Purpose

1. At import time: read `CTCC_INPUT_MODE` (default `yaml`); if `json`, from `json_loaders import *`; else from `yaml_loaders import *`.
2. Expose `get_input_mode()` so callers can query the current mode ("`json`" or "`yaml`").
3. Provide mode-aware raw-data helpers: `get_project_data_raw()`, `get_financing_data_raw()`, `get_physical_data_raw()` (return the raw dict for project technical details, financing, or physical details from either YAML or JSON).
4. Provide `load_terrain_miles()` that uses `get_physical_data_raw()` and returns `terrain["terrain_miles"]`.
5. Re-export all `load_*` and other symbols from the active backend so scripts use a single import.

5.2.2 Calculation Approach

Dispatch only; no cost/benefit formulas. The module does not implement load logic; it only selects the backend and re-exports. All `load_*` implementations live in `yaml_loaders.py` or `json_loaders.py` (see their documentation).

5.2.3 Inputs and Outputs

Inputs:

- CTCC_INPUT_MODE environment variable (read once at import): `yaml` (default) or `json`.

Outputs:

- Same `load_*` API and return types as `yaml_loaders` or `json_loaders`; callers receive dataclasses or dicts from the chosen backend.

5.2.4 Integration with Other Scripts

- Every calculation script that loads configuration imports from `smart_loaders` (e.g. `from smart_loaders import load_project_technical_details, load_financing_details`).
- `smart_loaders` imports either `json_loaders` or `yaml_loaders` at import time; it does not call both in the same run.

5.3 Key Functions

5.3.1 Module-level logic (conditional import)

At import time:

```
1 _input_mode = os.environ.get('CTCC_INPUT_MODE', 'yaml').lower()
2 if _input_mode == 'json':
3     from json_loaders import *
4 else:
5     from yaml_loaders import *
```

All `load_*` and other public symbols from the selected module are then available as `smart_loaders.load_project_technical_details`, etc.

5.3.2 `get_input_mode()`

Returns the current input mode: `"json"` or `"yaml"` (the value determined at import time).

5.3.3 `get_project_data_raw()`

Returns the raw dict for project technical details (keys `project`, `timeline`). In JSON mode calls `json_loaders.data_source.get_data("01_project_technical_details")`; in YAML mode loads `YAMLS_DIR / 01_project_technical_details.yaml`.

5.3.4 `get_financing_data_raw()`

Returns the raw financing dict. In JSON mode `get_data("03_financing")`; in YAML mode loads `03_financing.yaml`.

5.3.5 `get_physical_data_raw()`

Returns the raw physical details dict (includes `terrain` with `terrain_miles`, etc.). In JSON mode `get_data("02_project_physical_details")`; in YAML mode loads `02_project_physical_details.yaml`.

5.3.6 `load_terrain_miles()`

Calls `get_physical_data_raw()`, then returns `physical_details["terrain"]["terrain_miles"]` (dict mapping terrain type to miles). Raises `KeyError` if `terrain` or `terrain_miles` is missing.

5.3.7 Re-exported `load_*` (see `yaml/json loader docs`)

All other `load_*` functions (e.g. `load_project_technical_details`, `load_financing_details`, `load_congestion_curtailment_reductions`, `load_physical_details`, `load_primary_bcr_config`) are defined in `yaml_loaders.py` or `json_loaders.py` and re-exported here; their parameters, return values, and key logic are documented in those modules.

5.4 Example

Script uses the same imports regardless of mode:

```
1 from smart_loaders import load_project_technical_details,
   load_financing_details
2 details = load_project_technical_details()
3 financing = load_financing_details()
```

To force JSON mode (e.g. before importing):

```
1 import os
2 os.environ["CTCC_INPUT_MODE"] = "json"
3 # Then import smart_loaders (or run script with CTCC_INPUT_MODE=json
   set in shell)
4 from smart_loaders import load_project_technical_details
5 details = load_project_technical_details() # data from JSON
```

Chapter 6

JSON Loaders Documentation

6.1 Introduction

The `json_loaders.py` module loads CTCC configuration from a single combined JSON file when `CTCC_INPUT_MODE=json`. It provides a singleton `JSONDataSource` that holds the combined JSON in memory and exposes `get_data(key)` for section access. A set of `load_*` functions mirror `yaml_loaders.py`: each maps a JSON key (e.g. `01_project_technical_details`) to the same dataclasses and structures used in YAML mode, so calculation scripts can use the same API regardless of input mode.

6.2 Method Overview

6.2.1 Purpose

1. Provide a singleton `JSONDataSource` that stores the combined JSON; data can be set via `set_json_data()` or auto-loaded from `CTCC_JSON_DATA_FILE` when `get_data()` is first called.
2. Expose `get_data(key)` to return the dict for a given top-level key (e.g. `03_financing`).
3. Provide `load_*` functions that call `get_data()` with the appropriate key and map the dict into the same dataclasses as `yaml_loaders` (e.g. `ProjectTechnicalDetails`, `FinancingDetails`, `CongestionCurtailmentParams`).
4. Use `calculation_utils` and `yaml_loaders` for normalization (e.g. `normalize_capacity_mw`, `normalize_construction_type`) and type definitions; `financial_utils` for WACC when loading financing.

6.2.2 Calculation Approach

I/O only; no cost/benefit formulas. Keys in the combined JSON match YAML file names (e.g. `01_project_technical_details`, `17_congestion_curtailment_reductions`). There is no scenario ID in the loader; the combined JSON is assumed to represent one scenario. Scenario

selection, if needed, is handled by the caller (e.g. server or CLI) before setting the JSON data.

6.2.3 Inputs and Outputs

Inputs:

- Combined JSON: set programmatically via `set_json_data(combined_json)`, or loaded from the file path in `CTCC_JSON_DATA_FILE` when `get_data()` is first called.
- Top-level keys in the JSON correspond to YAML section names (e.g. `01_project_technical_details`, `02_project_physical_details`, `03_financing`, `17_congestion_curtailment_reductions`, `22_primary_bcr_config`).

Outputs:

- In-memory dicts or dataclass instances returned to callers (calculation scripts); no files written by this module.

6.2.4 Integration with Other Scripts

- `smart_loaders.py` — When `CTCC_INPUT_MODE=json`, imports all `load_*` and `get_data` from `json_loaders`; calculation scripts then call `load_project_technical_details()`, `load_financing_details()`, etc., without knowing the backend.
- `yaml_loaders.py` — Supplies dataclass and type definitions (`ProjectTechnicalDetails`, `CongestionCurtailmentParams`, `PhysicalDetailsDetailed`, `CircuitAndResistanceDetails`, `FinancingDetails`) and `normalize_construction_type`.
- `calculation_utils.py` — `normalize_capacity_mw`, `get_converter_type`.
- `financial_utils.py` — `calculate_real_wacc` used in `load_financing_details`.

6.3 Key Functions

6.3.1 JSONDataSource

Singleton class that holds the combined JSON. `get_data(key)` returns `self._json_data[key]`; if `_json_data` is `None`, it first tries `load_from_env_file()` (read from `CTCC_JSON_DATA_FILE`).

Key logic (`get_data`):

```
1 if self._json_data is None:
2     self._load_from_env_file()
3 if self._json_data is None:
4     raise RuntimeError("JSON data not set...")
5 if key not in self._json_data:
```

```

6     raise KeyError(f"Key '{key}' not found in JSON data.")
7 return self._json_data[key]

```

Key logic (load from env file):

```

1 json_file_path = os.environ.get("CTCC_JSON_DATA_FILE")
2 if json_file_path and os.path.exists(json_file_path):
3     with open(json_file_path, "r") as f:
4         self._json_data = json.load(f)

```

6.3.2 set_json_data(combined_json)

Sets the combined JSON dict on the global JSONDataSource instance so all load_* functions use it.

6.3.3 load_project_technical_details()

Loads 01_project_technical_details, normalizes construction type and capacity, and returns ProjectTechnicalDetails (same as yaml_loaders).

Key logic:

```

1 project_details = _data_source.get_data("01
   _project_technical_details")
2 pd = project_details["project"]
3 tl = project_details["timeline"]
4 construction_type = normalize_construction_type(pd["
   construction_type"])
5 capacity_mw = normalize_capacity_mw(pd["capacity_mw"])
6 # ... converter_type, row_agreement_type, etc.
7 return ProjectTechnicalDetails(
8     construction_type=construction_type,
9     ac_dc=pd["ac_dc"],
10    capacity_mw=capacity_mw,
11    ...
12 )

```

6.3.4 load_financing_details()

Loads 03_financing, computes real WACC via calculate_real_wacc, returns FinancingDetails.

6.3.5 load_congestion_curtailment_reductions()

Loads 17_congestion_curtailment_reductions; checks 01_project_technical_details for reconductoring to choose greenfield_* vs reconductoring_* section. Returns CongestionCurtailmentParams with flow factor, binding hours, prices, curtailment parameters, etc.

6.3.6 load_primary_bcr_config()

Loads 22_primary_bcr_config and returns a dict of module enable/disable flags (operational, risk, energy, benefits). If the section is missing or invalid, returns default (all enabled) and optionally warns.

6.3.7 Other load_* functions

- load_physical_details() — 02_project_physical_details; returns total miles (sum of terrain.terrain_miles values).
- load_physical_details_detailed() — returns PhysicalDetailsDetailed with per-terrain miles.
- load_circuit_and_resistance_details(category) —
21_project_category_circuit_and_resistance_detail; returns
CircuitAndResistanceDetails for the given category.
- load_row_widths(category), load_row_details(), load_delay_costs(), load_emissions_details(), load_contingencies(), load_environmental_mitigation(), load_cost_timing_patterns(), load_afudc_config(), load_insurance_details(), load_wildfire_costs(), load_outage_costs(), load_terrain_data() — each calls get_data with the corresponding key and returns the section dict or derived values.

6.4 Example

Combined JSON top-level keys (same as YAML file names):

```
1 {  
2   "01_project_technical_details": { "project": {...}, "timeline":  
3     {...} },  
4   "02_project_physical_details": { "terrain": {...} },  
5   "03_financing": { "financial": {...} },  
6   "17_congestion_curtailment_reductions": { "greenfield_...": {...},  
7     "reconductoring_...": {...} },  
8   "22_primary_bcr_config": { "primary_bcr_config": {...} }  
9 }
```

Using the loader (e.g. in a calculation script):

```
1 # Set by caller or auto-load from CTCC_JSON_DATA_FILE  
2 from smart_loaders import load_project_technical_details,  
3   load_financing_details  
4 details = load_project_technical_details()  
5 financing = load_financing_details()
```

Chapter 7

CSV Output Manager Documentation

7.1 Introduction

The `csv_output_manager.py` script provides the `CTCCOutputManager` class, which manages all CSV outputs for CTCC batch analysis, sensitivity studies, and Monte Carlo simulations. It coordinates data from all calculation modules into a unified `batch_summary.csv` and individual module-specific CSVs.

7.2 Method Overview

7.2.1 Purpose

The CSV output manager:

1. Maintains a single `batch_summary.csv` with one row per scenario
2. Creates module-specific CSVs with detail and summary rows
3. Automatically prepends technical parameters to all CSV rows
4. Manages scenario IDs for coordinated batch runs
5. Updates existing rows instead of creating duplicates
6. Handles both detail rows (terrain-specific, component-specific) and summary rows

7.2.2 Architecture

The manager uses a two-tier CSV structure:

- **Batch Summary:** One comprehensive row per scenario with all metrics
- **Module CSVs:** Detail rows (e.g., by terrain, by component) plus summary row

All CSVs automatically include technical parameters as the first columns for easy filtering and comparison.

7.2.3 Integration with Other Scripts

This script is used by:

- All cost/benefit calculation scripts — via `smart_output.CTCCOutputManager` (dispatches to `CSVOutputManager` or `JSONOutputManager` by mode); each calls `add_*` methods and `write_batch_summary`
- `bcr_calculator.py` — Reads `batch_summary.csv` and writes BCR columns
- `ctcc.py` — Sets `CTCC_SCENARIO_ID` and coordinates final summary

Dependencies:

- `path_config.py` — `OUTPUTS_DIR`, `YAMLS_DIR`
- `smart_loaders.py` — `load_project_technical_details`, `load_physical_details`, `load_financing_social_discount_rate`, `get_project_data_raw` (for `load_technical_details`; supports YAML and JSON input mode)

7.3 Key Functions

7.3.1 `__init__(output_dir, scenario_id)`

Initializes the output manager.

Parameters:

- `output_dir` (str): Base directory for outputs (default: `"../outputs"`)
- `scenario_id` (str): Unique scenario identifier (optional)

Key Logic:

```
1 # Scenario ID: explicit > CTCC_SCENARIO_ID env > auto-generated (
   with microseconds)
2 self.scenario_id = (scenario_id or os.environ.get("CTCC_SCENARIO_ID"
   ))
3     or datetime.now().strftime("%Y%m%d_%H%M%S_%f"))
4 self.timestamp = datetime.now().isoformat()
5 technical_details = self.load_technical_details()
6 self.batch_summary_data = {**technical_details, "scenario_id": self.
   scenario_id, "timestamp": self.timestamp}
```

7.3.2 `load_technical_details()`

Loads technical parameters using centralized loaders (`smart_loaders: load_project_technical_details`, `load_physical_details`, `load_financing_social_discount_rate`, `get_project_data_raw`) to include in CSV outputs. Supports both YAML and JSON input modes.

Returns:

- Dictionary of technical parameters (project name, construction type, capacity, line length, discount rate, timeline, etc.)

Technical Parameters Loaded:

- Project identification (name)
- Core specs (construction_type, ac_dc, capacity_mw, conductor_type, line_length_miles, line_utilization)
- Converter details (converter_type, number_of_converters)
- Reconductoring details (reconductoring, old_capacity_mw, old_conductor_type, old_ac_dc)
- Financial parameters (baseline_electricity_price_per_mwh, social_discount_rate)
- Timeline (construction_years, delay_years, project_lifetime_years)

7.3.3 write_batch_summary()

Writes or updates the batch summary to CSV. If a row with the same `scenario_id` exists, it updates that row; otherwise, it appends a new row.

Key Logic:

```

1 # Check if scenario_id already exists
2 scenario_row_idx = None
3 for idx, row in enumerate(existing_rows):
4     if row.get('scenario_id') == self.scenario_id:
5         scenario_row_idx = idx
6         break
7
8 # Update existing row or append new one
9 if scenario_row_idx is not None:
10     existing_rows[scenario_row_idx].update(self.batch_summary_data)
11 else:
12     existing_rows.append(self.batch_summary_data)

```

7.3.4 write_module_csv(module_name, detail_rows, summary_row, append)

Writes module-specific CSV with optional detail and summary rows. Automatically prepends technical parameters to each row.

Parameters:

- `module_name` (str): Name of the module (e.g., 'wildfire_costs')
- `detail_rows` (list): List of dicts for detail rows (optional)

- `summary_row` (dict): Dict for summary row (optional)
- `append` (bool): If True, append to existing file; if False, overwrite

Key Logic:

```

1 # Extract technical parameters
2 tech_params = {k: self.batch_summary_data.get(k, '')
3                 for k in tech_param_keys}
4 tech_params['scenario_id'] = self.scenario_id
5 tech_params['timestamp'] = self.timestamp
6
7 # Prepend technical parameters to each row
8 for row in all_rows:
9     enriched_row = {**tech_params, **row} # Tech params first
10    enriched_rows.append(enriched_row)

```

7.3.5 Module-Specific Add Methods

Each calculation module has a corresponding `add_*` method:

- `add_build_costs(results)` - Build costs
- `add_row_costs(results)` - Right-of-way costs
- `add_environmental_mitigation(results)` - Environmental mitigation
- `add_delay_costs(results)` - Delay costs
- `add_insurance_costs(results)` - Insurance costs
- `add_wildfire_costs(results)` - Wildfire costs (with detail rows by terrain)
- `add_outage_costs(results)` - Outage costs (with detail rows by terrain)
- `add_oandm_costs(results)` - O&M costs (with detail rows by component)
- `add_emissions_costs(results)` - Emissions costs (with detail rows by pollutant)
- `add_line_loss_costs(results)` - Line loss costs/benefits
- `add_congestion_curtailment(results)` - Congestion and curtailment benefits/costs
- `add_bcr_metrics(bcr_results)` - BCR metrics

Each method:

1. Updates `batch_summary_data` with summary metrics
2. Creates detail rows (if applicable)
3. Creates summary row
4. Calls `write_module_csv()` to write the module CSV

7.3.6 add_congestion_curtailement(results)

Special method that distinguishes benefits from costs in detail rows.

Detail Rows Created:

- **Benefits:** Congestion reduction (full and haircut), Curtailment reduction (full and haircut)
- **Costs:** Congestion during delay, Curtailment during delay, Residual congestion

Each detail row includes:

- **benefit_or_cost:** "benefit" or "cost"
- **constraint_type:** "congestion", "curtailment", "congestion-delay", etc.
- **value_type:** "full", "haircut", or "NA"
- Annual, nominal, and PV values

7.3.7 add_bcr_metrics(bcr_results)

Adds BCR metrics to the batch summary.

Expected Keys in bcr_results:

- **bcr_system**, **bcr_capital**, **bcr_haircut**, **bcr_excluding_risk**
- **net_benefit_pv**, **net_benefit_nominal**, **net_benefit_excluding_risk_pv**
- **total_benefits_pv**, **total_costs_pv**, etc.

7.4 Example

The following example demonstrates typical usage:

```
1 # When called from build_costs.py:
2 csv_manager = CTCCOutputManager()
3
4 # Add build cost results
5 results = {
6     "total_nominal": 100000000,
7     "total_afudc": 105000000,
8     "total_pv": 95000000,
9     "conductor_nominal": 60000000,
10    "structure_nominal": 30000000,
11    "converter_nominal": 10000000,
12 }
13 csv_manager.add_build_costs(results)
14 csv_manager.write_batch_summary()
15
```

```

16 # This:
17 # 1. Updates batch_summary_data with build cost metrics
18 # 2. Creates build_costs.csv with:
19 #    - Technical parameters (first 20 columns)
20 #    - Summary row with all build cost metrics
21 # 3. Updates batch_summary.csv with build cost columns

```

Multi-Module Example:

```

1 # When called from ctcc.py (coordinated run):
2 import os
3 os.environ['CTCC_SCENARIO_ID'] = '20250101_120000'
4
5 # Each script creates its own manager instance
6 # All use the same scenario_id from environment variable
7
8 # build_costs.py:
9 csv_manager = CTCCOutputManager() # Gets scenario_id from env
10 csv_manager.add_build_costs(results)
11 csv_manager.write_batch_summary() # Creates/updates row
12
13 # row_costs.py:
14 csv_manager = CTCCOutputManager() # Same scenario_id
15 csv_manager.add_row_costs(results)
16 csv_manager.write_batch_summary() # Updates same row
17
18 # Result: batch_summary.csv has one row with all metrics

```

Detail Rows Example:

```

1 # When called from wildfire_costs.py:
2 csv_manager = CTCCOutputManager()
3 results = {
4     "EAL": 500000,
5     "lambda_total": 0.01,
6     "lambda_by_terrain": {
7         "forested": {"miles": 40, "events_per_year": 0.006, ...},
8         "farmland": {"miles": 60, "events_per_year": 0.004, ...},
9     },
10     ...
11 }
12 csv_manager.add_wildfire_costs(results)
13
14 # This creates wildfire_costs.csv with:
15 # - Technical parameters (first 20 columns)
16 # - Detail rows (one per terrain) with terrain-specific metrics
17 # - Summary row with aggregated metrics

```

7.5 Key Implementation Details

7.5.1 Scenario ID Management

The manager uses a three-tier priority for scenario IDs:

1. Explicit parameter (if provided)
2. Environment variable `CTCC_SCENARIO_ID` (for coordinated batch runs)
3. Auto-generated timestamp (if neither above)

This ensures all scripts in a single CTCC run share the same scenario ID.

7.5.2 Row Update vs. Append

The `write_batch_summary()` method checks if a row with the same `scenario_id` exists. If it does, it updates that row; otherwise, it appends a new row. This allows multiple scripts to contribute to the same scenario row.

7.5.3 Technical Parameters Prepend

All module CSVs automatically include technical parameters as the first columns. This is done in `write_module_csv()` by prepending the technical parameters dictionary to each row.

7.5.4 Fieldname Management

When updating `batch_summary.csv`, the manager:

1. Reads existing fieldnames
2. Merges with new fieldnames (existing first, then new)
3. Writes all rows with the merged fieldnames

This ensures backward compatibility when new columns are added.

7.5.5 Detail vs. Summary Rows

Module CSVs can include:

- **Detail rows:** Granular data (e.g., by terrain, by component, by pollutant)
- **Summary row:** Aggregated totals

Each row is marked with `row_type`: "detail" or "summary".

7.5.6 Path Resolution

The manager uses dynamic path resolution to find YAML files:

```
1 script_dir = os.path.dirname(os.path.abspath(__file__))  
2 yaml_dir = os.path.join(os.path.dirname(script_dir), "ymls")
```

This ensures the script works regardless of the current working directory.

7.5.7 Error Handling

The `load_technical_details()` method includes `try/except` to handle missing YAML files gracefully, returning an empty dictionary and printing a warning.

Chapter 8

JSON Output Manager Documentation

8.1 Introduction

The `json_output_manager.py` module aggregates and writes CTCC results as JSON when `CTCC_OUTPUT_MODE=json`. Each calculation script instantiates `JSONOutputManager` (via `smart_output.CTCCOutputManager`), calls `add_*` methods to store its results in memory, then calls `save_to_file()` to write an intermediate `json_output_<scenario_id>_<module>.json`. The parent `ctcc.py` then aggregates these files with `load_from_file()`, computes BCR, calls `add_bcr_metrics()` and `calculate_summary()`, and writes the final `ctcc_results_<scenario_id>.json` using `get_json_results()`.

8.2 Method Overview

8.2.1 Purpose

1. On init: set `scenario_id` (or auto-generate), load `technical_details` via `load_technical_details()` (uses `json_loaders`), and initialize `costs`, `benefits`, `summary`, `bcr` dicts.
2. Each calculation script calls `add_build_costs()`, `add_row_costs()`, `add_congestion_curtailement()`, etc., to store its result dict under the appropriate key.
3. Each script calls `save_to_file(output_dir, module_name)` to write an intermediate JSON file for aggregation.
4. `ctcc.py` calls `aggregate_json_outputs()` which creates a new `JSONOutputManager`, then `load_from_file()` for each intermediate file to merge `costs`, `benefits`, `bcr`; then `add_bcr_metrics()`, `calculate_summary()`, and `get_json_results()` to build the final dict; `ctcc.py` writes `ctcc_results_<id>.json` to disk.

8.2.2 Calculation Approach

Output only; no cost/benefit formulas. `calculate_summary()` sums stored costs and benefits into `summary`: total capital (build + row + environmental), total operational (insurance + oandm), total energy/emissions (line_loss + emissions + residual exceedance from congestion_curtailment), total risk (wildfire + outage + wildfire_liability), delay costs, and grand total (nominal, AFUDC, PV). Residual exceedance and congestion/curtailment delay costs are taken from `benefits["congestion_curtailment"]` and included in the appropriate totals.

8.2.3 Inputs and Outputs

Inputs:

- In-memory results from calculation scripts passed to `add_*`() (e.g. `add_build_costs(results)`).
- `scenario_id` (optional at init; auto-generated if not set).
- Technical parameters loaded from `json_loaders` in `load_technical_details()`.
- BCR metrics from `ctcc.py` via `add_bcr_metrics(bcr_results)`.

Outputs:

- Per-script: `save_to_file()` writes `json_output_<scenario_id>_<module_name>.json` to `outputs/` (or given `output_dir`).
- Final: `get_json_results()` returns a dict with `scenario_id`, `timestamp`, `technical_parameters`, `costs`, `benefits`, `summary`, `bcr`; `ctcc.py` writes this to `ctcc_results_<scenario_id>.json`.

8.2.4 Integration with Other Scripts

- `smart_output.py` — When `CTCC_OUTPUT_MODE=json`, exports `JSONOutputManager` as `CTCCOutputManager`; all calculation scripts use it.
- `json_loaders.py` — `load_project_technical_details()`, `load_physical_details()`, `get_data()` used in `load_technical_details()`.
- `ctcc.py` — `aggregate_json_outputs()` uses `JSONOutputManager` and `load_from_file()`; `write_final_json_output()` uses `add_bcr_metrics()`, `calculate_summary()`, `get_json_results()`, then writes the final JSON file.

8.3 Key Functions

8.3.1 `JSONOutputManager.__init__(scenario_id=None)`

Sets `scenario_id` (or auto-generated `timestamp`), loads `technical_details` via `load_technical_details()`, and initializes `costs`, `benefits`, `summary`, `bcr` as empty dicts.

8.3.2 load_technical_details()

Loads project and financial data via `json_loaders` and returns a dict of technical parameters (project_name, construction_type, ac_dc, capacity_mw, line_length_miles, reconductoring, old_capacity_mw, delay_years, project_lifetime_years, social_discount_rate, etc.). On failure returns a minimal dict with an `error` key.

8.3.3 add_*() methods

Each `add_*()` stores the passed dict under a fixed key: `add_build_costs` → `costs["build"]`, `add_row_costs` → `costs["row"]`, `add_environmental_mitigation` → `costs["environmental"]`, `add_delay_costs` → `costs["delay"]`, `add_insurance_costs` → `costs["insurance"]`, `add_wildfire_liability_costs` → `costs["wildfire_liability"]`, `add_wildfire_costs` → `costs["wildfire"]`, `add_outage_costs` → `costs["outage"]`, `add_oandm_costs` → `costs["oandm"]`, `add_emissions_costs` → `costs["emissions"]`, `add_line_loss_costs` → `costs["line_loss"]`, `add_congestion_curtailement` → `benefits["congestion_curtailement"]`, `add_revenue` → `benefits["revenue"]`. `add_bcr_metrics(bcr_results)` sets `self.bcr`.

8.3.4 calculate_summary()

Computes `self.summary` from `self.costs` and `self.benefits`: capital (build + row + environmental), operational (insurance + oandm), energy/emissions (line_loss + emissions + residual exceedance from congestion_curtailement), risk (wildfire + outage + wildfire liability), delay; then grand total nominal, AFUDC, and PV.

Key logic (excerpt):

```
1 self.summary["total_capital_pv"] = (  
2     build.get("total_pv", 0) + row.get("total_pv", 0) + env.get("  
3         total_pv", 0)  
4 )  
5 # ... operational, risk, energy_emissions (incl. residual from  
6     congestion_curtailement)  
7 self.summary["grand_total_cost_pv"] = (  
8     self.summary["total_capital_pv"] + self.summary["  
9         total_operational_pv"]  
10 + self.summary["total_risk_pv"] + delay.get("total_pv", 0)  
11 + congestion_delay_pv + curtailement_delay_pv  
12 + self.summary["total_energy_emissions_pv"]  
13 )
```

8.3.5 get_json_results()

Calls `calculate_summary()` if `summary` is empty, then returns {scenario_id, timestamp, input_mode, technical_parameters, costs, benefits, summary, bcr}.

8.3.6 save_to_file(output_dir=None, module_name=None)

Writes current state (scenario_id, timestamp, technical_details, costs, benefits, summary, bcr) to json_output_<scenario_id>_<module_name>.json (or json_output_<scenario_id>.json if module_name is omitted). Default output_dir is ../outputs relative to the script directory. Returns the path to the saved file.

8.3.7 load_from_file(json_file)

Reads the JSON file and merges state["costs"], state["benefits"], state["bcr"] into self.costs, self.benefits, self.bcr. Used by ctcc.aggregate_json_outputs() to combine per-script outputs.

Key logic:

```
1 with open(json_file, "r") as f:
2     state = json.load(f)
3 if state.get("costs"):
4     for key, value in state["costs"].items():
5         if value is not None:
6             self.costs[key] = value
7 if state.get("benefits"):
8     for key, value in state["benefits"].items():
9         if value:
10            self.benefits[key] = value
11 if state.get("bcr"):
12     self.bcr = state["bcr"]
```

8.4 Example

Final output structure (from get_json_results()):

```
1 {
2     "scenario_id": "20250101_120000",
3     "timestamp": "2025-01-01T12:00:00",
4     "input_mode": "json",
5     "technical_parameters": { "project_name": "...", "capacity_mw":
6         500, ... },
7     "costs": {
8         "build": { "total_nominal": ..., "total_pv": ..., "total_afudc":
9             ... },
10        "row": { ... },
11        "environmental": { ... },
12        "delay": { ... },
13        "insurance": { ... },
14        "wildfire": { ... },
15        "outage": { ... },
16        "wildfire_liability": { ... },
```

```

15     "oandm": { ... },
16     "emissions": { ... },
17     "line_loss": { ... }
18 },
19 "benefits": {
20     "congestion_curtailment": { "congestion_benefit_pv": ..., "
    residual_exceedance_pv": ..., ... },
21     "revenue": { ... }
22 },
23 "summary": { "total_capital_pv": ..., "grand_total_cost_pv": ...,
    ... },
24 "bcr": { "bcr_system": ..., "bcr_capital": ..., ... }
25 }

```

Chapter 9

Smart Output Documentation

9.1 Introduction

The `smart_output.py` module provides output abstraction for CTCC: it chooses `CSVOutputManager` or `JSONOutputManager` at instantiation time based on the `CTCC_OUTPUT_MODE` environment variable, then delegates all method calls to the underlying manager. Calculation scripts import `CTCCOutputManager` from `smart_output` (which is an alias for `SmartOutputManager`) and use the same `add_*`, `write_batch_summary` API regardless of mode; the backend writes either CSV (`batch_summary.csv` and related) or JSON (intermediate `json_output_<id>_<module>.json` files for aggregation by `ctcc.py`).

9.2 Method Overview

9.2.1 Purpose

1. On `CTCCOutputManager()` (i.e. `SmartOutputManager()`): read `CTCC_OUTPUT_MODE` (default `csv`); if `json`, create `JSONOutputManager(scenario_id=CTCC_SCENARIO_ID` or `‘‘default’’`); else create `csv_output_manager.CTCCOutputManager()`.
2. Expose the same API as the underlying manager: `add_build_costs`, `add_row_costs`, `write_batch_summary`, etc., via `__getattr__` delegation to `self._manager`.
3. Override `write_batch_summary`: in JSON mode, infer the calling script name (from the call stack or `sys.argv[0]` when `__name__ == ‘__main__’`), then call `self._manager.save_to_file(module_name=module_name)`; in CSV mode, call `self._manager.write_batch_summary()`.

9.2.2 Calculation Approach

Dispatch only; no cost/benefit formulas. The module does not implement output logic; it only selects the backend and delegates. All `add_*` and write logic live in `csv_output_manager.py` or `json_output_manager.py` (see their documentation).

9.2.3 Inputs and Outputs

Inputs:

- `CTCC_OUTPUT_MODE` environment variable (read at `SmartOutputManager.__init__`): `csv` (default) or `json`.
- `CTCC_SCENARIO_ID` (optional, used in JSON mode when constructing `JSONOutputManager`).

Outputs:

- Same `CTCCOutputManager` API; backend writes CSV (`batch_summary.csv`, etc.) or JSON (intermediate files; final `ctcc_results_<id>.json` is written by `ctcc.py` after aggregation).

9.2.4 Integration with Other Scripts

- Every calculation script that writes results imports `CTCCOutputManager` from `smart_output` and calls `add_build_costs(results)`, `write_batch_summary()`, etc.
- `smart_output` delegates to `csv_output_manager.CTCCOutputManager` or `json_output_manager.JSONOutputManager`; it does not implement `add_*` or file format logic.

9.3 Key Functions

9.3.1 SmartOutputManager (exported as CTCCOutputManager)

Wrapper class that holds either a `CSVOutputManager` or `JSONOutputManager` and delegates all attribute access to it.

9.3.2 __init__()

Reads `CTCC_OUTPUT_MODE` (default `csv`). If `json`, instantiates `JSONOutputManager(scenario_id=os.environ.get('CTCC_SCENARIO_ID', 'default'))`; else instantiates `csv_output_manager.CTCCOutputManager()`. Stores the instance in `self._manager`.

Key logic:

```
1 self.output_mode = os.environ.get('CTCC_OUTPUT_MODE', 'csv').lower()
2 if self.output_mode == 'json':
3     from json_output_manager import JSONOutputManager
4     scenario_id = os.environ.get('CTCC_SCENARIO_ID', 'default')
5     self._manager = JSONOutputManager(scenario_id=scenario_id)
6 else:
7     from csv_output_manager import CTCCOutputManager
8     self._manager = CTCCOutputManager()
```

9.3.3 write_batch_summary()

In JSON mode: infers the calling script name (inspect stack; if `__name__ == '__main__'` use `os.path.basename(sys.argv[0])` without extension), then calls `self._manager.save_to_file(module_name=module_name)` and returns the result. In CSV mode: calls `self._manager.write_batch_summary()` and returns the result.

9.3.4 __getattr__(name)

Delegates to `getattr(self._manager, name)` so all `add_build_costs`, `add_row_costs`, `add_bcr_metrics`, etc., are forwarded to the underlying CSV or JSON manager.

9.4 Example

Script uses the same code for both modes:

```
1 from smart_output import CTCCOutputManager
2 output_manager = CTCCOutputManager()
3 output_manager.add_build_costs(results)
4 output_manager.write_batch_summary()
```

To force JSON mode (e.g. before instantiating):

```
1 import os
2 os.environ["CTCC_OUTPUT_MODE"] = "json"
3 os.environ["CTCC_SCENARIO_ID"] = "test"
4 from smart_output import CTCCOutputManager
5 output_manager = CTCCOutputManager()
6 output_manager.add_build_costs(results)
7 output_manager.write_batch_summary() # writes
   json_output_test_build_costs.json
```

Chapter 10

Weighted Miles Documentation

10.1 Introduction

The `weighted_miles.py` script computes terrain-adjusted miles for a transmission line from project physical details. It reads miles per terrain type and terrain multipliers from `02_project_physical_details.yaml`, then returns weighted miles and an average terrain multiplier. These values are used downstream by `build_costs.py` and `insurance_costs.py` for cost-per-mile calculations.

10.2 Method Overview

10.2.1 Purpose

1. Load `02_project_physical_details.yaml` (via `path_config.YAMLS_DIR`).
2. For each terrain type, multiply `terrain_miles[terrain_type]` by `terrain_multipliers[terrain_type]` and sum to get weighted miles.
3. Compute average terrain multiplier as weighted miles divided by total (unweighted) miles.
4. Return (`weighted_miles`, `average_terrain_multiplier`); when run as `__main__`, print both values.

10.2.2 Calculation Approach

Data come from `project_physical_details["terrain"]`: `terrain_miles` is a dict mapping terrain type to miles; `terrain_multipliers` maps terrain type to a cost/mile multiplier.

$$\text{weighted_miles} = \sum_{\text{terrain_type}} \text{terrain_miles}[\text{terrain_type}] \times \text{terrain_multipliers}[\text{terrain_type}] \quad (10.1)$$

$$\text{total_miles} = \sum_{\text{terrain_type}} \text{terrain_miles}[\text{terrain_type}] \quad (10.2)$$

$$\text{average_terrain_multiplier} = \frac{\text{weighted_miles}}{\text{total_miles}} \quad (10.3)$$

10.2.3 Inputs and Outputs

Inputs:

- `02_project_physical_details.yaml` (path from `path.config.YAMLS_DIR`): under `terrain`, keys `terrain_miles` (dict: terrain type $\rightarrow$ miles) and `terrain_multipliers` (dict: terrain type $\rightarrow$ multiplier).

Outputs:

- Return value: (`weighted_miles: float`, `average_terrain_multiplier: float`). When the script is run standalone, both values are printed to stdout. The script does not write to the CSV or JSON output managers; downstream scripts (`build_costs.py`, `insurance_costs.py`) call `calculate_weighted_miles()` and use the result.

10.2.4 Integration with Other Scripts

- `path_config.py` — `YAMLS_DIR` for the physical details file path.
- `build_costs.py` — Imports `calculate_weighted_miles`, uses `weighted_miles` for conductor and structure cost per mile; passes `weighted_miles` and `average_terrain_multiplier` to the output manager.
- `insurance_costs.py` — Imports `calculate_weighted_miles` for use in its calculations.

10.3 Key Functions

10.3.1 `calculate_weighted_miles()`

Calculates weighted miles and average terrain multiplier from project physical details (YAML `02_project_physical_details.yaml`).

Parameters: None (reads from YAML path provided by `path.config`).

Returns: tuple: (`weighted_miles: float`, `average_terrain_multiplier: float`).

Key logic:

```

1 weighted_miles = 0
2 for terrain_type, terrain_miles in project_physical_details_df["
  terrain"][
3     "terrain_miles"
4 ].items():
5     if terrain_miles is None:
6         terrain_miles = 0
7     weighted_miles += (
8         terrain_miles
9         * project_physical_details_df["terrain"]["
10            terrain_multipliers"][
11                terrain_type
12            ]
13 )
14 average_terrain_multiplier = weighted_miles / sum(
15     project_physical_details_df["terrain"]["terrain_miles"].values()
16 )
17 return weighted_miles, average_terrain_multiplier

```

10.4 Example

Standalone run (prints weighted miles and average terrain multiplier):

```

1 cd scripts
2 python weighted_miles.py

```

Typical YAML structure under **terrain**:

```

1 terrain:
2   terrain_miles:
3     forested: 25
4     scrubbed_flat: 35
5     wetland: 5
6   terrain_multipliers:
7     forested: 2.25
8     scrubbed_flat: 1.0
9     wetland: 1.5

```

Example: weighted miles = $25 \times 2.25 + 35 \times 1.0 + 5 \times 1.5 = 56.25 + 35 + 7.5 = 98.75$; total miles = 65; average terrain multiplier = $98.75/65 \approx 1.52$.

Chapter 11

Build Costs Calculation Documentation

11.1 Introduction

The `build_costs.py` script calculates the construction costs for a transmission line project. It computes costs for conductors, structures, and converters, applies terrain adjustments via weighted miles, incorporates contingency factors, and calculates both regulatory (AFUDC) and societal (present value) perspectives.

11.2 Method Overview

11.2.1 Purpose

This script calculates the total build cost of a transmission line project by:

1. Loading project technical details and constructing category via `calculation_utils.build_category_string`
2. Loading base costs from `10_project_category_build_costs.yaml` for that category
3. Adjusting costs for terrain complexity using `weighted_miles.calculate_weighted_miles()`
4. Applying per-component contingency factors (conductor, structure, converter)
5. Loading AFUDC setup via `financial_utils.load_afudc_setup`; if enabled, calculating AFUDC capitalized cost using timing patterns for regulatory perspective
6. Computing present value of build costs (spread evenly over construction period) for societal perspective
7. Writing results via `CTCCOutputManager.add_build_costs` and `write_batch_summary`

11.2.2 Calculation Approach

The build cost calculation follows this structure:

$$\text{Conductor Cost} = (\text{Variable Cost/Mile} \times \text{Weighted Miles}) + \text{Fixed Cost} \quad (11.1)$$

$$\text{Structure Cost} = \text{Variable Cost/Mile} \times \text{Weighted Miles} \quad (\text{if not reconductoring}) \quad (11.2)$$

$$\text{Converter Cost} = \text{Fixed Cost} \times \text{Number of Converters} \quad (\text{if not reconductoring}) \quad (11.3)$$

$$\text{Total Base Cost} = \text{Conductor} + \text{Structure} + \text{Converter} \quad (11.4)$$

$$\text{Conductor with Conting.} = \text{Conductor} \times (1 + \text{conductor_contingency}) \quad (11.5)$$

$$\text{Structure with Conting.} = \text{Structure} \times (1 + \text{structure_contingency}) \quad (11.6)$$

$$\text{Converter with Conting.} = \text{Converter} \times (1 + \text{converter_contingency}) \quad (11.7)$$

$$\text{Total with Contingencies} = \text{sum of above three} \quad (11.8)$$

For reconductoring projects, structure and converter costs are set to zero since existing infrastructure is reused.

11.2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type, AC/DC, capacity, conductor type, etc.)
- Physical details (terrain distribution)
- Build cost parameters from `10_project_category_build_costs.yaml` (by category from `build_category_string`)
- Per-component contingency percentages (`conductor_contingency`, `structure_contingency`, `converter_contingency`) from financing
- Financing parameters (WACC, inflation); AFUDC from `load_afudc_setup` (timing patterns, AFUDC rate)

Outputs:

- Nominal costs (with contingencies)
- AFUDC capitalized costs (regulatory perspective)
- Present value costs (societal perspective)
- Component-level costs (conductor, structure, converter)

11.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` - `load_project_technical_details`, `load_physical_details`, `load_contingencies`, `load_financing_details`, `load_cost_timing_patterns`, `load_afudc_config`, `get_project_data_raw`, `get_financing_data_raw`
- `weighted_miles.py` - `calculate_weighted_miles` for terrain-adjusted miles
- `calculation_utils.py` - `build_category_string` for project category
- `financial_utils.py` - `calculate_present_value`, `calculate_afudc_capitalized_cost`, `load_afudc_setup`, `calculate_construction_start_year`, `validate_discount_rate`
- `path_config.py` - `YAMLS_DIR` for build costs YAML
- `smart_output.py` - `CTCCOutputManager` (writes CSV or JSON via `add_build_costs`, `write_batch_summary`)

11.3 Key Functions

11.3.1 `load_costs(category, total_miles, number_of_converters, contingencies, reconductoring)`

Loads and calculates build costs for the specified project category.

Parameters:

- `category` (str): Project category identifier in format "ConstructionType/AC_DC/CapacityMW/ConductorType/ConverterType"
- `total_miles` (float): Total miles of transmission line
- `number_of_converters` (int): Number of converters needed (0 for AC)
- `contingencies` (dict): Dictionary with keys "conductor_contingency", "structure_contingency", "converter_contingency"
- `reconductoring` (bool): Whether this is a reconductoring project

Returns: A `BuildCosts` dataclass with fields: `total_cost`, `total_cost_with_contingencies`, `conductor_cost`, `structure_cost`, `converter_cost`, `conductor_cost_with_contingencies`, `structure_cost_with_contingencies`, `converter_cost_with_contingencies`, `weighted_miles`, `average_terrain_multiplier`. Structure and converter costs are 0 for reconductoring.

Key Logic:

```

1 weighted_miles, average_terrain_multiplier =
    calculate_weighted_miles()
2 conductor_cost = (variable_conductor_cost_per_mile * weighted_miles)
    + fixed_conductor_cost
3 if reconductoring:
4     structure_cost = 0
5     converter_cost = 0
6 else:
7     structure_cost = variable_structure_cost_per_mile *
        weighted_miles
8     converter_cost = fixed_converter_cost * number_of_converters
9 total_cost = conductor_cost + structure_cost + converter_cost
10 conductor_cost_with_contingencies = conductor_cost * (1 +
    contingencies["conductor_contingency"])
11 structure_cost_with_contingencies = structure_cost * (1 +
    contingencies["structure_contingency"])
12 converter_cost_with_contingencies = converter_cost * (1 +
    contingencies["converter_contingency"])
13 total_cost_with_contingencies = (conductor_cost_with_contingencies
    + structure_cost_with_contingencies +
        converter_cost_with_contingencies)
14
15 return BuildCosts(...)

```

11.3.2 main()

Main execution function that orchestrates the build cost calculation and output.

Workflow:

1. Loads project technical details; constructs category via `build_category_string(project_details)`
2. Determines `number_of_converters` from `get_project_data_raw()["project"]["number_of_converters"]`; if DC, else 0
3. Loads physical details, contingencies, financing; calls `load_costs(category, total_miles, number_of_converters, contingencies, project_details.reconductoring)`
4. Loads AFUDC setup via `load_afudc_setup()`; if `apply_afudc`, calculates AFUDC capitalized cost via `calculate_afudc_capitalized_cost(...)`
5. Computes `construction_start_year = calculate_construction_start_year(delay_years)`; if `construction_years > 0`, `annual_build_cost = total with contingencies / construction_years`, `build_cost_pv = calculate_present_value(annual_build_cost, wacc_real, construction_years, construction_start_year)`; else one-time PV at `construction_start_year`

```

6. Builds results dict (total_nominal, total_afudc, total_pv,
   component nominals and zeros for component AFUDC);
   CTCCOutputManager().add_build_costs(results).write_batch_summary()

```

11.4 Example

The following example demonstrates a typical usage of the build costs calculation:

```

1 # The script is typically called from ctcc.py, but can be run
  standalone
2 # It automatically loads configuration from YAML files
3
4 # Example: 500 MW Overhead AC line with Advanced Aluminum Conductor
5 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
6 #
7 # Inputs (from YAML):
8 #   - Construction type: Overhead
9 #   - AC/DC: AC
10 #   - Capacity: 500 MW
11 #   - Conductor: Advanced Aluminum
12 #   - Total miles: 100 miles
13 #   - Terrain: 60% flat, 40% forested
14 #   - Contingencies: 15% conductor, 20% structure, 10% converter
15 #
16 # Calculation:
17 #   1. Weighted miles = 100 * (0.6*1.0 + 0.4*1.2) = 108 miles
18 #   2. Conductor cost = ($500k/mile * 108) + $1M = $55M
19 #   3. Structure cost = $200k/mile * 108 = $21.6M
20 #   4. Converter cost = 0 (AC project)
21 #   5. Total base = $76.6M
22 #   6. With contingencies = $76.6M * (1 + weighted avg) approx $88M
23 #
24 # Output:
25 #   - Nominal cost: $88M
26 #   - AFUDC capitalized: $92M (if AFUDC enabled)
27 #   - Present value: $88M (discounted at real WACC)

```

Integration Example:

```

1 # When called from ctcc.py:
2 # 1. Script loads project details from YAML
3 # 2. Calculates build costs
4 # 3. Writes to CSV via CTCCOutputManager:
5 csv_manager = CTCCOutputManager()
6 results = {
7     "total_nominal": costs.total_cost_with_contingencies,
8     "total_afudc": capitalized_cost_with_contingencies if
      afudc_setup.apply_afudc else 0,

```

```

9      "total_pv": build_cost_pv,
10     "conductor_nominal": costs.conductor_cost_with_contingencies,
11     "structure_nominal": costs.structure_cost_with_contingencies,
12     "converter_nominal": costs.converter_cost_with_contingencies,
13     "conductor_afudc": 0, "structure_afudc": 0, "converter_afudc":
        0,
14 }
15 csv_manager.add_build_costs(results)
16 csv_manager.write_batch_summary()

```

11.5 Key Implementation Details

11.5.1 Reconductoring Handling

For reconductoring projects, the script sets structure and converter costs to zero since existing infrastructure is reused. Only conductor costs are calculated.

11.5.2 AFUDC Calculation

If AFUDC is enabled (`load_afudc_setup().apply_afudc`), the script calculates capitalized costs via `calculate_afudc_capitalized_cost` using the AFUDC rate and cost timing patterns (`afudc_setup.timing_patterns["build_costs"]`). This represents the regulatory perspective where construction financing costs are capitalized.

11.5.3 Present Value of Build Costs

The script computes present value of build costs spread evenly over the construction period: `annual_build_cost = total_cost_with_contingencies / construction_years`, then `calculate_present_value(annual_build_cost, wacc_real, construction_years, construction_start_year)`. If `construction_years` is 0, build cost is treated as a one-time cost at `construction_start_year`.

11.5.4 Weighted Miles

The script uses `calculate_weighted_miles()` to adjust costs for terrain complexity. Different terrain types have different cost multipliers, and the weighted average is applied to the total miles.

Chapter 12

Right-of-Way Costs Calculation Documentation

12.1 Introduction

The `row_costs.py` script calculates right-of-way (ROW) costs for a transmission line project. ROW costs depend on the **ROW agreement type**: who owns or controls the corridor and how the utility secures the right to use it. The script uses a **ROW agreement type** switch to determine which cost terms apply, so that acquisition (upfront payment) and annual ROW payment (ongoing permission) are **mutually exclusive** unless the regime explicitly allows both (e.g., federal/hybrid).

Cost terms used in the model:

- **Acquisition**: One-time upfront payment (easement compensation, fee-simple purchase, or similar).
- **Holding cost**: Option fee—an explicit annual payment during the delay (permitting) period to secure the ROW so it is not taken by others. Not the same as “cost of capital” on acquisition; cost of capital on acquisition during delay is handled by AFUDC.
- **Annual ROW payment**: Ongoing payment for permission to use the corridor (lease, license, or federal annual use fee). Labeled in data and outputs as “annual ROW payment”; in legacy inputs the parameter may still be named `rent_cost`.

Canonical notation. The canonical variable names and equations for ROW are in `COST METHODOLOGY.md`: Capital ROW (acquisition and holding) under “Capital ROW costs (1.b)”, and operational ROW rent under “Operational ROW rent (2.c)”. Capital uses $C_{\text{ROW},\text{capital},\text{nominal}}$, $C_{\text{ROW},\text{capital},\text{real}}$; rent uses $C_{\text{rent},\text{annual}}$, T_{rent} , $t_{\text{rent},\text{start}}$, $C_{\text{rent},\text{total},\text{nominal}}$, $C_{\text{rent},\text{total},\text{real}}$.

12.2 ROW Agreement Types

The following agreement types drive which cost terms are applied. Only one regime applies per scenario.

12.2.1 Permanent easement (new)

Typical greenfield: utility secures a **permanent** right to build and operate (easement or similar). Landowner or agency is paid upfront; there is no ongoing “rent” for the same corridor.

- **Acquisition:** Yes (upfront payment).
- **Annual ROW payment:** No (set to zero in the calculator).
- **Holding cost:** Optional. If the user provides an option fee (`hold_cost` per acre per year), it is applied during the delay period only.

12.2.2 Lease / license (existing)

Utility uses an **existing** corridor (e.g., highway, railroad, another utility) and pays the owner for ongoing permission. No upfront purchase of the ROW.

- **Acquisition:** No (set to zero).
- **Holding cost:** No (set to zero).
- **Annual ROW payment:** Yes (paid over delay + construction + project lifetime, or as configured).

Reconductoring or “uses existing ROW” scenarios typically use this regime.

12.2.3 Fee simple purchase

Utility **buys** the strip of land. Same cost treatment as permanent easement (new): upfront only, no annual ROW payment. Holding (option fee) optional during delay if applicable.

12.2.4 Federal / hybrid

Some federal or negotiated arrangements have both an upfront fee and an annual use fee for the same corridor. Both acquisition and annual ROW payment are applied.

- **Acquisition:** Yes.
- **Annual ROW payment:** Yes.
- **Holding cost:** Optional (option fee during delay if provided).

12.3 Mutual Exclusivity and Defaults

- For **permanent easement (new)** and **fee simple**, annual ROW payment from the ROW-details file is **ignored** (set to zero) so that “buy the ROW” does not also charge rent for the same miles.
- For **lease / license (existing)**, acquisition and holding are set to zero; only annual ROW payment applies.
- If `row_agreement_type` is not set in the project technical details, it is **inferred**: if `reconductoring` or `uses_existing_row` is true, regime is **lease_license_existing**; otherwise **permanent_easement_new**. This keeps existing scenarios consistent.

12.4 Configuring YAMLS for ROW Costs

This section describes how to set up the YAML files for each ROW scenario. No method re-explanation—only which files and keys to use.

Files involved:

- `01_project_technical_details.yaml` — `project` section: `reconductoring`, `uses_existing_row`, and optionally `row_agreement_type`.
- `11_project_row_details.yaml` — `right_of_way` with one entry per zone; each zone has `miles`, `acquisition_cost`, `hold_cost` (\$/acre/year), and `rent_cost` (\$/acre/year, annual ROW payment).

12.4.1 New greenfield (permanent easement)

Result: Acquisition + optional option fee (holding). No annual ROW payment for the same miles.

- **01:** `reconductoring: False`, `uses_existing_row: False`. Omit `row_agreement_type` (inferred as `permanent_easement_new`).
- **11:** Set `miles`, `acquisition_cost`, and `hold_cost` per zone. `rent_cost` is ignored. To get **acquisition only** (no option fee), set `hold_cost: 0` for every zone that has `miles > 0`.

12.4.2 Existing corridor (lease / license)

Result: Annual ROW payment only. No acquisition, no holding.

- **01:** `uses_existing_row: True` or `reconductoring: True`. Omit `row_agreement_type` (inferred as `lease_license_existing`).
- **11:** Set `miles` and `rent_cost` per zone. `acquisition_cost` and `hold_cost` are ignored.

12.4.3 Fee simple purchase

Result: Same as new greenfield (acquisition + optional holding; no annual ROW payment).

- **01:** `row_agreement_type:` `fee_simple`. Or use the same 01 as new greenfield (inference gives `permanent_easement_new`, which has the same cost treatment).
- **11:** Same as new greenfield: `acquisition_cost`, `hold_cost`; `rent_cost` ignored.

12.4.4 Federal / hybrid (upfront + annual)

Result: Acquisition and annual ROW payment (and optional holding). Both upfront and ongoing payments apply.

- **01:** `row_agreement_type:` `federal_hybrid`.
- **11:** Set miles, `acquisition_cost`, `rent_cost`, and optionally `hold_cost` per zone. All three cost fields are used.

12.4.5 Explicit override of inferred type

To force a regime that does not match the default inference (e.g., new build but you want lease treatment, or vice versa), set `row_agreement_type` in `01_project_technical_details.yaml` to one of: `permanent_easement_new`, `lease_license_existing`, `fee_simple`, `federal_hybrid`. Then configure `11_project_row_details.yaml` as in the corresponding subsection above.

12.4.6 Quick reference

Scenario	01 settings	11: which costs used
New greenfield	<code>reconductoring:</code> <code>False</code> , <code>uses_existing_row:</code> <code>False</code>	<code>acquisition_cost</code> , <code>hold_cost</code>
Acquisition only	Same as above	Same; set <code>hold_cost:</code> <code>0</code> everywhere
Existing corridor	<code>uses_existing_row:</code> <code>True</code> or <code>reconductoring:</code> <code>True</code>	<code>rent_cost</code> only
Federal/hybrid	<code>row_agreement_type:</code> <code>federal_hybrid</code>	<code>acquisition_cost</code> , <code>rent_cost</code> , <code>hold_cost</code>

12.5 Method Overview

12.5.1 Purpose

The script:

1. Determines ROW width from project category.
2. Computes zone acres from zone miles and ROW width.
3. Loads ROW agreement type (or infers it).

4. For each zone with miles > 0, computes potential acquisition, holding, and annual ROW payment from per-acre rates; then applies the **agreement type** to zero out terms that do not apply.
5. Aggregates costs across zones.
6. Applies AFUDC capitalization (acquisition only, when eligible) for the regulatory perspective.
7. Computes present values for the societal perspective.

12.5.2 Acres and Zone-Level Potential Costs

Zone area (acres):

$$\text{Zone Acres} = \frac{\text{Zone Miles} \times 5280 \times \text{ROW Width (ft)}}{43560}. \quad (12.1)$$

From the ROW details file, each zone has (per acre): **acquisition_cost**, **hold_cost** (\$/acre/year), and **rent_cost** (\$/acre/year, interpreted as annual ROW payment). The script first computes:

$$\text{Potential acquisition} = \sum_{\text{zones}} \text{Zone Acres} \times \text{Acquisition cost/acre}, \quad (12.2)$$

$$\text{Potential yearly holding} = \sum_{\text{zones}} \text{Zone Acres} \times \text{Hold cost/acre/year}, \quad (12.3)$$

$$\text{Potential yearly annual ROW payment} = \sum_{\text{zones}} \text{Zone Acres} \times \text{Rent cost/acre/year}. \quad (12.4)$$

Then the **ROW agreement type** is applied:

- **Permanent easement (new) or fee simple**: Use acquisition and (optionally) holding; set annual ROW payment to zero.
- **Lease / license (existing)**: Use annual ROW payment only; set acquisition and holding to zero.
- **Federal / hybrid**: Use acquisition, annual ROW payment, and (optionally) holding.

Nominal totals (after applying the regime) are then computed for the chosen terms over the appropriate time periods (delay, construction, lifetime).

12.5.3 Inputs and Outputs

Inputs:

- Project technical details (construction type, AC/DC, capacity, etc.), including optional **row_agreement_type** (if absent, inferred from **reconductoring** and **uses_existing_row**).

- Physical details (terrain/zone miles).
- ROW width for project category (`20_project_category_row_widths`).
- ROW cost parameters by zone (`11_project_row_details`): `acquisition_cost`, `hold_cost`, `rent_cost` (annual ROW payment per acre per year).
- Financing (WACC, base year), AFUDC configuration, and cost timing patterns (`19_cost_timing_patterns`).
- Timeline (delay years, construction years, project lifetime).

Outputs:

- Nominal costs (acquisition, holding, annual ROW payment).
- AFUDC capitalized cost: ROW capital = acquisition + holding (when applicable); rent is excluded from capital.
- Present value costs (societal perspective); `row_capital_pv`, `row_rent_pv` for aggregation.
- Total acres.

Legacy output keys may still use “rent” (e.g., `rent_nominal`) for compatibility; conceptually the amount is “annual ROW payment.”

12.6 AFUDC, Capital vs. Operational

- **Acquisition** is AFUDC-eligible when the regime includes it; the cost timing pattern `row_acquisition` (e.g., 80% during delay, 20% during construction) is used to capitalize it to COD.
- **Holding cost** (option fee during delay) is **AFUDC-eligible**: it is capitalized using `row_holding` timing (100% during delay) so that ROW capital for rate base includes both acquisition and holding.
- **Annual ROW payment** (rent) is an **operational** expense: it is not AFUDC-eligible and is included in operational costs (with O&M and insurance), not in capital or rate base.
- **ROW capital** for rate base and for cost aggregation = acquisition + holding only. **Rate base** in CTCC = AFUDC capital (build + ROW capital + environmental mitigation).

12.7 Key Functions

12.7.1 calculate_zone_costs(row_width_feet)

Returns (yearly_holding_cost, acquisition_cost, yearly_rent_cost, total_acres) for all zones with miles > 0. Uses load_row_details() and miles_to_acres(miles, row_width_feet); sums zone acres × acquisition_cost, hold_cost, rent_cost per zone. The caller applies the ROW agreement type to zero out terms that do not apply.

Key logic:

```
1 row_details = load_row_details()
2 for zone_details in row_details["right_of_way"].values():
3     if zone_details["miles"] > 0:
4         zone_acres = miles_to_acres(zone_details["miles"],
5                                     row_width_feet)
6         acquisition_cost += zone_acres * zone_details["
7             acquisition_cost"]
8         yearly_rent_cost += zone_acres * zone_details["rent_cost"]
9         yearly_holding_cost += zone_acres * zone_details["hold_cost"]
10    ]
11 return yearly_holding_cost, acquisition_cost, yearly_rent_cost,
12        total_acres
```

12.7.2 main()

1. Load project technical details; construct category via build_category_string; get row_agreement_type from getattr(project_details, "row_agreement_type", None) or infer (lease_license_existing if reconductoring or uses_existing_row, else permanent_easement_new).
2. Load ROW width via load_row_widths(category); call calculate_zone_costs(row_width_feet).
3. Apply agreement type: lease_license_existing → acquisition = 0, holding = 0; permanent_easement_new or fee_simple → yearly_rent_cost = 0; federal_hybrid keeps all.
4. Compute nominal totals (holding × delay years; rent over appropriate years; acquisition_cost_used); rent_start_year and rent_total_years depend on agreement type (lease: year 1, delay+construction+lifetime; else: construction start, construction+lifetime).
5. Load load_afudc_setup(); if apply_afudc and acquisition used, calculate_afudc_capitalized_cost(acquisition_cost_used, timing_patterns["row_acquisition"], ...); else acquisition PV = acquisition / $(1 + r)^{\text{delay_years}}$.

```
6. add_row_costs(results)    with    total_nominal,    total_afudc,    total_pv,  
    acquisition_nominal, holding_nominal, rent_nominal; write_batch_summary().
```

12.8 Present Value Timing

- **Holding cost:** Annually during delay (years 1 to `delay_years`).
- **Acquisition:** One-time at end of delay (year `delay_years`), discounted to base.
- **Annual ROW payment:** For lease/license (existing), from year 1 for (delay + construction + lifetime) years; for new builds with annual payment (federal/hybrid), from construction start for (construction + lifetime) years.

12.9 Acres Calculation

$$\text{Acres} = \frac{\text{Miles} \times 5280 \text{ ft/mile} \times \text{ROW Width (ft)}}{43560 \text{ ft}^2/\text{acre}}.$$

12.10 Integration

The script integrates with:

- `smart_loaders.py` — `load_project_technical_details`, `load_physical_details`, `load_row_widths`, `load_row_details`, `load_financing_details`, `load_cost_timing_patterns`, `load_afudc_config`, `get_financing_data_raw`
- `calculation_utils.py` — `build_category_string`, `miles_to_acres`
- `financial_utils.py` — `calculate_present_value`, `calculate_afudc_capitalized_cost`, `validate_discount_rate`, `calculate_construction_start_year`, `load_afudc_setup`
- `path_config.py` — `YAMLS_DIR` (used by loaders)
- `smart_output.py` — `CTCCOutputManager` (`add_row_costs`, `write_batch_summary`)

Chapter 13

Environmental Mitigation Costs Calculation Documentation

13.1 Introduction

The `environmental_mitigation.py` script calculates environmental mitigation costs for a transmission line project. It computes base construction/restoration costs and wetland/habitat credit purchases across different construction types and terrain types, applying uplift factors for effective acreage.

13.2 Method Overview

Canonical notation. The canonical variable names and equations (including $A_{\text{terrain,habitat}}^{\text{effective impact}}$ for effective acres of habitat impact per terrain, and $C_{\text{credits,habitat}}$ as a sum over habitat terrains using that per-terrain acreage) are defined in `COST_METHODODOLOGY.md` under “Environmental mitigation (1.c)”. This document summarizes the implementation in code.

13.2.1 Purpose

This script calculates environmental mitigation costs by:

1. Calculating ROW acres by terrain type
2. Applying uplift factor to determine effective acres (TCE - Total Cost Equivalent)
3. Computing base mitigation/restoration costs per effective acre
4. Calculating wetland credit purchases for impacted wetland acres
5. Calculating habitat credit purchases for impacted natural habitat acres
6. Applying AFUDC capitalization for regulatory perspective
7. Computing present values for societal perspective

13.2.2 Calculation Approach

The environmental mitigation cost calculation follows this structure:

$$\text{Terrain Acres} = \frac{\text{Terrain Miles} \times 5280 \times \text{ROW Width (feet)}}{43560} \quad (13.1)$$

$$\text{Effective Acres} = \text{Base Acres} \times \text{Uplift Factor} \quad (13.2)$$

$$\text{Base Cost} = \sum_{\text{terrains}} \text{Cost/Acre} \times \text{Effective Acres} \quad (13.3)$$

$$\text{Wetland Credits} = \text{Wetland Cost/Acre} \times \text{Ratio} \times \text{Wetland Impacted Acres} \quad (13.4)$$

$$\text{Habitat Credits} = \text{Habitat Cost/Acre} \times \text{Ratio} \times \text{Habitat Impacted Acres} \quad (13.5)$$

$$\text{Total} = \text{Base Cost} + \text{Wetland Credits} + \text{Habitat Credits} \quad (13.6)$$

The uplift factor accounts for additional mitigation requirements beyond the physical ROW footprint (e.g., buffer zones, restoration areas).

13.2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type, AC/DC, capacity, etc.)
- Physical details (terrain distribution)
- ROW width for project category
- Environmental mitigation parameters (base costs, credit costs, credit ratios, uplift factor)
- Financing parameters (WACC, inflation, AFUDC rate)
- Timeline (delay years, construction years)

Outputs:

- Nominal costs (base mitigation, wetland credits, habitat credits)
- AFUDC capitalized costs (regulatory perspective)
- Present value costs (societal perspective)
- Acreage metrics (base acres, effective acres, impacted acres)

13.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_project_technical_details`, `load_physical_details`, `load_row_widths`, `load_environmental_mitigation`, `load_financing_details`, `load_cost_timing_patterns`, `load_afudc_config`, `get_physical_data_raw`, `get_financing_data_raw`, `load_terrain_miles`
- `calculation_utils.py` — `build_category_string`, `normalize_construction_type_for_yaml`, `miles_to_acres`
- `financial_utils.py` — `calculate_present_value`, `calculate_afudc_capitalized_cost`, `validate_discount_rate`, `calculate_construction_start_year`, `load_afudc_setup`
- `path_config.py` — `YAMLS_DIR`
- `smart_output.py` — `CTCCOutputManager` (`add_environmental_mitigation`, `write_batch_summary`)

13.3 Key Functions

13.3.1 `calculate_environmental_mitigation_costs(em_yaml, category, terrain_miles_dict, row_width_feet, reconductoring)`

Calculates environmental mitigation costs including base mitigation and credit purchases.

Parameters:

- `em_yaml` (dict): Loaded environmental mitigation YAML data
- `category` (str): Project category identifier
- `terrain_miles_dict` (dict): Dictionary mapping terrain type to miles
- `row_width_feet` (float): Right-of-way width in feet
- `reconductoring` (bool): If True, sets wetland and habitat credits to zero. Reconductoring projects use existing ROW and structures, so they don't create new permanent environmental impacts requiring mitigation credits. Base mitigation costs still apply for temporary construction impacts.

Returns:

- Dictionary containing:
 - `base_cost` (float): Base mitigation/restoration cost
 - `wetlands_credits` (float): Wetland credit purchase cost

- `habitat_credits` (float): Habitat credit purchase cost
- `total_credits` (float): Sum of wetland and habitat credits
- `total` (float): Total mitigation cost
- `uplift_factor_applied` (float): Uplift factor used
- `total_base_acres` (float): Base ROW acres
- `total_effective_acres` (float): Effective acres with uplift
- `wetland_impacted_acres` (float): Wetland acres impacted
- `habitat_impacted_acres` (float): Habitat acres impacted

Key Logic:

```

1 construction_type = category.split("/") [0]
2 yaml_construction_type = normalize_construction_type_for_yaml(
   construction_type, context="environmental")
3 for terrain, miles in terrain_miles_dict.items():
4     if miles > 0:
5         terrain_acres = miles_to_acres(miles, row_width_feet)
6         acres_by_terrain[terrain] = terrain_acres
7         total_base_acres += terrain_acres
8         effective_acres = terrain_acres * uplift_factor
9         cost_per_acre = base_costs.get(yaml_construction_type, {}).
           get(terrain, 0.0)
10        base_cost += cost_per_acre * effective_acres
11 wetland_impacted_acres = acres_by_terrain.get("wetland", 0.0) *
   uplift_factor
12 habitat_impacted_acres = sum(acres_by_terrain.get(t, 0.0) for t in
   habitat_terrains) * uplift_factor
13 wetlands_credits = wetland_cost_per_acre * wetland_ratio *
   wetland_impacted_acres
14 habitat_dict = credits.get("habitat", {})
15 ratios_habitat = ratios.get("habitat", {})
16 default_cost, default_ratio = habitat_dict.get("default", 0.0),
   ratios_habitat.get("default", 1.0)
17 habitat_credits = 0.0
18 for terrain in habitat_terrains:
19     terrain_acres = acres_by_terrain.get(terrain, 0.0)
20     if terrain_acres > 0:
21         effective_acres = terrain_acres * uplift_factor
22         cost_per_acre = habitat_dict.get(terrain, default_cost)
23         ratio_val = ratios_habitat.get(terrain, default_ratio)
24         habitat_credits += cost_per_acre * ratio_val *
           effective_acres
25 if reconductoring:
26     wetlands_credits = 0.0
27     habitat_credits = 0.0

```

13.3.2 main()

Main execution function that orchestrates the environmental mitigation cost calculation and output.

Workflow:

1. Loads project technical details; constructs category via `build_category_string`; loads ROW width via `load_row_widths(category)`; loads terrain via `load_terrain_miles()`; loads em_yaml via `load_environmental_mitigation()`
2. Calls `calculate_environmental_mitigation_costs(em_yaml, category, terrain_miles, row_width_feet, project_details.reconductoring)`
3. Loads `load_afudc_setup()`; if `apply_afudc`, calculates AFUDC for base and credits separately using `timing_patterns["environmental_mitigation_base"]` and `["environmental_mitigation_credits"]`
4. Credits PV: one-time at `credit_start_year` = `calculate_construction_start_year(delay_years)`; base PV: `calculate_present_value(annual_base_cost, wacc_real, construction_years, credit_start_year)` (or one-time at `credit_start_year` if `construction_years` is 0)
5. `add_environmental_mitigation(results); write_batch_summary()`

13.4 Example

The following example demonstrates a typical usage of the environmental mitigation cost calculation:

```
1 # Example: 500 MW Overhead AC line
2 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
3 #
4 # Inputs (from YAML):
5 #   - ROW width: 200 feet
6 #   - Terrain distribution:
7 #     * Forested: 40 miles
8 #     * Wetland: 10 miles
9 #     * Farmland: 50 miles
10 #   - Uplift factor: 1.5x
11 #   - Base cost: $2000/acre (overhead, forested), $3000/acre (
    overhead, wetland)
12 #   - Wetland credit: $5000/acre, ratio 2.0:1
13 #   - Habitat credit: $3000/acre, ratio 1.5:1 (in full
    implementation, cost and ratio can vary by habitat terrain)
14 #   - Construction: 2 years
15 #
16 # Calculation:
17 #   Forested acres = (40 * 5280 * 200) / 43560 = 969.7 acres
```

```

18 # Wetland acres = (10 * 5280 * 200) / 43560 = 242.4 acres
19 # Farmland acres = (50 * 5280 * 200) / 43560 = 1212.1 acres
20 #
21 # Effective acres (with 1.5x uplift):
22 # * Forested: 969.7 * 1.5 = 1454.6 acres
23 # * Wetland: 242.4 * 1.5 = 363.6 acres
24 #
25 # Base cost = (1454.6 * $2000) + (363.6 * $3000) = $4.4M
26 # Wetland credits = $5000 * 2.0 * 363.6 = $3.6M
27 # Habitat credits = $3000 * 1.5 * 1454.6 = $6.5M
28 # Total = $14.5M
29 #
30 # Output:
31 # - Nominal cost: $14.5M
32 # - AFUDC capitalized: $15.2M (if AFUDC enabled)
33 # - Present value: $13.8M (base spread over 2 years, credits
    upfront)

```

Integration Example:

```

1 # When called from ctcc.py:
2 csv_manager = CTCCOutputManager()
3 csv_results = {
4     "total_nominal": results["total"],
5     "total_afudc": total_capitalized if apply_afudc else 0,
6     "total_pv": total_pv,
7     "base_cost_nominal": results["base_cost"],
8     "credits_nominal": results["total_credits"],
9 }
10 csv_manager.add_environmental_mitigation(csv_results)
11 csv_manager.write_batch_summary()

```

13.5 Key Implementation Details

13.5.1 Uplift Factor

The uplift factor (TCE - Total Cost Equivalent) accounts for mitigation requirements beyond the physical ROW footprint. For example, a 1.5x uplift means that for every acre of ROW, 1.5 acres of mitigation is required (including buffer zones, restoration areas, etc.).

13.5.2 Credit Ratios

Credit ratios represent the mitigation requirement. For example, a 2.0:1 ratio for wetlands means that for every acre of wetland impacted, 2 acres of wetland credits must be purchased.

13.5.3 Habitat Terrain Classification

The script identifies habitat-impacted acres by summing natural terrain types: `forested`, `scrubbed_flat`, `desert_barren`, `rolling_hills`, and `mountain`. This is a conservative approach that captures all potential habitat impacts. Habitat credit cost and ratio can be specified per terrain in the configuration (with a `default` fallback); the total habitat credit cost is the sum over habitat terrains of $(\text{cost_per_acre} \times \text{ratio} \times \text{effective_acres})$ for each terrain.

13.5.4 Present Value Timing

- **Base mitigation costs:** Spread evenly over construction period (annual payments)
- **Credit costs:** One-time payment at start of construction (end of delay period)

13.5.5 AFUDC Treatment

Base mitigation and credit costs are treated separately for AFUDC calculation, as they may have different timing patterns during construction.

13.5.6 Reconductoring Projects

For reconductoring projects, environmental mitigation credits (wetland and habitat credits) are set to zero. This reflects that reconductoring projects use existing ROW and structures, so they do not create new permanent environmental impacts requiring mitigation credits. However, base environmental mitigation costs still apply to account for temporary construction impacts such as:

- Temporary access roads and staging areas
- Erosion control and sediment management during construction
- Temporary disturbance of existing ROW during conductor replacement

This treatment makes reconductoring economics more realistic by avoiding the large credit costs that would be inappropriate for projects that don't disturb new land.

Chapter 14

Insurance Costs Calculation Documentation

14.1 Introduction

The `insurance_costs.py` script calculates operational insurance costs and, when enabled, wildfire liability insurance costs for transmission line assets. Operational premiums are based on insurable asset value (conductors, structures, converters) and are paid annually over the project lifetime. Wildfire liability uses a rate-on-line (ROL) formula: annual premium = ROL $\times$ liability limit. Both are written to the batch summary (operational via `add_insurance_costs`; wildfire liability via `add_wildfire_liability_costs`, with zeros when disabled).

14.2 Method Overview

Canonical notation. The canonical variable names and equations for operational insurance (asset-based) are defined in `COST_METHODOLOGY.md` under “Operational costs” / “Operational insurance (2.b)”. Insurable value $V_{\text{insurable}}$ is the sum of included build-cost components (with contingency); annual premium $C_{\text{insurance,annual}} = V_{\text{insurable}} \times r_{\text{premium}}(CT)$; nominal and present-value formulas follow as in the methodology.

14.2.1 Purpose

This script calculates insurance costs by:

1. Loading project details and constructing category via `build_category_string`; loading build costs via `build_costs.load_costs` to get conductor/structure/converter costs with contingencies
2. Determining insurable asset value from which components are insured (`insurable_components`; default all)
3. Applying premium rate from `premium_by_construction_type` or default `premium_rate`

4. Calculating annual premium and lifetime nominal cost for operational insurance
5. Optionally calculating wildfire liability premium ($\text{ROL} \times \text{liability limit}$) when `insurance.wildfire_liability.enabled` is true
6. Calculating present value of annual premiums (payment start at COD) for both operational and wildfire liability
7. Writing `add_insurance_costs` and `add_wildfire_liability_costs` (zeros when wildfire liability disabled), then `write_batch_summary`

14.2.2 Calculation Approach

The insurance cost calculation follows this structure:

$$\text{Insurable Value} = \sum_{\text{insured components}} \text{Component Cost with Contingencies} \quad (14.1)$$

$$\text{Annual Premium} = \text{Insurable Value} \times \text{Premium Rate} \quad (14.2)$$

$$\text{Lifetime Nominal Cost} = \text{Annual Premium} \times \text{Project Lifetime} \quad (14.3)$$

$$\text{Present Value} = \sum_{t=\text{COD}}^{\text{COD}+T-1} \frac{\text{Annual Premium}}{(1+r)^t} \quad (14.4)$$

where T is the project lifetime, r is the real WACC discount rate. The first payment year is COD (Commercial Operation Date) = `delay_years` + `construction_years` + 1, i.e. the first year after construction ends.

Wildfire liability (when enabled): Annual premium = `rate_on_line` × `liability_limit`; nominal lifetime cost = annual premium × `project_lifetime`; PV same as above starting at COD. Returns `None` if `wildfire_liability.enabled` is false or parameters invalid.

14.2.3 Inputs and Outputs

Inputs:

- Build costs (conductor, structure, converter with contingencies) via `build_costs.load_costs`
- Insurance configuration from `load_insurance_details` (04_insurance): `insurance.insurable_components`, `premium_by_construction_type`, `premium_rate`, `insurance.wildfire_liability.enabled`, `liability_limit`, `rate_on_line`
- Project technical details (construction type, project lifetime, delay years, construction years)
- Financing parameters (WACC for present value)

Outputs:

- Operational: insurable value, annual premium, premium rate, nominal lifetime cost, PV total (via `add_insurance_costs`)
- Wildfire liability: when enabled, same metrics plus liability limit and rate-on-line (via `add_wildfire_liability_costs`); when disabled, zeros written

14.2.4 Integration with Other Scripts

This script integrates with:

- `build_costs.py` - `load_costs` for conductor/structure/converter costs with contingencies
- `smart_loaders.py` - `load_project_technical_details`, `load_physical_details`, `load_contingencies`, `load_financing_details`, `load_insurance_details`, `get_project_data_raw`
- `calculation_utils.py` - `build_category_string`
- `financial_utils.py` - `calculate_present_value`, `calculate_cod_year`
- `constants.py` - `TRANSMISSION_TYPE_DC` for DC check
- `smart_output.py` - `CTCCOutputManager` (`add_insurance_costs`, `add_wildfire_liability_costs`, `write_batch_summary`)

14.3 Key Functions

14.3.1 `calculate_insurance_costs(insurance_yaml, conductor_cost_with_contingencies, structure_cost_with_contingencies, converter_cost_with_contingencies, construction_type, project_lifetime)`

Calculates operational insurance costs based on insurable asset value.

Parameters:

- `insurance_yaml` (dict): Loaded insurance YAML data
- `conductor_cost_with_contingencies` (float): Conductor cost with contingencies
- `structure_cost_with_contingencies` (float): Structure cost with contingencies
- `converter_cost_with_contingencies` (float): Converter cost with contingencies
- `construction_type` (str): Type of construction (overhead, underground, subsea)
- `project_lifetime` (int): Project lifetime in years

Returns:

- Dictionary containing:

- insurable_value (float): Total insurable asset value
- annual_premium (float): Annual insurance premium
- premium_rate (float): Premium rate applied
- nominal_lifetime_cost (float): Total lifetime cost in nominal terms

Key Logic:

```

1 # Calculate insurable value based on which components are insured
2 insurable_value = 0.0
3 if components.get("conductors", True):
4     insurable_value += conductor_cost_with_contingencies
5 if components.get("structures", True):
6     insurable_value += structure_cost_with_contingencies
7 if components.get("converters", True):
8     insurable_value += converter_cost_with_contingencies
9
10 # Get premium rate (construction type specific or default)
11 premium_rate = premium_by_type.get(
12     construction_type.lower(),
13     ins_cfg.get("premium_rate", 0.002)
14 )
15
16 # Calculate annual premium
17 annual_premium = insurable_value * premium_rate
18 nominal_lifetime_cost = annual_premium * project_lifetime

```

14.3.2 calculate_wildfire_liability_premium(insurance_yaml, project_lifetime)

Calculates wildfire liability insurance premium using rate-on-line (ROL). Returns None if insurance.wildfire_liability.enabled is false or liability_limit / rate_on_line ≤ 0.

Returns: Dict with liability_limit, rate_on_line, annual_premium, nominal_lifetime_cost, or None.

Key logic:

```

1 if not wildfire_liability.get("enabled", False):
2     return None
3 liability_limit = wildfire_liability.get("liability_limit", 0)
4 rate_on_line = wildfire_liability.get("rate_on_line", 0)
5 if liability_limit <= 0 or rate_on_line <= 0:
6     return None
7 annual_premium = rate_on_line * liability_limit
8 nominal_lifetime_cost = annual_premium * project_lifetime

```

```

9 return {"liability_limit": ..., "rate_on_line": ..., "annual_premium": ..., "nominal_lifetime_cost": ...}

```

14.3.3 main()

Main execution function that orchestrates the insurance cost calculation and output.

Workflow:

1. Loads project technical details; constructs category via `build_category_string`; gets `number_of_converters` from `get_project_data_raw` if DC
2. Loads build costs via `load_costs(category, total_miles, number_of_converters, contingencies, reconductoring)`; loads insurance via `load_insurance_details()`
3. Calls `calculate_insurance_costs(...)` and `calculate_wildfire_liability_premium(insurance_yaml, project_lifetime)`.
4. `insurance_start_year = calculate_cod_year(delay_years, construction_years);`
`insurance_pv = calculate_present_value(annual_premium, wacc_real, project_lifetime, insurance_start_year);` same for wildfire liability PV if enabled
5. `add_insurance_costs(csv_results); add_wildfire_liability_costs(...)` (with zeros when disabled); `write_batch_summary()`

14.4 Example

The following example demonstrates a typical usage of the insurance cost calculation:

```

1 # Example: 500 MW Overhead AC line
2 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
3 #
4 # Inputs:
5 #   - Conductor cost (with contingencies): $55M
6 #   - Structure cost (with contingencies): $25M
7 #   - Converter cost: $0 (AC project)
8 #   - Premium rate: 0.2% (overhead construction)
9 #   - Project lifetime: 50 years
10 #   - Payment start: Year 3 (after 2-year construction)
11 #
12 # Calculation:
13 #   Insurable value = $55M + $25M + $0 = $80M
14 #   Annual premium = $80M * 0.002 = $160,000
15 #   Lifetime nominal = $160,000 * 50 = $8M
16 #   Present value = $160,000 * PV annuity (50 years, 3% discount) = $4.2M

```

```

17 #
18 # Output:
19 #   - Insurable value: $80M
20 #   - Annual premium: $160,000
21 #   - Lifetime nominal: $8M
22 #   - Present value: $4.2M

```

Integration Example:

```

1 # When called from ctcc.py:
2 # 1. Script loads build costs from build_costs.py
3 # 2. Calculates insurance costs
4 # 3. Writes to CSV via CTCCOutputManager:
5 csv_manager = CTCCOutputManager()
6 csv_results = {
7     "annual_premium": results["annual_premium"],
8     "nominal_lifetime_cost": results["nominal_lifetime_cost"],
9     "pv_total": insurance_pv,
10    "insurable_value": results["insurable_value"],
11    "premium_rate": results["premium_rate"],
12 }
13 csv_manager.add_insurance_costs(csv_results)
14 if wildfire_liability_results:
15     csv_manager.add_wildfire_liability_costs(wildfire_csv_results)
16 else:
17     csv_manager.add_wildfire_liability_costs({"annual_premium": 0, "
18         nominal_lifetime_cost": 0, "pv_total": 0, "liability_limit":
19         0, "rate_on_line": 0})
20 csv_manager.write_batch_summary()

```

14.5 Key Implementation Details

14.5.1 Insurable Components

The script allows configuration of which components are insurable (conductors, structures, converters). By default, all components are insured, but this can be customized in the YAML configuration.

14.5.2 Premium Rate by Construction Type

Different construction types may have different premium rates. The script first checks for a construction-type-specific rate, then falls back to a default rate if not specified.

14.5.3 Payment Timing

Insurance premiums start at Commercial Operation Date (COD), which is the first year after the delay and construction periods ($COD = \text{delay_years} + \text{construction_years} + 1$).

Premiums are paid annually for the project lifetime.

14.5.4 Discount rate and perspective

Operational and wildfire liability insurance PV use the **real WACC** (utility/private perspective). The resulting PV is a private cost to the utility. CTCC can report both societal-cost PV (e.g. wildfire expected loss in file 06, which uses the social discount rate) and utility-cost PV (insurance); totals that combine them are hybrid unless interpreted accordingly.

14.5.5 Wildfire Liability

Wildfire liability insurance covers the **insured** portion of wildfire risk. The wildfire cost module (file 06) uses **uninsured** severity (residual damages), so including both this module and wildfire liability insurance does not double-count. Wildfire liability is optional and controlled by `insurance.wildfire_liability.enabled` in the insurance YAML. When enabled, $\text{annual premium} = \text{rate_on_line} \times \text{liability_limit}$. The script always writes `add_wildfire_liability_costs` (with zeros when disabled) so batch summary columns remain consistent.

14.5.6 AFUDC Treatment

Operational insurance is not AFUDC-eligible because it is an operating expense, not a construction cost. Only construction-period costs are eligible for AFUDC capitalization.

14.5.7 Dependency on Build Costs

This script requires build costs to be calculated first (via `build_costs.py`) to determine insurable asset values. It imports and calls `load_costs()` from `build_costs.py`.

Chapter 15

Delay Costs Calculation Documentation

15.1 Introduction

The `delay_costs.py` script calculates the costs incurred during the delay period (permitting/regulatory delays) for a transmission line project. These costs include legal, administrative, labor, materials, regulatory, public relations, project management, and miscellaneous expenses.

15.2 Method Overview

15.2.1 Purpose

This script calculates delay costs by:

1. Loading annual delay cost components from YAML configuration
2. Summing all annual delay cost components
3. Multiplying by the number of delay years to get total nominal cost
4. Calculating present value of annual delay costs

15.2.2 Calculation Approach

The delay cost calculation follows this structure:

$$\text{Total Yearly Delay Cost} = \sum_{\text{components}} \text{Annual Component Cost} \quad (15.1)$$

$$\text{Total Delay Cost} = \text{Total Yearly Delay Cost} \times \text{Delay Years} \quad (15.2)$$

$$\text{Present Value} = \sum_{t=1}^D \frac{\text{Total Yearly Delay Cost}}{(1 + r)^t} \quad (15.3)$$

where D is the number of delay years and r is the real WACC discount rate.

15.2.3 Inputs and Outputs

Inputs:

- Annual delay cost components (legal, admin, labor, materials, regulatory, public relations, project management, miscellaneous)
- Delay years (from project timeline)
- Financing parameters (WACC for present value)

Outputs:

- Total yearly delay cost
- Total nominal delay cost
- Present value of delay costs

15.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_delay_costs`, `load_financing_details`, `load_project_technical_details` (centralized; local wrapper)
`load_project_technical_details()` returns `delay_years` only)
- `financial_utils.py` — `calculate_present_value`
- `smart_output.py` — `CTCCOutputManager` (`add_delay_costs`, `write_batch_summary`)

15.3 Key Functions

15.3.1 `load_project_technical_details()`

Local wrapper that loads project technical details via `smart_loaders.load_project_technical_details_centralized` and returns only `delay_years`.

Returns:

- `delay_year` (float): Number of delay years

15.3.2 main()

Main execution function that orchestrates the delay cost calculation and output.

Workflow:

1. Loads delay cost components from YAML
2. Loads delay years from project technical details
3. Sums all annual delay cost components
4. Calculates total nominal cost (annual cost $\times$ delay years)
5. Calculates present value of annual delay costs
6. Displays results and writes to CSV

Key Logic:

```
1 # Load annual delay cost components
2 legal = delay_cost_df["annual_delay_costs"]["legal"]
3 admin = delay_cost_df["annual_delay_costs"]["admin"]
4 labor = delay_cost_df["annual_delay_costs"]["labor"]
5 material_and_equipment = delay_cost_df["annual_delay_costs"]["
    material_and_equipment"]
6 regulatory = delay_cost_df["annual_delay_costs"]["regulatory"]
7 public_relations = delay_cost_df["annual_delay_costs"]["
    public_relations"]
8 project_management = delay_cost_df["annual_delay_costs"]["
    project_management"]
9 miscellaneous = delay_cost_df["annual_delay_costs"]["miscellaneous"]
10
11 # Sum components
12 total_yearly_delay_cost = (legal + admin + labor +
    material_and_equipment
13                             + regulatory + public_relations +
    project_management
14                             + miscellaneous)
15
16 # Calculate total nominal cost
17 total_delay_cost = total_yearly_delay_cost * delay_year
18
19 # Calculate present value (payments from year 1 for delay_year years
    )
20 total_delay_cost_pv = calculate_present_value(
21     total_yearly_delay_cost,
22     financing.wacc_real,
23     delay_year,
24 )
```

15.4 Example

The following example demonstrates a typical usage of the delay cost calculation:

```
1 # Example: 2-year delay period
2 #
3 # Inputs (from YAML):
4 #   - Legal: $50,000/year
5 #   - Admin: $30,000/year
6 #   - Labor: $100,000/year
7 #   - Materials: $20,000/year
8 #   - Regulatory: $40,000/year
9 #   - Public Relations: $25,000/year
10 #   - Project Management: $80,000/year
11 #   - Miscellaneous: $15,000/year
12 #   - Delay years: 2
13 #   - Real WACC: 3%
14 #
15 # Calculation:
16 #   Total yearly delay cost = $50k + $30k + $100k + $20k + $40k
17 #                           + $25k + $80k + $15k = $360,000/year
18 #   Total nominal cost = $360,000 * 2 = $720,000
19 #   Present value = $360,000 * PV annuity (2 years, 3%) = $697,000
20 #
21 # Output:
22 #   - Total yearly delay cost: $360,000
23 #   - Total nominal cost: $720,000
24 #   - Present value: $697,000
```

Integration Example:

```
1 # When called from ctcc.py:
2 csv_manager = CTCCOutputManager()
3 results = {
4     "total_nominal": total_delay_cost,
5     "total_afudc": 0, # Delay costs are not AFUDC-eligible
6     "total_pv": total_delay_cost_pv,
7 }
8 csv_manager.add_delay_costs(results)
9 csv_manager.write_batch_summary()
```

15.5 Key Implementation Details

15.5.1 Delay Cost Components

The script includes eight delay cost components:

- **Legal:** Legal fees and expenses

- **Admin:** Administrative costs
- **Labor:** Labor costs during delay
- **Material and Equipment:** Material and equipment holding costs
- **Regulatory:** Regulatory compliance costs
- **Public Relations:** Public relations and community engagement
- **Project Management:** Project management overhead
- **Miscellaneous:** Other delay-related expenses

15.5.2 AFUDC Treatment

Delay costs are not AFUDC-eligible because they are operating expenses during the delay period, not construction costs. AFUDC applies only to construction work in progress (CWIP).

15.5.3 Present Value Timing

Delay costs are incurred annually during the delay period (years 1 to delay_year). The present value calculation discounts these annual payments back to the base year.

15.5.4 Simplified Structure

This script has a simpler structure than other cost scripts because delay costs are straightforward: annual costs multiplied by delay years, with no complex terrain adjustments or component-level calculations.

Chapter 16

Wildfire Costs Calculation Documentation

16.1 Introduction

The `wildfire_costs.py` script calculates expected wildfire costs using a probabilistic risk approach. It models ignition rates by terrain and construction type, event severity, and applies a growing annuity formula to account for increasing risk over the project lifetime.

16.2 Method Overview

16.2.1 Purpose

This script calculates wildfire costs by:

1. Calculating segment-specific ignition rates by terrain and construction type
2. Aggregating to total annual event rate (λ)
3. Computing Expected Annual Loss (EAL) = $\lambda \times \text{severity}$
4. Calculating nominal total cost using growing annuity (accounting for risk growth)
5. Computing present value using growing annuity formula

16.2.2 Calculation Approach

The wildfire cost calculation follows this structure:

$$f_{i,t} = \text{Base Rate}_{\text{terrain}} \times \text{Construction Multiplier} \quad (16.1)$$

$$\lambda_{\text{segment}} = \text{Miles}_{\text{terrain}} \times f_{i,t} \quad (16.2)$$

$$\lambda_{\text{total}} = \sum_{\text{terrains}} \lambda_{\text{segment}} \quad (16.3)$$

$$\text{EAL} = \lambda_{\text{total}} \times \text{Severity} \quad (16.4)$$

$$\text{Nominal Total} = \text{EAL} \times \frac{(1+g)^N - 1}{g} \quad (\text{if } g \neq 0) \quad (16.5)$$

$$\text{PV} = \text{EAL} \times \frac{1 - \left(\frac{1+g}{1+d}\right)^N}{d - g} \quad (\text{if } d \neq g) \quad (16.6)$$

The PV is discounted for the delay and construction period so the cost stream begins at Commercial Operation Date (COD), not year 1; see “Payment timing (COD)” below.

where:

- $f_{i,t}$ = effective ignition rate per mile for terrain i and construction type t
- λ = annual event rate (events/year)
- g = risk growth rate (annual increase in expected losses)
- d = discount rate
- N = project lifetime

16.2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type)
- Physical details (terrain distribution)
- Wildfire parameters (ignition rates by terrain, severity, growth rate, construction multipliers)
- Discount rate source (social or wacc_real; societal perspective by default)
- Project lifetime

Outputs:

- Total annual event rate (lambda)
- Expected Annual Loss (EAL)
- Event rates by terrain segment
- Nominal total cost (growing annuity)
- Present value cost (growing annuity with discounting)

16.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_project_technical_details`, `load_physical_details`,
`load_financing_details`, `load_wildfire_costs`, `get_physical_data_raw`,
`get_financing_data_raw`, `load_terrain_miles`
- `constants.py` — `MIN_DISCOUNT_RATE`, `GROWTH_RATE_TOLERANCE`
- `financial_utils.py` — `get_discount_rate_from_config`,
`calculate_growing_annuity_pv`, `validate_discount_rate`,
`calculate_nominal_growing_series`
- `calculation_utils.py` — `build_category_string`, `normalize_construction_type_for_yaml`
- `smart_output.py` — `CTCCOutputManager` (`add_wildfire_costs`,
`write_batch_summary`)

16.3 Key Functions

16.3.1 `get_discount_rate_from_config` (from `financial_utils`)

The script uses `financial_utils.get_discount_rate_from_config(config_yaml, financing_yaml, rate_key="discount_rate_source")` to get the discount rate. In `main()`, it is called with `wildfire_yaml`, `get_financing_data_raw()`, and `rate_key="discount_rate_source"`. Returns `(discount_rate, source_description)`. Supported sources are `social` (societal perspective, default) and `wacc_real` (utility perspective).

16.3.2 `calculate_wildfire_costs(wildfire_yaml, construction_type, terrain_miles, project_lifetime, discount_rate, delay_years=0, construction_years=0)`

Calculates expected wildfire costs using segment-based ignition rates.

Parameters:

- `wildfire_yaml` (dict): Loaded wildfire YAML data
- `construction_type` (str): Construction type (overhead, underground, etc.)
- `terrain_miles` (dict): Dictionary of terrain type to miles
- `project_lifetime` (int): Project lifetime in years
- `discount_rate` (float): Discount rate for PV calculation
- `delay_years` (float): Delay years before construction (default 0)

- `construction_years` (float): Construction period years (default 0)

Returns:

- Dictionary containing:
 - `lambda_total` (float): Total annual event rate
 - `EAL` (float): Expected Annual Loss
 - `lambda_by_terrain` (dict): Event rates by terrain segment
 - `severity` (float): Mean uninsured loss per event (see “Severity definition” below)
 - `growth_rate` (float): Risk growth rate
 - `nominal_total` (float): Total nominal cost
 - `pv_cost` (float): Present value cost

Key Logic:

```

1 # Calculate segment-specific ignition rates
2 for terrain, miles in terrain_miles.items():
3     if miles > 0:
4         base_rate = ignition_rates_by_terrain.get(terrain, 0.0)
5         f_i_t = base_rate * construction_multiplier
6         lambda_segment = miles * f_i_t
7         lambda_total += lambda_segment
8
9 # Calculate EAL
10 EAL = lambda_total * severity
11
12 # Calculate nominal total (growing annuity)
13 if abs(growth_rate) < 1e-9:
14     nominal_total = EAL * project_lifetime
15 else:
16     nominal_total = EAL * ((1 + growth_rate) ** project_lifetime -
17                             1) / growth_rate
18
19 # Calculate PV (growing annuity with discounting)
20 if abs(discount_rate - growth_rate) < 1e-9:
21     pv_cost = EAL * project_lifetime
22 else:
23     pv_cost = EAL * ((1 - ((1 + growth_rate) / (1 + discount_rate))
24                     ** project_lifetime)
25                     / (discount_rate - growth_rate))

```

16.3.3 `main()`

Main execution function that orchestrates the wildfire cost calculation and output.

Workflow:

1. Loads project technical details; constructs category via `build_category_string`; loads terrain via `load_terrain_miles()`; loads wildfire_yaml via `load_wildfire_costs()`; loads financing_yaml via `get_financing_data_raw()`
2. `discount_rate, discount_source = get_discount_rate_from_config(wildfire_yaml, financing_yaml, rate_key="discount_rate_source")`
3. Calls `calculate_wildfire_costs(wildfire_yaml, construction_type, terrain_miles, project_lifetime, discount_rate, delay_years=..., construction_years=...)`
4. Displays results; `add_wildfire_costs(results); write_batch_summary()`

16.4 Example

The following example demonstrates a typical usage of the wildfire cost calculation:

```

1 # Example: 500 MW Overhead AC line
2 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
3 #
4 # Inputs (from YAML):
5 #   - Terrain distribution:
6 #     * Forested: 40 miles, base rate: 0.0001 events/mi/yr
7 #     * Farmland: 60 miles, base rate: 0.00005 events/mi/yr
8 #   - Construction multiplier: 1.0 (overhead baseline)
9 #   - Severity: $50M per event
10 #   - Risk growth rate: 2% per year
11 #   - Project lifetime: 50 years
12 #   - Discount rate: 3% (social)
13 #
14 # Calculation:
15 #   Forested segment: 40 * 0.0001 * 1.0 = 0.004 events/year
16 #   Farmland segment: 60 * 0.00005 * 1.0 = 0.003 events/year
17 #   Total lambda = 0.007 events/year
18 #   EAL = 0.007 * $50M = $350,000/year
19 #   Nominal total = $350k * ((1.02^50 - 1) / 0.02) = $28.5M
20 #   PV = $350k * ((1 - (1.02/1.03)^50) / (0.03 - 0.02)) = $12.3M
21 #
22 # Output:
23 #   - Total annual events: 0.007 events/year
24 #   - EAL: $350,000/year
25 #   - Nominal total: $28.5M
26 #   - Present value: $12.3M

```

Integration Example:

```

1 # When called from ctcc.py:
2 csv_manager = CTCCOutputManager()

```

```

3 # Results dict already has all needed values from
   calculate_wildfire_costs()
4 csv_manager.add_wildfire_costs(results)
5 csv_manager.write_batch_summary()

```

16.5 Key Implementation Details

16.5.1 Payment timing (COD)

The cost stream starts at Commercial Operation Date (COD), i.e. when the line is in operation (after delay and construction). Wildfire risk is treated as operating-period risk: the PV is discounted for the delay and construction period so the first year of expected loss is at COD, not at year 1. This matches the timing used for wildfire liability insurance (file 04).

16.5.2 Severity definition (uninsured residual)

Severity (`severity_per_event`) is the expected **uninsured** loss per wildfire event: deductibles, amounts above the liability limit, exclusions, or societal/external damages. The insured portion of wildfire risk is captured by the optional wildfire liability insurance module (file 04). Defining severity as uninsured ensures that including both this module and wildfire liability insurance does not double-count; the former is uninsured expected loss (societal or residual), the latter is the utility's cost to transfer insured risk.

16.5.3 Discount rate and perspective

The wildfire module uses the social discount rate (societal perspective) when `discount_rate_source` is `social` (the default). The resulting PV is a **societal** cost. Use `wacc_real` for a utility perspective if desired.

16.5.4 Probabilistic Risk Approach

The script uses a probabilistic approach where wildfire events are modeled as Poisson processes. The annual event rate (`lambda`) represents the expected number of events per year, and EAL represents the expected annual loss.

16.5.5 Construction Type Multipliers

Different construction types have different ignition rates. Overhead lines have the highest risk (multiplier = 1.0), while underground and subsea have much lower multipliers (e.g., 0.01 for underground).

16.5.6 Growing Annuity Formula

The script accounts for increasing wildfire risk over time using a growing annuity formula. This reflects the reality that climate change and other factors may increase wildfire frequency and severity.

16.5.7 Edge Cases

The script handles edge cases:

- If growth rate = 0: Uses simple annuity formula
- If discount rate = growth rate: Uses simplified formula ($PV = EAL \times N$)

16.5.8 AFUDC Treatment

Wildfire costs are probabilistic future losses, not capital costs. Therefore, they are not AFUDC-eligible. AFUDC applies only to construction work in progress.

16.5.9 Warning for High Growth Rates

If the risk growth rate exceeds the discount rate, the script issues a warning because the present value grows over time, indicating that risk is escalating faster than discounting.

Chapter 17

Outage Costs Calculation Documentation

17.1 Introduction

The `outage_costs.py` script calculates expected outage costs using a simplified probabilistic reliability approach. It models outage rates by terrain and construction type, applies a multiplicative duration model, and uses piecewise value of lost load (VoLL) to calculate costs.

17.2 Method Overview

17.2.1 Purpose

This script calculates outage costs by:

1. Calculating segment-specific outage rates by terrain and construction type
2. Applying duration multipliers based on construction type
3. Calculating unserved energy per event
4. Applying piecewise VoLL to calculate cost per event
5. Computing Expected Annual Cost (EAC)
6. Calculating nominal total cost using growing annuity
7. Computing present value using growing annuity formula

17.2.2 Calculation Approach

The outage cost calculation follows this structure:

$$\lambda_{segment} = \text{Miles}_{terrain} \times \text{Outage Rate}_{terrain, construction} \quad (17.1)$$

$$H_{eff} = H_{base} \times \text{Duration Multiplier}_{construction} \quad (17.2)$$

$$\text{MW Lost} = \phi \times \text{Capacity} \quad (17.3)$$

$$\text{Unserved MWh} = H_{eff} \times \text{MW Lost} \quad (17.4)$$

$$\text{Cost/Event} = \text{Piecewise VoLL}(\text{Unserved MWh}) \quad (17.5)$$

$$\text{EAC} = \sum_{\text{terrains}} \lambda_{segment} \times \text{Cost/Event} \quad (17.6)$$

$$\text{Nominal Total} = \text{EAC} \times \frac{(1+g)^N - 1}{g} \quad (17.7)$$

$$\text{PV} = \text{EAC} \times \frac{1 - \left(\frac{1+g}{1+d}\right)^N}{d - g} \quad (17.8)$$

where:

- λ = annual outage rate (outages/year)
- H_{eff} = effective outage duration (hours)
- ϕ = capacity at risk factor
- g = risk growth rate
- d = discount rate
- N = project lifetime

17.2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type, capacity, project lifetime, delay years, construction years)
- Terrain miles (`load_terrain_miles`)
- Outage parameters (outage rates, duration by terrain, duration multipliers, VoLL tiers, capacity at risk factor, growth rate)
- Discount rate (`discount_rate_type` from financing)
- Financing data (for discount rate)

Outputs:

- Total annual outage rate (lambda)

- Expected Annual Cost (EAC)
- Outage details by terrain segment
- Nominal total cost (growing annuity)
- Present value cost (growing annuity with discounting)

17.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_project_technical_details`, `load_financing_details`, `load_outage_costs`, `load_terrain_miles`, `get_financing_data_raw`
- `constants.py` — `MIN_DISCOUNT_RATE`, `GROWTH_RATE_TOLERANCE`
- `financial_utils.py` — `get_discount_rate_from_config`, `calculate_growing_annuity_pv`, `calculate_nominal_growing_series`, `validate_discount_rate`
- `calculation_utils.py` — `build_category_string`, `normalize_construction_type_for_yaml`
- `smart_output.py` — `CTCCOutputManager` (`add_outage_costs`, `write_batch_summary`)

17.3 Key Functions

17.3.1 `get_discount_rate_from_config` (from `financial_utils`)

The script uses `financial_utils.get_discount_rate_from_config(config_yaml, financing_yaml, rate_key="discount_rate_type")`. In `main()`, it is called with `outage_yaml`, `get_financing_data_raw()`, and `rate_key="discount_rate_type"`. Returns `(discount_rate, source_description)`.

17.3.2 `calculate_voll_cost_pieewise(duration_hours, mw_lost, tiers)`

Calculates total cost using piecewise VoLL with tiered pricing.

Parameters:

- `duration_hours` (float): Duration of outage in hours
- `mw_lost` (float): MW capacity lost during outage
- `tiers` (list): List of tier dicts with `max_hours` and `value_per_mwh`

Returns:

- `total_cost` (float): Total cost for the outage

Key Logic:

```
1 # Piecewise VoLL: Hours 0-4 at tier 1, hours 4-24 at tier 2, hours
   24+ at tier 3
2 total_cost = 0
3 hours_remaining = duration_hours
4 prev_threshold = 0
5
6 for tier in tiers:
7     max_hours = float("inf") if tier["max_hours"] == ".inf" else
       tier["max_hours"]
8     value_per_mwh = tier["value_per_mwh"]
9     hours_in_tier = min(hours_remaining, max_hours - prev_threshold)
10    if hours_in_tier > 0:
11        mwh_in_tier = hours_in_tier * mw_lost
12        total_cost += mwh_in_tier * value_per_mwh
13        hours_remaining -= hours_in_tier
14        prev_threshold = max_hours
15    if hours_remaining <= 0:
16        break
```

17.3.3 calculate_outage_costs(outage_yaml, construction_type, terrain_miles, capacity_mw, project_lifetime, discount_rate)

Calculates expected outage costs using simplified outage rate model.

Parameters:

- outage_yaml (dict): Loaded outage YAML data
- construction_type (str): Construction type (overhead, underground, subsea)
- terrain_miles (dict): Dictionary of terrain type to miles
- capacity_mw (float): Project capacity in MW
- project_lifetime (int): Project lifetime in years
- discount_rate (float): Discount rate for PV calculation
- delay_years (float, optional): Years of delay before construction starts
- construction_years (float, optional): Years of construction (PV annuity starts after delay + construction)

Returns:

- Dictionary containing:

- `lambda_total` (float): Total annual outage rate
- `EAC` (float): Expected Annual Cost
- `outage_by_terrain` (dict): Outage details by terrain segment
- `capacity_at_risk` (float): Capacity at risk factor
- `growth_rate` (float): Risk growth rate
- `nominal_total` (float): Total nominal cost
- `pv_cost` (float): Present value cost

Key Logic:

```

1 yml_construction_type = normalize_construction_type_for_yaml(
    construction_type)
2
3 for terrain, miles in terrain_miles.items():
4     if miles > 0:
5         outage_rate = outage_rates[yml_construction_type].get(
            terrain, 0.0)
6         lambda_segment = miles * outage_rate
7
8         H_base = duration_by_terrain.get(terrain, 0)
9         duration_mult = duration_multiplier.get(
            yml_construction_type, 1.0)
10        H_eff = H_base * duration_mult
11
12        mw_lost_per_event = capacity_at_risk * capacity_mw
13        unserved_mwh_per_event = H_eff * mw_lost_per_event
14        cost_per_event = calculate_voll_cost_pieewise(
15            H_eff, mw_lost_per_event, voll_tiers
16        )
17        annual_cost = lambda_segment * cost_per_event
18        # ... store in outage_by_terrain, accumulate lambda_total,
        EAC
19
20 nominal_total = calculate_nominal_growing_series(...)
21 pv_cost = calculate_growing_annuity_pv(..., delay_years=...,
    construction_years=...)

```

17.3.4 main()

Main execution function that orchestrates the outage cost calculation and output.

17.4 Example

The following example demonstrates a typical usage of the outage cost calculation:

```

1 # Example: 500 MW Overhead AC line
2 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
3 #
4 # Inputs (from YAML):
5 #   - Terrain distribution:
6 #     * Forested: 40 miles, outage rate: 0.05 outages/mi/yr
7 #     * Farmland: 60 miles, outage rate: 0.02 outages/mi/yr
8 #   - Duration: 8h (forested), 4h (farmland)
9 #   - Duration multiplier: 1.0 (overhead)
10 #   - Capacity at risk: 50%
11 #   - VoLL tiers:
12 #     * 0-4h: $10,000/MWh
13 #     * 4-24h: $5,000/MWh
14 #     * 24h+: $2,000/MWh
15 #   - Risk growth rate: 1% per year
16 #   - Project lifetime: 50 years
17 #   - Discount rate: 3% (social)
18 #
19 # Calculation:
20 #   Forested segment:
21 #     Outages/year = 40 * 0.05 = 2.0
22 #     Duration = 8h * 1.0 = 8h
23 #     MW lost = 0.5 * 500 = 250 MW
24 #     Unserved MWh = 8 * 250 = 2000 MWh
25 #     Cost/event = (4h * 250MW * $10k) + (4h * 250MW * $5k) = $15M
26 #     Annual cost = 2.0 * $15M = $30M/year
27 #
28 #   Farmland segment:
29 #     Outages/year = 60 * 0.02 = 1.2
30 #     Duration = 4h * 1.0 = 4h
31 #     MW lost = 250 MW
32 #     Unserved MWh = 4 * 250 = 1000 MWh
33 #     Cost/event = 4h * 250MW * $10k = $10M
34 #     Annual cost = 1.2 * $10M = $12M/year
35 #
36 #   Total EAC = $42M/year
37 #   Nominal total = $42M * ((1.01^50 - 1) / 0.01) = $2.7B
38 #   PV = $42M * ((1 - (1.01/1.03)^50) / (0.03 - 0.01)) = $1.2B
39 #
40 # Output:
41 #   - Total outages/year: 3.2 events/year
42 #   - EAC: $42M/year
43 #   - Nominal total: $2.7B
44 #   - Present value: $1.2B

```

Integration Example:

```

1 # When called from ctcc.py:

```

```

2 csv_manager = CTCCOutputManager()
3 # Results dict already has all needed values
4 csv_manager.add_outage_costs(results)
5 csv_manager.write_batch_summary()

```

17.5 Key Implementation Details

17.5.1 Piecewise Value of Lost Load (VoLL)

The script uses a tiered VoLL structure where different outage durations have different values:

- **Tier 1** (0-4 hours): Highest value (e.g., \$10,000/MWh) - immediate impacts
- **Tier 2** (4-24 hours): Medium value (e.g., \$5,000/MWh) - short-term impacts
- **Tier 3** (24+ hours): Lower value (e.g., \$2,000/MWh) - longer-term impacts

This reflects the reality that shorter outages have higher economic impact per MWh.

17.5.2 Multiplicative Duration Model

The effective duration is calculated as:

$$H_{eff} = H_{base} \times \text{Duration Multiplier}_{construction}$$

Different construction types have different duration multipliers. For example, underground lines may have shorter repair times (lower multiplier) than overhead lines.

17.5.3 Capacity at Risk Factor

The capacity at risk factor (ϕ) represents the fraction of project capacity that is lost during an outage. This accounts for partial outages and system redundancy.

17.5.4 Growing Annuity Formula

Similar to wildfire costs, the script accounts for increasing outage risk over time using a growing annuity formula. Present value uses `calculate_growing_annuity_pv` with `delay_years` and `construction_years` so the annuity stream starts after construction (delay + construction years).

17.5.5 AFUDC Treatment

Outage costs are probabilistic future losses, not capital costs. Therefore, they are not AFUDC-eligible.

Chapter 18

Congestion and Curtailment Benefits Calculation Documentation

18.1 Introduction

The `congestion_curtailment_reduction.py` script calculates the benefits from reducing transmission congestion and renewable energy curtailment. It allocates effective capacity relief between curtailment and congestion to avoid double-counting, then monetizes both benefits over the project lifetime.

18.2 Method Overview

18.2.1 Purpose

This script calculates congestion and curtailment benefits by:

1. Determining effective capacity relief (greenfield vs. reconductoring)
2. Allocating capacity relief between curtailment and congestion
3. Calculating congestion reduction energy and costs
4. Calculating curtailment reduction energy and benefits
5. Applying saturation factors (haircuts) to account for diminishing returns
6. Computing residual exceedance costs (unmet capacity needs; one cost from congestion and/or curtailment)
7. Calculating present values of all benefits and costs

18.2.2 Calculation Approach

The calculation follows this structure:

Effective Capacity Relief:

$$\Delta C_{eff} = \begin{cases} \phi \times \text{Capacity} & (\text{Greenfield}) \\ \text{Capacity} - \text{Old Capacity} & (\text{Reconductoring}) \end{cases} \quad (18.1)$$

where ϕ is the flow factor.

Curtailment Benefits:

$$E_{curt} = H_c \times \min(\Delta C_{eff}, C_c) \quad (18.2)$$

$$\text{Benefit}_{curt} = E_{curt} \times \lambda_c \quad (18.3)$$

where H_c is curtailment hours, C_c is average curtailment MW, and λ_c is curtailment price.

Congestion Benefits:

$$E_{cong} = E_{cong,bc} + E_{cong,non} + E_{near} \quad (18.4)$$

$$\text{Benefit}_{cong} = E_{cong} \times \lambda_{cong} \times (1 - \text{saturation}) \quad (18.5)$$

where the energy is split between overlap hours (with curtailment) and non-overlap binding hours.

Residual Exceedance: Residual exceedance is one cost that can have two sources—congestion binding hours and/or curtailment hours—so it works for congestion-only, curtailment-only, or both. When only one of congestion or curtailment is used, the other term is zero; when both are used, the user should set `residual_exceedance_value` if the single default (\$/MWh) is not appropriate.

$$E_{res,cong} = (H_{bc} + H_{bnon}) \times \max(0, X_{cong} - \Delta C_{eff}) \quad (18.6)$$

$$E_{res,curt} = H_c \times \max(0, C_c - \Delta C_{eff}) \quad (18.7)$$

$$E_{residual} = E_{res,cong} + E_{res,curt} \quad (18.8)$$

$$\text{Cost}_{residual} = E_{residual} \times \lambda, \quad \lambda = \text{residual_exceedance_value if set, else } \lambda_{cong} \quad (18.9)$$

18.2.3 Inputs and Outputs

Inputs:

- Project technical details (capacity, old capacity, reconductoring status)
- Congestion parameters (binding hours, average exceedance, flow factor, saturation factor, congestion price; `residual_exceedance_value` optional, null = use average congestion price)
- Curtailment parameters (curtailment hours, average curtailment MW, curtailment price, saturation factor)

- Financing parameters (WACC, timeline)

Outputs:

- Annual and lifetime congestion benefits (raw and haircut)
- Annual and lifetime curtailment benefits (raw and haircut)
- Residual exceedance costs
- Congestion and curtailment costs during delay/construction (allocated, no double count)
- Present values of all metrics

18.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_congestion_curtailment_reductions,`
`load_project_technical_details,` `load_financing_details,`
`get_project_data_raw`
- `yaml_loaders.py` — `ProjectTechnicalDetails` (via centralized loader)
- `financial_utils.py` — `calculate_present_value,` `calculate_cod_year`
- `constants.py` — `MIN_DISCOUNT_RATE`
- `smart_output.py` — `CTCCOutputManager` (`add_congestion_curtailment,`
`write_batch_summary`)

18.3 Key Functions

18.3.1 `allocate_curtailment_then_congestion(DC_eff_mw, ...)`

Allocates effective capacity relief between curtailment and congestion to avoid double-counting.

Parameters:

- `DC_eff_mw` (float): Effective capacity relief in MW
- `binding_hours_total` (float): Total binding constraint hours
- `average_exceedance_mw` (float): Average exceedance in MW
- `curtailment_hours_total` (float): Total curtailment hours
- `average_curtailment_mw` (float): Average curtailment in MW
- `average_curtailment_price` (float): Average curtailment price

Returns:

- Dictionary with curtailment benefits, overlap metrics, and remaining capacity for congestion

Key Logic:

```
1 Hb = max(0.0, binding_hours_total)
2 Hc = max(0.0, curtailment_hours_total)
3 # Curtailment relief (MWh/yr)
4 E_curt = Hc * min(DeltaC_eff_mw, Cc)
5 curt_benefit = E_curt * lambda_c
6
7 if Hb <= 0:
8     return {..., "H_bc": 0, "H_bnon": 0, "DeltaC_remain_bc_mw":
9         DeltaC_eff_mw}
10
11 # Overlap proxy between binding and curtailment time
12 theta = min(1.0, Hc / Hb) if Hc > 0 else 0.0
13 H_bc = theta * Hb
14 H_bnon = (1.0 - theta) * Hb
15 DeltaC_used_bc = min(DeltaC_eff_mw, Cc)
16 DeltaC_remain_bc = max(0.0, DeltaC_eff_mw - DeltaC_used_bc)
17 # Returns E_curt_mwh_yr, curtailment_benefit_$_yr, theta_overlap,
18     H_bc, H_bnon, DeltaC_remain_bc_mw
```

18.3.2 calculate_congestion_reduction_costs(...)

Main function that calculates congestion and curtailment benefits.

Parameters:

- Multiple parameters including reconductoring status, capacities, congestion/curtailment parameters, financing parameters

Returns:

- Dictionary with all calculated benefits, costs, and present values

Key Logic:

```
1 if not reconductoring:
2     effective_capacity_relief = max(0, flow_factor * capacity_mw)
3 else:
4     effective_capacity_relief = max(0, capacity_mw - old_capacity_mw
5         )
6 alloc = allocate_curtailment_then_congestion(DeltaC_eff_mw=
7     effective_capacity_relief, ...)
8 DeltaC_rem = alloc["DeltaC_remain_bc_mw"]
```

```

8 H_bc, H_bnon = alloc["H_bc"], alloc["H_bnon"]
9
10 E_cong_bc = H_bc * min(DeltaC_rem, average_exceedance)
11 E_cong_non = H_bnon * min(effective_capacity_relief,
    average_exceedance)
12 E_near = near_binding_hours * min(effective_capacity_relief,
    near_average_exceedance)
13 energy_congestion_reduction = E_cong_bc + E_cong_non + E_near
14
15 annual_congestion_reduction_cost_raw = energy_congestion_reduction *
    average_congestion_price
16 annual_congestion_reduction_cost_haircut = (1 - saturation_factor) *
    annual_congestion_reduction_cost_raw
17
18 total_binding_hours = H_bc + H_bnon
19 energy_residual_congestion = total_binding_hours * max(0,
    average_exceedance - effective_capacity_relief)
20 energy_residual_curtailment = curtailment_hours_total * max(0,
    average_curtailment_mw - effective_capacity_relief)
21 energy_residual_exceedance = energy_residual_congestion +
    energy_residual_curtailment
22 residual_exceedance_value_used = residual_exceedance_value or
    average_congestion_price
23 annual_residual_exceedance_cost = energy_residual_exceedance *
    residual_exceedance_value_used

```

18.3.3 main()

Main execution function that orchestrates the calculation and output.

18.4 Example

The following example demonstrates a typical usage:

```

1 # Example: 500 MW Greenfield Overhead AC line
2 #
3 # Inputs (from YAML):
4 #   - Capacity: 500 MW
5 #   - Flow factor: 0.7
6 #   - Binding hours: 2000 hours/year
7 #   - Average exceedance: 300 MW
8 #   - Curtailment hours: 1500 hours/year
9 #   - Average curtailment: 200 MW
10 #   - Congestion price: $50/MWh
11 #   - Curtailment price: $30/MWh
12 #   - Saturation factor: 0.2 (20% haircut)

```

```

13 # - Project lifetime: 50 years
14 #
15 # Calculation:
16 #   Effective capacity relief = 0.7 * 500 = 350 MW
17 #
18 #   Curtailment:
19 #     E_curt = 1500 * min(350, 200) = 300,000 MWh/year
20 #     Benefit = 300,000 * $30 = $9M/year
21 #
22 #   Congestion (after curtailment allocation):
23 #     Overlap theta = min(1.0, 1500/2000) = 0.75
24 #     H_bc = 0.75 * 2000 = 1500 hours
25 #     H_bnon = 0.25 * 2000 = 500 hours
26 #     DC_remain = 350 - 200 = 150 MW
27 #     E_cong_bc = 1500 * min(150, 300) = 225,000 MWh/year
28 #     E_cong_non = 500 * min(350, 300) = 150,000 MWh/year
29 #     E_cong = 375,000 MWh/year
30 #     Benefit_raw = 375,000 * $50 = $18.75M/year
31 #     Benefit_haircut = (1 - 0.2) * $18.75M = $15M/year
32 #
33 #   Residual exceedance (congestion contribution here; curtailment-
34 #   only: E_res,cong=0,
35 #   E_res,curt = H_c * max(0, C_c - Delta_C_eff)):
36 #   E_residual = 1500 * max(0, 300-150) + 500 * max(0, 300-350)
37 #               = 225,000 MWh/year
38 #   Cost = 225,000 * $50 = $11.25M/year
39 #
40 # Output:
41 #   - Annual curtailment benefit: $9M
42 #   - Annual congestion benefit (haircut): $15M
43 #   - Annual residual exceedance cost: $11.25M
44 #   - Net annual benefit: $12.75M

```

Integration Example:

```

1 # When called from ctcc.py (main() builds results from
2   CongestionReductionResults):
3 csv_manager = CTCCOutputManager()
4 results = {
5     "congestion_benefit_annual": congestion_results.
6       annual_congestion_reduction_cost_raw,
7     "congestion_benefit_nominal": congestion_results.
8       lifetime_congestion_reduction_cost,
9     "congestion_benefit_pv": congestion_results.
10       lifetime_congestion_reduction_cost_pv,
11     "curtailment_benefit_annual": congestion_results.
12       annual_curtailment_benefit,
13     "curtailment_benefit_pv": congestion_results.

```

```

    lifetime_curtailment_benefit_pv,
9    "residual_exceedance_annual": congestion_results.
      annual_residual_exceedance_cost,
10    "residual_exceedance_pv": congestion_results.
      lifetime_residual_exceedance_cost_pv,
11    # ... delay costs, haircut variants, physical metrics
12 }
13 csv_manager.add_congestion_curtailment(results)
14 csv_manager.write_batch_summary()

```

18.5 Key Implementation Details

18.5.1 Double-Counting Avoidance

The script prevents double-counting by allocating capacity relief first to curtailment, then to congestion. The overlap factor (θ) accounts for hours when both curtailment and congestion occur. This same allocation is used for delay-period opportunity costs.

18.5.2 Saturation Factors

Saturation factors (haircuts) account for diminishing returns as more capacity is added. They reduce the monetized benefits to reflect that not all theoretical benefits are realized.

18.5.3 Residual Exceedance

The script calculates residual exceedance costs, representing unmet capacity when effective relief is less than the constraint (exceedance or curtailment). Residual exceedance can come from (1) congestion binding hours, (2) curtailment hours, or both. One \$/MWh price is applied (default: average congestion price; the user can set `residual_exceedance_value` when both apply). This is important for partial solutions (e.g., reconductoring or a line that does not fully relieve curtailment). In CTCC, residual exceedance is a system/societal cost categorized under energy/emissions (not under the utility or ratepayer perspective).

18.5.4 Flow Factor

For greenfield projects, the flow factor (ϕ) represents the fraction of nameplate capacity that effectively relieves congestion, accounting for network constraints and power flow physics.

18.5.5 Near-Binding Hours

The script includes near-binding hours in a way that is methodologically consistent with binding hours: relief is capped by `near_average_exceedance`. The formula is $E_{near} = H_n \times \min(\Delta C_{eff}, X_n)$ where H_n is near-binding hours and X_n is near-average exceedance (MW).

18.5.6 Delay and Construction Period Costs

The script calculates delay-period opportunity costs using the same curtailment-first allocation as operations, so overlapping hours are not double counted. This represents the value of forgone relief while the project is delayed. Present values use `financial_utils.calculate_present_value` with `start_year=calculate_cod_year(delay_years, construction_years)` for benefits (annuity starts at COD); delay/construction costs use `start_year=1` over `delay_years + construction_years`.

Chapter 19

Benefit of Delivered Energy

19.1 Introduction

The benefit of delivered energy is the societal value of the energy the project enables to be delivered each year. For greenfield projects, deliverable capacity equals effective capacity (flow factor times nameplate). For reconductoring, deliverable capacity is the additional capacity the upgrade enables (new minus old), capturing the value of additional energy the upgrade enables, including any headroom beyond clearing the constraint. This benefit is not double-counted with congestion or curtailment: those value relief of constraints; this values throughput. It is not part of rate base or AFUDC.

19.2 Method

19.2.1 Variables

Variable	Meaning / units	Notes
$\Delta C_{\text{effective}}$	Effective capacity relief, (MW)	From system constraints. Greenfield: ϕC_{new} ; reconductoring: $C_{\text{new}} - C_{\text{old}}$.
u	Line utilization, (dimensionless)	$0 \leq u \leq 1$.
H	Hours per year, (h/year)	e.g. 8760.
$\gamma_{\text{electricity}}$	Electricity price, (\$/MWh)	e.g. baseline or market price.
$E_{\text{delivered,annual}}$	Deliverable energy per year, (MWh/year)	
$B_{\text{delivered,annual}}$	Annual benefit from delivered energy, (\$/year)	
T_{lifetime}	Project lifetime, (years)	
T_{COD}	Commercial operation date (year index), (years)	First year of operation; benefit stream starts here.
$r_{\text{WACC,real}}$	Real WACC, (decimal)	Used to discount benefits.
$B_{\text{delivered,lifetime,nominal}}$	Nominal lifetime benefit from delivered energy, (\$)	
$B_{\text{delivered,lifetime,real}}$	Real (PV) lifetime benefit from delivered energy, (\$)	

Table 19.1: Variables for the benefit of delivered energy.

19.2.2 Equations

1. Deliverable energy (annual). Deliverable energy per year is deliverable capacity times line utilization times hours per year:

$$E_{\text{delivered,annual}} = \Delta C_{\text{effective}} \times u \times H.$$

2. Annual benefit. Annual benefit is deliverable energy valued at the electricity price:

$$B_{\text{delivered,annual}} = E_{\text{delivered,annual}} \times \gamma_{\text{electricity}}.$$

3. Nominal lifetime benefit.

$$B_{\text{delivered,lifetime,nominal}} = B_{\text{delivered,annual}} \times T_{\text{lifetime}}.$$

4. Real (present value) lifetime benefit. Level annual benefit from T_{COD} for T_{lifetime} years, discounted at $r_{\text{WACC,real}}$:

$$B_{\text{delivered,lifetime,real}} = \sum_{t=0}^{T_{\text{lifetime}}-1} \frac{B_{\text{delivered,annual}}}{(1 + r_{\text{WACC,real}})^{T_{\text{COD}}+t}}.$$

19.3 Implementation

This benefit is implemented as a distinct block in the same script that computes congestion and curtailment benefits. It is reported as its own line item in CSV and JSON outputs and is included in total benefits. Full notation and definitions are in the paper appendix, Section 10 (Benefit of delivered energy).

Chapter 20

Energy Losses Calculation Documentation

20.1 Introduction

The `energy_losses.py` script calculates transmission line losses and converter losses for a given project configuration. It exposes `get_total_energy_losses()` as the main entry point (used by `line_loss_costs.py` and `emissions.py`) and can be run standalone to print results. For reconductoring projects it uses the old configuration's voltage (towers unchanged) and the new configuration's resistance (new conductors).

20.2 Method Overview

Canonical notation. The canonical variable names and equations for energy losses are defined in `COST METHODOLOGY.md` under “Energy/Emissions” / “Energy losses”. Key quantities: C_{new} , V , L , u , phase current I , full-load adjustment f_{load} , line loss $P_{\text{lineloss,MW}}$, converter loss $P_{\text{converterloss,MW}}$, annual and lifetime energy losses E_{loss} , $E_{\text{totalloss,lifetime}}$.

20.2.1 Purpose

This script calculates energy losses by:

1. Loading project technical and physical details
2. Building category identifier (`build_category_string`); for reconductoring, also building old category for voltage lookup
3. Loading circuit and resistance details for the (new and, if reconductoring, old) category
4. For reconductoring: using old voltage and new resistance; for greenfield: using new config throughout
5. Calculating phase current (`calculate_phase_current`), full load adjustment (`full_load_adjusted`)

6. Calculating line losses (`calculate_line_losses`) and converter losses (`calculate_converter_losses`; DC only)
7. Returning a dict of losses and inputs; standalone run displays results via `print_results`

20.2.2 Calculation Approach

The energy loss calculation follows this structure:

Line Losses:

$$I_{phase} = \frac{P}{\sqrt{3} \times V \times N_{circuits}} \quad (\text{AC}) \quad (20.1)$$

$$I_{phase} = \frac{P}{V \times N_{poles}} \quad (\text{DC}) \quad (20.2)$$

$$P_{loss} = I^2 \times R \times N_{phases} \times N_{circuits} \quad (20.3)$$

$$\text{Losses (MWh/year)} = P_{loss} \times \text{Full Load Adj} \times 8760 \quad (20.4)$$

Converter Losses (DC only):

$$P_{converter} = P \times \text{Utilization} \times \text{Loss Rate} \quad (20.5)$$

$$\text{Converter Losses (MWh/year)} = P_{converter} \times 8760 \quad (20.6)$$

20.2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type, AC/DC, capacity, conductor type, converter type, line utilization, converter loss percentage; for reconductoring: old capacity, old conductor, old AC/DC)
- Physical details (line length via `load_physical_details`)
- Circuit and resistance details (via `load_circuit_and_resistance_details` for category; YAML keys: voltage, conductors per phase, phases, circuits/poles, AC_75_resistance, DC_20_resistance)

Outputs:

- Line losses (MW/mile, MWh/year, percentage)
- Converter losses (MW, MWh/year, percentage)
- Total losses (line + converter)
- Lifetime losses

20.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_project_technical_details`, `load_physical_details`, `load_circuit_and_resistance_details`, `get_project_data_raw`
- `calculation_utils.py` — `build_category_string`, `get_converter_type`, `calculate_phase_current`, `full_load_adjusted`, `calculate_line_losses`, `calculate_converter_losses`, `to_percent`
- `path_config.py` — `YAMLS_DIR` (for loader path context)
- `line_loss_costs.py` and `emissions.py` — import `get_total_energy_losses()` to obtain loss data

20.3 Key Functions

20.3.1 `get_total_energy_losses()`

Returns a dictionary of line and converter losses and inputs for the current project. For reconductoring, uses old configuration's voltage and new configuration's resistance.

Returns:

- `losses_mwh_per_year`, `lifetime_losses_mwh`, `losses_mw_per_mile`, `resistance_per_mile`, `line_loss_per_mile_percent`, `total_line_loss_mw`, `total_line_loss_percent`
- `total_converter_losses_mw`, `total_converter_losses_mwh`, `converter_loss_percent`
- `total_losses_mwh_per_year`, `project_lifetime`, `voltage_kv`, `phase_current`, `full_load_adj`, `line_length`, and other circuit parameters

Key Logic:

```
1 new_category = build_category_string(project_details=project_details
2 )
3 if project_details.reconductoring:
4     old_category = build_category_string(construction_type,
5     old_ac_dc, old_capacity, old_conductor, old_converter)
6     old_circuit = load_circuit_and_resistance_details(old_category)
7     voltage_kv = old_circuit.voltage_kv
8     new_circuit = load_circuit_and_resistance_details(new_category)
9     # use new_circuit for resistance and counts
10 else:
11     circuit = load_circuit_and_resistance_details(new_category)
12     voltage_kv = circuit.voltage_kv
13     # ...
```

```

12
13 phase_current = calculate_phase_current(capacity_mw, voltage_kv,
14     number_of_phases, number_of_circuits_poles, ac_dc)
15 full_load_adj = full_load_adjusted(project_details.line_utilization)
16 # calculate_line_losses(...), calculate_converter_losses(...,
17     converter_loss_percentage=project_details.
18     converter_loss_percentage)

```

20.3.2 print_results(...)

Prints organized results in sections (system configuration, line losses, converter losses, summary). Used by main() when run standalone.

20.3.3 main()

Calls get_total_energy_losses(), then print_results(...) with the returned dict and project details.

20.4 Example

The following example demonstrates a typical usage:

```

1 # Example: 500 MW Overhead AC line
2 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
3 #
4 # Inputs (from YAML):
5 #   - Capacity: 500 MW
6 #   - Voltage: 345 kV
7 #   - Conductors per phase: 2
8 #   - Number of phases: 3
9 #   - Number of circuits: 1
10 #   - AC resistance (75 C): 0.05 ohms/mile
11 #   - Line utilization: 70%
12 #   - Line length: 100 miles
13 #   - Project lifetime: 50 years
14 #
15 # Calculation:
16 #   Phase current = 500 MW / (sqrt(3) * 345 kV * 1) = 837 A
17 #   Full load adj = 0.7^2 = 0.49
18 #   Line loss = (837^2 * 0.05 * 3 * 1) * 0.49 = 51.4 MW
19 #   Line loss % = 51.4 / 500 = 10.3%
20 #   Losses MWh/year = 51.4 * 8760 = 450,000 MWh/year
21 #   Lifetime losses = 450,000 * 50 = 22.5M MWh
22 #
23 # Output:
24 #   - Line loss: 0.514 MW/mile

```

```

25 # - Line loss %: 10.3%
26 # - Losses MWh/year: 450,000
27 # - Lifetime losses: 22.5M MWh

```

Integration Example:

```

1 # Used by line_loss_costs.py and emissions.py:
2 from energy_losses import get_total_energy_losses
3
4 loss_data = get_total_energy_losses()
5 losses_mwh_per_year = loss_data["losses_mwh_per_year"]
6 lifetime_losses_mwh = loss_data["lifetime_losses_mwh"]
7 total_losses_mwh_per_year = loss_data["total_losses_mwh_per_year"]
8 # ... other keys as needed

```

20.5 Key Implementation Details

20.5.1 AC vs. DC Calculations

The script handles both AC and DC systems:

- **AC:** Uses $\sqrt{3}$ factor, resistance at 75°C
- **DC:** No $\sqrt{3}$ factor, resistance at 20°C

20.5.2 Full Load Adjustment

The full load adjustment accounts for the fact that losses are proportional to current squared, and average current is less than peak current due to utilization. The adjustment is typically u^2 where u is the utilization factor.

20.5.3 Converter Losses

Converter losses only apply to DC projects and are calculated in `calculation_utils.calculate_converter_losses` based on number of converters, converter type, line utilization, capacity, and `converter_loss_percentage` from project details.

20.5.4 Resistance Values

The script uses different resistance values:

- **AC:** Resistance at 75°C (operating temperature)
- **DC:** Resistance at 20°C (reference temperature)

20.5.5 Multiple Circuits/Poles

The script accounts for multiple circuits (AC) or poles (DC) by multiplying losses by the number of circuits/poles.

20.5.6 Reconductoring: Old Voltage, New Resistance

For reconductoring projects, physical towers (and voltage clearances) are unchanged, so the script uses the *old* configuration's voltage for phase current. Conductor resistance comes from the *new* configuration (new conductors). Old category is built from `old_capacity_mw`, `old_conductor_type`, `old_ac_dc`, and `get_converter_type(old_ac_dc, ...)`.

20.5.7 Standalone vs. Library Use

This script can be run standalone (displays results via `print_results`) or imported for `get_total_energy_losses()`. The underlying calculation functions (phase current, full load adjustment, line losses, converter losses) live in `calculation_utils.py` for reuse.

Chapter 21

Emissions Costs Calculation Documentation

21.1 Introduction

The `emissions.py` script calculates the societal costs of emissions due to line losses being compensated by generators. When transmission lines lose energy, generators must produce additional energy to compensate, resulting in additional emissions (CO₂, SO_x, NO_x) that have societal costs.

21.2 Method Overview

21.2.1 Purpose

This script calculates emissions costs by:

1. Calculating total energy losses (line losses + converter losses)
2. Determining total energy compensated (TEC) = losses × compensation percentage
3. Projecting energy mix evolution over project lifetime
4. Calculating emissions by pollutant for each year
5. Applying societal costs per kg of pollutant
6. Computing present value of lifetime emissions costs

21.2.2 Calculation Approach

The emissions cost calculation follows this structure:

$$\text{TEC} = \alpha \times \text{Total Energy Losses} \quad (21.1)$$

$$\text{Emissions}_k = \text{TEC} \times \sum_j (p_j \times I_{j,k}) \quad (21.2)$$

$$\text{Cost}_k = \text{Emissions}_k \times c_k \quad (21.3)$$

$$\text{Total Cost} = \sum_k \text{Cost}_k \quad (21.4)$$

$$\text{PV} = \sum_{t=1}^N \frac{\text{Annual Cost}_t}{(1 + r)^{t+\text{start_year}-1}} \quad (21.5)$$

where:

- α = compensation percentage (fraction of losses compensated)
- p_j = percentage of energy from source j in year t
- $I_{j,k}$ = emission intensity of pollutant k from source j (kg/MWh)
- c_k = societal cost per kg of pollutant k
- r = social discount rate

21.2.3 Inputs and Outputs

Inputs:

- Total energy losses (from `energy_losses.py`)
- Compensation percentage (fraction of losses compensated by generators)
- Energy source mix (initial percentages and growth/decay rates)
- Emission intensities by source and pollutant
- Societal costs per kg of pollutant
- Social discount rate
- Project lifetime, delay years, construction years

Outputs:

- Average annual emissions by pollutant (CO₂, SO_x, NO_x)
- Average annual costs by pollutant
- Lifetime total emissions and costs
- Present value of lifetime emissions costs

21.2.4 Integration with Other Scripts

This script integrates with:

- `energy_losses.py` — `get_total_energy_losses()` (loss data),
`load_project_technical_details`
- `smart_loaders.py` — `load_emissions_details`, `load_financing_social_discount_rate`,
`get_project_data_raw`
- `financial_utils.py` — `calculate_present_value`, `calculate_cod_year`
- `calculation_utils.py` — `from_percent`
- `smart_output.py` — `CTCCOutputManager` (`add_emissions_costs`,
`write_batch_summary`)

21.3 Key Functions

21.3.1 Loss data from `energy_losses.get_total_energy_losses()`

The script does not compute losses itself; it calls `get_total_energy_losses()` from `energy_losses.py` and uses:

- `loss_data["total_losses_mwh_per_year"]` — total line + converter losses (MWh/year)
- `loss_data["project_lifetime"]` — project lifetime (years)

Project delay/construction years come from `load_project_technical_details()`.

21.3.2 `calculate_energy_mix_by_year(energy_source_mix_details, project_lifetime)`

Calculates energy mix for each year with growth/decay rates.

Parameters:

- `energy_source_mix_details` (dict): Dictionary with percentage and `rate_of_change` for each source
- `project_lifetime` (int): Number of years in project

Returns:

- `energy_mix_by_year` (list): List of dictionaries, one per year, with normalized percentages

Key Logic:

```

1 # sources: coal, oil, natural_gas, solar, wind, hydro, nuclear,
  other
2 # Initial mix and rates from energy_source_mix_details (percentage,
  rate_of_change)
3 initial_mix[source] = from_percent(details.get(source, {}).get("
  percentage", 0.0))
4 rates[source] = details.get(source, {}).get("rate_of_change", 0.0)
5 for year in range(1, project_lifetime + 1):
6     next_mix[source] = current_mix[source] * (1 + rates[source])
7     total = sum(next_mix.values())
8     normalized_mix = {s: next_mix[s] / total for s in sources} if
        total > 0 else next_mix
9     energy_mix_by_year.append(normalized_mix)
10    current_mix = normalized_mix

```

21.3.3 calculate_emissions_by_year(tec, energy_mix, emission_intensities)

Calculates emissions for a single year.

Parameters:

- **tec** (float): Total energy compensated in MWh
- **energy_mix** (dict): Dictionary of energy source percentages for the year
- **emission_intensities** (dict): Dictionary of emission intensities by pollutant and source

Returns:

- **emissions** (dict): Emissions in kg for each pollutant (CO₂, SO_x, NO_x)

Key Logic:

```

1 # Intensity keys: co2_intensity_kg_per_mwh, sox_intensity_kg_per_mwh
  , nox_intensity_kg_per_mwh
2 # Each maps source -> kg/MWh
3 intensity_key_map = {"co2": "co2_intensity_kg_per_mwh", "sox": "...",
  , "nox": "..."}
4 for pollutant in ["co2", "sox", "nox"]:
5     intensity_dict = emission_intensities.get(intensity_key_map[
        pollutant], {})
6     for source in sources:
7         p_j = energy_mix.get(source, 0.0)
8         I_jk = intensity_dict.get(source, 0.0)
9         emissions[pollutant] += total_energy_compensated_mwh * p_j *
            I_jk

```

21.3.4 calculate_lifetime_emissions(...)

Calculates emissions across project lifetime.

Parameters:

- `total_losses_mwh_per_year` (float): Total energy losses per year
- `compensation_percent` (float): Fraction of losses compensated
- `energy_source_mix_details` (dict): Initial mix and growth rates
- `emission_intensities` (dict): Emission intensities by source and pollutant
- `societal_costs` (dict): Costs per kg of pollutant
- `project_lifetime` (int): Project lifetime
- `social_discount_rate` (float): Social discount rate
- `delay_years` (float): Delay years
- `construction_years` (float): Construction years

Returns:

- Multiple values including yearly emissions, total emissions, average annual costs, lifetime costs, and present values

Key Logic:

```
1 start_year = calculate_cod_year(delay_years, construction_years)
2 total_energy_compensated_mwh = compensation_percent *
  total_losses_mwh_per_year
3 energy_mix_by_year = calculate_energy_mix_by_year(
  energy_source_mix_details, project_lifetime)
4
5 for year, energy_mix in enumerate(energy_mix_by_year, 1):
6     emissions = calculate_emissions_by_year(
7         total_energy_compensated_mwh, energy_mix,
8         emission_intensities)
9     year_cost = 0.0
10    for pollutant, emissions_kg in emissions.items():
11        cost_per_kg = societal_costs.get(f"{pollutant}_cost_per_kg",
12            0.0)
13        pollutant_cost = emissions_kg * cost_per_kg
14        year_cost += pollutant_cost
15        total_costs_by_pollutant[pollutant] += pollutant_cost
16    year_discount_year = start_year + year - 1
17    year_pv = year_cost / ((1 + social_discount_rate) **
18        year_discount_year)
19    lifetime_cost_pv += year_pv
20    # total_costs_by_pollutant_pv updated per pollutant similarly
```

21.3.5 main()

Main execution function that orchestrates the emissions cost calculation and output.

21.4 Example

The following example demonstrates a typical usage of the emissions cost calculation:

```
1 # Example: 500 MW Overhead AC line
2 #
3 # Inputs (from YAML):
4 #   - Total energy losses: 10,000 MWh/year
5 #   - Compensation percentage: 100% (all losses compensated)
6 #   - Initial energy mix:
7 #     * Coal: 20%, rate: -2%/year
8 #     * Natural gas: 30%, rate: -1%/year
9 #     * Solar: 15%, rate: +5%/year
10 #     * Wind: 20%, rate: +3%/year
11 #     * Other: 15%, rate: 0%
12 #   - Emission intensities:
13 #     * Coal: CO2=1000 kg/MWh, SOx=5 kg/MWh, NOx=3 kg/MWh
14 #     * Natural gas: CO2=400 kg/MWh, SOx=0.1 kg/MWh, NOx=0.5 kg/MWh
15 #     * Solar/Wind: CO2=0, SOx=0, NOx=0
16 #   - Societal costs:
17 #     * CO2: $0.05/kg
18 #     * SOx: $10/kg
19 #     * NOx: $15/kg
20 #   - Project lifetime: 50 years
21 #   - Social discount rate: 3%
22 #
23 # Calculation (Year 1):
24 #   TEC = 1.0 * 10,000 = 10,000 MWh/year
25 #   CO2 = 10,000 * (0.20*1000 + 0.30*400 + 0.15*0 + 0.20*0) = 3.2M
26 #       kg
27 #   SOx = 10,000 * (0.20*5 + 0.30*0.1) = 10,300 kg
28 #   NOx = 10,000 * (0.20*3 + 0.30*0.5) = 7,500 kg
29 #   Cost = (3.2M * $0.05) + (10,300 * $10) + (7,500 * $15) = $250
30 #       ,000/year
31 #
32 #   (Costs decrease over time as grid decarbonizes)
33 #
34 # Output:
35 #   - Average annual CO2: 2.5M kg/year
36 #   - Average annual cost: $150,000/year
37 #   - Lifetime cost: $7.5M
38 #   - Present value: $4.2M
```

Integration Example:

```

1 # When called from ctcc.py (main() builds results from
  LifetimeEmissionsResults):
2 csv_manager = CTCCOutputManager()
3 results = {
4     "total_nominal": emissions_results.lifetime_cost,
5     "total_pv": emissions_results.lifetime_cost_pv,
6     "annual_cost": emissions_results.avg_annual_costs,
7     "co2_emissions_kg": emissions_results.total_emissions["co2"],
8     "co2_cost_nominal": emissions_results.total_costs_by_pollutant["
    co2"],
9     "co2_cost_pv": emissions_results.total_costs_by_pollutant_pv["
    co2"],
10    "sox_emissions_kg": ..., "sox_cost_nominal": ..., "sox_cost_pv":
    ...,
11    "nox_emissions_kg": ..., "nox_cost_nominal": ..., "nox_cost_pv":
    ...,
12 }
13 csv_manager.add_emissions_costs(results)
14 csv_manager.write_batch_summary()

```

21.5 Key Implementation Details

21.5.1 Compensation Percentage

The compensation percentage (α) represents the fraction of line losses that are compensated by generators. A value of 1.0 means all losses are compensated, while 0.0 means no compensation (no emissions cost).

21.5.2 Energy Mix Evolution

The script models how the energy mix evolves over the project lifetime using growth/decay rates. Each year, the mix is updated and normalized to sum to 100%. This captures grid decarbonization trends.

21.5.3 Emission Intensities

Emission intensities vary by energy source:

- **Fossil fuels** (coal, oil, natural gas): High CO₂, SO_x, NO_x
- **Renewables** (solar, wind, hydro): Zero emissions
- **Nuclear**: Zero CO₂, minimal SO_x/NO_x

21.5.4 Societal Costs

Societal costs represent the external costs of emissions (health impacts, climate change, etc.). These are typically higher than market prices and reflect the true societal impact.

21.5.5 Present Value Timing

Emissions costs start when the project becomes operational (after delay and construction periods). The present value calculation discounts annual costs back to the base year using the social discount rate.

21.5.6 Year-by-Year Calculation

The script calculates emissions and costs for each year individually, accounting for:

- Changing energy mix over time
- Time-varying discounting
- Pollutant-specific tracking

Chapter 22

Line Loss Costs/Benefits Calculation Documentation

22.1 Introduction

The `line_loss_costs.py` script calculates line loss costs for greenfield projects and line loss benefits for reconductoring projects. For greenfield projects, line losses represent a cost (energy that must be generated to compensate). For reconductoring projects, reduced losses compared to the old conductor represent a benefit.

22.2 Method Overview

22.2.1 Purpose

This script calculates line loss costs/benefits by:

1. Determining if project is greenfield or reconductoring
2. For greenfield: Calculating losses and converting to costs
3. For reconductoring: Comparing old vs. new conductor losses to calculate benefits
4. Computing present values using appropriate discount rates

22.2.2 Calculation Approach

Greenfield Projects:

$$\text{Annual Loss Cost} = \text{Losses (MWh/year)} \times \text{Electricity Price (\$/MWh)} \quad (22.1)$$

$$\text{Lifetime Nominal} = \text{Annual Cost} \times \text{Project Lifetime} \quad (22.2)$$

$$\text{Present Value} = \sum_{t=1}^N \frac{\text{Annual Cost}}{(1 + r)^{t+\text{start_year}-1}} \quad (22.3)$$

Reconductoring Projects:

$$\text{Loss Reduction} = \text{Old Losses} - \text{New Losses} \quad (22.4)$$

$$\text{Annual Benefit} = \text{Loss Reduction} \times \text{Electricity Price} \quad (22.5)$$

$$\text{Lifetime Benefit} = \text{Annual Benefit} \times \text{Project Lifetime} \quad (22.6)$$

$$\text{Present Value} = \sum_{t=1}^N \frac{\text{Annual Benefit}}{(1+r)^{t+\text{start_year}-1}} \quad (22.7)$$

The script uses a normalized comparison method that accounts for capacity differences between old and new configurations.

22.2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type, AC/DC, capacity, conductor type, reconductoring status)
- Old configuration details (for reconductoring: old capacity, old conductor type, old AC/DC)
- Physical details (line length)
- Circuit and resistance details
- Electricity price
- Social discount rate
- Project lifetime, delay years, construction years

Outputs:

- Annual cost/benefit
- Lifetime nominal cost/benefit
- Present value cost/benefit

22.2.4 Integration with Other Scripts

This script integrates with:

- `energy_losses.py` — `get_total_energy_losses` (primary config), `load_physical_details`, `load_circuit_and_resistance_details`, `calculate_phase_current`, `full_load_adjusted`, `calculate_line_losses`
- `smart_loaders.py` — `load_financing_social_discount_rate`, `load_project_technical_details`, `get_project_data_raw`

- `financial_utils.py` — `calculate_present_value`, `calculate_cod_year`
- `calculation_utils.py` — `to_percent`, `from_percent`, `build_category_string`, `calculate_converter_losses`
- `constants.py` — `HOURS_PER_YEAR`
- `smart_output.py` — `CTCCOutputManager` (`add_line_loss_costs`, `write_batch_summary`)

22.3 Key Functions

22.3.1 `load_project_details()`

Loads project technical details with `line_loss_costs` specific fields.

Returns:

- Multiple values including construction type, AC/DC, capacity, conductor type, utilization, reconductoring status, timeline parameters

22.3.2 `calculate_configuration_losses(...)`

Calculates line losses (conductor only) for a given configuration. Used for reconductoring baseline (old) and for greenfield comparison configurations. Optional `voltage_kv_override` allows using a different voltage (e.g., reconductoring: old tower voltage).

Parameters:

- `construction_type`, `ac_dc`, `capacity_mw`, `conductor_type`, `converter_type`, `line_utilization_percent`, `project_lifetime`
- `voltage_kv_override` (optional): If set, used instead of category lookup voltage

Returns:

- `losses_mwh_per_year`, `lifetime_losses_mwh` (conductor losses only; converter losses computed separately when needed)

Key Logic:

```

1 category = build_category_string(construction_type, ac_dc,
2     capacity_mw, conductor_type, converter_type)
3 line_length = load_physical_details()
4 circuit = load_circuit_and_resistance_details(category)
5 voltage_kv = voltage_kv_override if voltage_kv_override is not None
6     else circuit.voltage_kv
7 capacity_mw_str = f"{capacity_mw}MW"
8 phase_current = calculate_phase_current(capacity_mw_str, voltage_kv,
9     circuit.number_of_phases, circuit.number_of_circuits_poles,
10    ac_dc)

```

```

7 full_load_adj = full_load_adjusted(line_utilization_percent)
8 losses_mwh_per_year, lifetime_losses_mwh, ... =
    calculate_line_losses(phase_current, full_load_adj, circuit.
        AC_75_resistance, circuit.DC_20_resistance, ac_dc, ...)

```

22.3.3 main()

Main execution function that orchestrates the line loss cost/benefit calculation.

Workflow for Greenfield:

1. Gets losses via `get_total_energy_losses()` (line + converter)
2. Converts to costs (losses * `baseline_electricity_price`), `start_year` = `calculate_cod_year(delay_years, construction_years)`
3. Calculates present value; writes results (including optional line/converter breakdown) to CSV

Workflow for Reconductoring:

1. Baseline (old) losses: `calculate_configuration_losses(old_...)` + `calculate_converter_losses` for old config; new (primary) losses: `get_total_energy_losses()`
2. Counterfactual: old conductor at new capacity via `calculate_configuration_losses` with new capacity
3. Benefit = (baseline or counterfactual total losses - new total losses) * price; PV and CSV as benefits

22.4 Example

The following example demonstrates a typical usage:

```

1 # Example 1: Greenfield Project
2 # 500 MW Overhead AC line
3 #
4 # Inputs:
5 #   - Capacity: 500 MW
6 #   - Line utilization: 70%
7 #   - Line length: 100 miles
8 #   - Losses: 5,000 MWh/year
9 #   - Electricity price: $50/MWh
10 #   - Project lifetime: 50 years
11 #   - Start year: 3 (after 2-year construction)
12 #
13 # Calculation:

```

```

14 # Annual cost = 5,000 * $50 = $250,000/year
15 # Lifetime nominal = $250,000 * 50 = $12.5M
16 # PV = $250,000 * PV annuity (50 years, 3%, start year 3) = $8.2M
17 #
18 # Output:
19 #   - Annual cost: $250,000
20 #   - Lifetime nominal: $12.5M
21 #   - Present value: $8.2M
22
23 # Example 2: Reconductoring Project
24 # Upgrade 200 MW line to 350 MW
25 #
26 # Inputs:
27 #   - Old: 200 MW, Standard Aluminum, losses: 3,000 MWh/year
28 #   - New: 350 MW, Advanced Aluminum, losses: 4,000 MWh/year
29 #   - Counterfactual (old conductor @ 350 MW): 5,500 MWh/year
30 #   - Electricity price: $50/MWh
31 #
32 # Calculation (Normalized Method):
33 #   Loss reduction = 5,500 - 4,000 = 1,500 MWh/year
34 #   Annual benefit = 1,500 * $50 = $75,000/year
35 #   Lifetime benefit = $75,000 * 50 = $3.75M
36 #   PV = $75,000 * PV annuity (50 years, 3%) = $1.8M
37 #
38 # Output:
39 #   - Annual benefit: $75,000
40 #   - Lifetime benefit: $3.75M
41 #   - Present value: $1.8M

```

Integration Example:

```

1 # When called from ctcc.py (main() builds results):
2 csv_manager = CTCCOutputManager()
3 results = {
4     "annual_cost": total_annual_cost,
5     "total_nominal": total_lifetime_nominal,
6     "total_afudc": 0,
7     "total_pv": total_pv_loss_cost,
8     "line_loss_mwh_yr": losses_mwh_per_year,
9     "converter_loss_mwh_yr": total_converter_losses_mwh,
10    "total_loss_mwh_yr": total_losses_mwh_per_year,
11    "line_annual_cost": ..., "converter_annual_cost": ...,
12    "line_cost_pv": ..., "converter_cost_pv": ...,
13 }
14 csv_manager.add_line_loss_costs(results)
15 csv_manager.write_batch_summary()

```

22.5 Key Implementation Details

22.5.1 Greenfield vs. Reconductoring

The script handles two distinct cases:

- **Greenfield:** Line losses are a cost (energy that must be generated)
- **Reconductoring:** Reduced losses are a benefit (compared to old conductor)

22.5.2 Comparison Methods for Reconductoring

The script implements three comparison methods:

1. **Direct Comparison:** Old losses @ old capacity vs. new losses @ new capacity
2. **Counterfactual Comparison:** Old conductor @ new capacity vs. new conductor @ new capacity (used for output)
3. **Normalized Comparison:** Loss percentages weighted by delivered energy

The normalized method is used for the final output as it accounts for capacity differences.

22.5.3 Counterfactual Baseline

For reconductoring, the script calculates what losses would be if the old conductor were used at the new capacity. This provides a fair comparison by isolating the effect of the conductor upgrade.

22.5.4 Present Value Timing

Costs/benefits start when the project becomes operational (after delay and construction periods). The present value calculation discounts annual payments back to the base year.

22.5.5 AFUDC Treatment

Line loss costs/benefits are not AFUDC-eligible because they are operational costs/benefits, not construction costs.

Chapter 23

Operations and Maintenance Costs Documentation

23.1 Introduction

The `oandm.py` script calculates operations and maintenance (O&M) costs for a transmission line project. It computes annual and lifetime costs for conductors, converters, structures, and vegetation management, then calculates present values.

23.2 Method Overview

Canonical notation. The canonical variable names and equations for O&M are defined in `COST_METHODODOLOGY.md` under “Operational costs” / “O&M (2.a)”. Structure cost is structure-only and vegetation is a separate component: $C_{\text{structure,annual}}$ is piecewise (overhead vs. non-overhead), $C_{\text{veg,annual}} = \sum_{\text{terrain}} M_{\text{terrain}} \times c_{\text{veg}}(\text{terrain})$, and $C_{\text{O\&M,annual}} = C_{\text{conductor,annual}} + C_{\text{converter,annual}} + C_{\text{structure,annual}} + C_{\text{veg,annual}}$.

23.2.1 Purpose

This script calculates O&M costs by:

1. Loading O&M cost parameters by component (conductor, converter, structure, vegetation)
2. Calculating annual costs per component
3. Computing lifetime nominal costs
4. Calculating present values of annual O&M payments

23.2.2 Calculation Approach

The O&M cost calculation follows this structure. Structure Annual is structure-only (vegetation is a separate line):

$$\text{Conductor Annual} = \text{Cost/Mile/Year} \times \text{Total Miles} \quad (23.1)$$

$$\text{Converter Annual} = \text{Cost/Mile/Year} \times \text{Total Miles} \quad (\text{DC only}) \quad (23.2)$$

$$\text{Structure Annual (structure only)} = \begin{cases} \text{Cost/Structure/Year} \times \text{Total Structures} & (\text{Overhead}) \\ \text{Cost/Mile/Year} \times \text{Total Miles} & (\text{Other}) \end{cases} \quad (23.3)$$

$$\text{Vegetation Annual} = \sum_{\text{terrains}} \text{Cost/Mile/Year}_{\text{terrain}} \times \text{Miles}_{\text{terrain}} \quad (23.4)$$

$$\text{Total Annual} = \text{Conductor} + \text{Converter} + \text{Structure} + \text{Vegetation} \quad (23.5)$$

$$\text{Lifetime Nominal} = \text{Annual} \times \text{Project Lifetime} \quad (23.6)$$

$$\text{Present Value} = \sum_{t=1}^N \frac{\text{Annual}}{(1+r)^t} \quad (23.7)$$

23.2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type, AC/DC, capacity, conductor type, converter type)
- Physical details (total miles, terrain distribution)
- O&M cost parameters (conductor, converter, structure, vegetation management)
- Financing parameters (WACC for present value)
- Project lifetime

Outputs:

- Annual costs by component
- Lifetime nominal costs by component
- Present value costs by component
- Total annual, nominal, and PV costs

23.2.4 Integration with Other Scripts

This script integrates with:

- `yaml_loaders.py` — `load_financing_details`, `load_project_technical_details`, `load_physical_details_detailed`, `load_circuit_and_resistance_details`

- `financial_utils.py` — `calculate_present_value`, `calculate_cod_year`
- `calculation_utils.py` — `build_category_string`
- `path_config.py` — `YAMLS_DIR`
- `constants.py` — `CONSTRUCTION_TYPE_OVERHEAD`
- `smart_output.py` — `CTCCOutputManager` (`add_oandm_costs`, `write_batch_summary`)

23.3 Key Functions

23.3.1 `load_vegetation_management_om_costs(construction_type)`

Loads vegetation management O&M costs from YAML file.

Parameters:

- `construction_type` (str): Type of construction (Overhead/Subsea)

Returns:

- Dict mapping terrain key (e.g. `forested`, `farmland`) to cost per mile per year

23.3.2 `load_conductor_om_costs(construction_type, ac_dc, capacity_mw, conductor_type, converter_type)`

Loads conductor O&M costs from YAML file.

Parameters:

- `construction_type` (str): Type of construction
- `ac_dc` (str): AC or DC designation
- `capacity_mw` (float): Capacity in megawatts
- `conductor_type` (str): Type of conductor
- `converter_type` (str): Type of converter

Returns:

- `variable_conductor_cost_per_mile_year` (float): Variable conductor cost per mile per year

23.3.3 load_converter_om_costs(construction_type, ac_dc, capacity_mw, conductor_type, converter_type)

Loads converter O&M costs from YAML file. Returns 0 for AC projects.

Parameters:

- `construction_type` (str): Type of construction
- `ac_dc` (str): AC or DC designation
- `capacity_mw` (float): Capacity in megawatts
- `conductor_type` (str): Type of conductor
- `converter_type` (str): Type of converter

Returns:

- `variable_converter_cost_per_mile_year` (float): Variable converter cost per mile per year (0 for AC)

23.3.4 load_structure_om_costs(construction_type, forested_miles, ...)

Loads structure O&M from `14.category_om_structures.yaml`. For overhead: total structures by terrain (`structures_per_mile_forested`, etc.), cost per structure per year, then adds vegetation management (terrain-specific \$/mile/year from `load_vegetation_management_om_costs`). For non-overhead: returns `variable_cost_per_mile_year` only.

Parameters:

- `construction_type`; then `forested_miles`, `scrubbed_flat_miles`, `wetland_miles`, `farmland_miles`, `desert_barren_miles`, `urban_miles`, `rolling_hills_miles`, `mountain_miles`, `subsea_miles`

Returns:

- `variable_structure_cost_per_mile_year` (float), `variable_structure_cost_per_year` (float), `structure_dict` (counts by terrain), `total_vegetation_management_cost_per_year` (float)

Key Logic for Overhead:

```
1 category = construction_type # structure YAML keyed by construction
  type
2 # structures_per_mile_forested, structures_per_mile_farmland, ...
  from YAML
3 forested_structures = forested_miles * structures_per_mile_forested
4 # ... same for each terrain; subsea_structures = 0
5 total_structures = sum(terrain_structures)
```

```

6 variable_structure_cost_per_year = cost_per_structure_per_year *
  total_structures
7 vegetation_management_cost_per_mile_year =
  load_vegetation_management_om_costs(construction_type)
8 # per terrain: miles * vegetation_management_cost_per_mile_year [
  terrain]
9 total_vegetation_management_cost_per_year = sum(...)
10 variable_structure_cost_per_year +=
  total_vegetation_management_cost_per_year

```

23.3.5 main()

Main execution function that orchestrates the O&M cost calculation and output.

Workflow:

1. Loads project technical details and physical details
2. Loads O&M cost parameters for each component
3. Calculates annual costs per component
4. Calculates lifetime nominal costs
5. Calculates present values
6. Displays results and writes to CSV

23.4 Example

The following example demonstrates a typical usage of the O&M cost calculation:

```

1 # Example: 500 MW Overhead AC line
2 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
3 #
4 # Inputs (from YAML):
5 #   - Total miles: 100 miles
6 #   - Terrain: 60% farmland, 40% forested
7 #   - Conductor O&M: $5,000/mile/year
8 #   - Converter O&M: $0 (AC project)
9 #   - Structure O&M: $500/structure/year
10 #   - Structures: 2.5/mile (farmland), 3.0/mile (forested)
11 #   - Vegetation: $2,000/mile/year (farmland), $3,000/mile/year (
    forested)
12 #   - Project lifetime: 50 years
13 #   - Real WACC: 3%
14 #
15 # Calculation:
16 #   Conductor annual = $5,000 * 100 = $500,000/year

```

```

17 # Converter annual = $0
18 # Total structures = (60 * 2.5) + (40 * 3.0) = 270 structures
19 # Structure annual = $500 * 270 = $135,000/year
20 # Vegetation annual = (60 * $2,000) + (40 * $3,000) = $240,000/
    year
21 # Total annual = $875,000/year
22 # Lifetime nominal = $875,000 * 50 = $43.75M
23 # PV = $875,000 * PV annuity (50 years, 3%) = $20.1M
24 #
25 # Output:
26 #   - Total annual: $875,000
27 #   - Lifetime nominal: $43.75M
28 #   - Present value: $20.1M

```

Integration Example:

```

1 # When called from ctcc.py:
2 csv_manager = CTCCOutputManager()
3 results = {
4     "total_annual": total_annual,
5     "total_nominal": total_nominal,
6     "total_pv": pv_total,
7     "conductor_annual": total_conductor_cost_per_year,
8     # ... component-level costs
9 }
10 csv_manager.add_oandm_costs(results)
11 csv_manager.write_batch_summary()

```

23.5 Key Implementation Details

23.5.1 Structure O&M for Overhead Projects

For overhead projects, structure O&M is calculated based on the number of structures, which varies by terrain type. The script:

1. Calculates structures per terrain based on structures per mile
2. Sums to get total structures
3. Multiplies by cost per structure per year

23.5.2 Vegetation Management

Vegetation management costs are terrain-specific and only apply to overhead projects. The script multiplies terrain-specific costs per mile by miles in each terrain type.

23.5.3 Converter O&M

Converter O&M costs only apply to DC projects. For AC projects, the function returns 0.

23.5.4 Present Value Timing

O&M costs start when the project becomes operational (after delay and construction). The script uses `calculate_cod_year(delay_years, construction_years)` as `start_year` in `calculate_present_value` to discount annual O&M payments back to the base year.

23.5.5 Component-Level Tracking

The script tracks costs at the component level (conductor, converter, structure, vegetation) for detailed analysis and reporting.

Chapter 24

Revenue Documentation

24.1 Introduction

The `revenue.py` script calculates the rate-based revenue requirement from a utility perspective using **Option A: real stream and real WACC**. The nominal rate base (AFUDC capital at COD) is deflated to base year; annual revenue is a constant real (base-year) \$/year; present value uses real WACC. **Rate base (nominal at COD)** = `build_cost_afudc` + `row_cost_afudc` + `env_mitigation_afudc`. The result is written to the batch summary (CSV or JSON) and used in BCR and reporting. Revenue calculation can be disabled via `financial.revenue.rate_based.enabled` in the financing config.

24.2 Method Overview

24.2.1 Purpose

1. Load rate-based revenue parameters from financing data: `enabled` and `allowed_return_rate` (from `financial.revenue.rate_based`); `allowed_return_rate` is interpreted as the *real* return.
2. If not enabled: write zero revenue to the output manager and return.
3. Load project technical details (`delay_years`, `construction_years`, `project_lifetime`) and financing details (WACC real, `inflation_rate`, base year).
4. Get rate base *nominal at COD* (AFUDC capital) from `batch_summary.csv` (CSV: `build_cost_afudc`, `row_cost_afudc`, `env_mitigation_afudc`) or from per-module JSON `total_afudc` (JSON mode).
5. Option A: Deflate rate base to base year: $\text{rate_base_real} = \text{rate_base} / (1 + \text{inflation_rate})^{t_{\text{COD}}}$; compute annual revenue (real) = `rate_base_real` × `allowed_return_rate`; undiscounted total (real) = `annual_revenue_real` × `project_lifetime`.

6. Compute present value of the *real* annual revenue stream using `calculate_present_value(annual_revenue_real, real WACC, project_lifetime, start_year = COD)`.
7. Write `revenue_nominal` (undiscounted total in real terms), `revenue_pv`, `annual_revenue (real)`, `rate_base`, `rate_base_real`, `rate_base_pv`, `annual_revenue_real`, `allowed_return_rate` to the output manager and call `write_batch_summary()`.

24.2.2 Calculation Approach (Option A: Real Stream, Real WACC)

Rate base *nominal at COD* is **AFUDC capital at COD** (build + ROW capital + environmental mitigation). For the utility benefit, CTCC uses Option A: the revenue stream is defined in **real** terms (constant base-year \$/year) and discounted at **real WACC**, consistent with real discounting elsewhere in CTCC.

1. **COD** (years from base): $t_{\text{COD}} = \text{delay_years} + \text{construction_years} + 1$.
2. **Real rate base** (deflate nominal at COD to base year): $\text{RateBase}_{\text{real}} = \text{RateBase}_{\text{nom}} / (1 + \pi)^{t_{\text{COD}}}$, where π is `inflation_rate`.
3. **Real annual revenue** (constant each year in base-year \$): $\text{AnnualRevenue}_{\text{real}} = \text{RateBase}_{\text{real}} \times \text{allowed_return_rate}$ (allowed return is the real return).
4. **Undiscounted total** (real): $\text{TotalRevenue}_{\text{real}} = \text{AnnualRevenue}_{\text{real}} \times \text{project_lifetime}$.
5. **Present value** of revenue (benefit to utility): PV of the constant real stream from COD over project lifetime, discounted at real WACC.

$$\text{Rate Base (nominal at COD)} = \text{build_cost_afudc} + \text{row_cost_afudc} + \text{env_mitigation_afudc} \quad (24.1)$$

$$t_{\text{COD}} = \text{delay_years} + \text{construction_years} + 1 \quad (24.2)$$

$$\text{RateBase}_{\text{real}} = \frac{\text{RateBase}}{(1 + \pi)^{t_{\text{COD}}}} \quad (24.3)$$

$$\text{AnnualRevenue}_{\text{real}} = \text{RateBase}_{\text{real}} \times \text{allowed_return_rate} \quad (24.4)$$

$$\text{TotalRevenue}_{\text{real}} = \text{AnnualRevenue}_{\text{real}} \times \text{project_lifetime} \quad (24.5)$$

$$\text{Revenue (PV)} = \sum_{t=t_{\text{COD}}}^{t_{\text{COD}} + \text{project_lifetime} - 1} \frac{\text{AnnualRevenue}_{\text{real}}}{(1 + r)^t} \quad (24.6)$$

where π is `inflation_rate` and r is the real WACC.

24.2.3 Inputs and Outputs

Inputs:

- Financing data (YAML or JSON): `financial.revenue.rate_based.enabled`, `financial.revenue.rate_based.allowed_return_rate`; `financial.wacc_nominal`, `inflation_rate`, `base_year` for PV.
- Project technical details: `delay_years`, `construction_years`, `project_lifetime`.
- Rate base (AFUDC capital): from `batch_summary.csv` columns `build_cost_afudc`, `row_cost_afudc`, `env_mitigation_afudc` (CSV mode) or from per-module JSON `total_afudc` (JSON mode); requires `CTCC_SCENARIO_ID` in JSON mode.

Outputs:

- `revenue_nominal` (undiscounted total in real terms), `revenue_pv`, `annual_revenue` (real), `rate_base`, `rate_base_real`, `rate_base_pv`, `annual_revenue_real`, `allowed_return_rate` passed to `CTCCOutputManager.add_revenue(results)`; then `write_batch_summary()`. Used by BCR calculator (benefits side) and in final JSON/CSV summary.

24.2.4 Integration with Other Scripts

- `ctcc.py` — Runs `revenue.py` after `weighted_miles`, `build_costs`, `row_costs`, `environmental_mitigation` so capital costs are available.
- `smart_loaders.py` — `load_financing_details`, `get_financing_data_raw`, `load_project_technical_details` (centralized).
- `financial_utils.py` — `calculate_present_value`, `calculate_cod_year`.
- `smart_output.py` — `CTCCOutputManager` for `add_revenue` and `write_batch_summary`.
- `path_config.py` — `OUTPUTS_DIR` for `batch_summary.csv` and JSON output paths.
- `bcr_calculator.py` — Reads revenue (e.g. `revenue_pv`, `revenue_nominal`) from batch summary or aggregated JSON for BCR benefits.

24.3 Key Functions

24.3.1 `load_rate_based_revenue_parameters()`

Loads `enabled` and `allowed_return_rate` from `get_financing_data_raw()["financial"]["revenue"]["rate_based"]`.

Defaults: `enabled=False`, `allowed_return_rate=0.10`.

Returns: tuple: (`enabled: bool`, `allowed_return_rate: float`).

24.3.2 load_project_technical_details()

Uses centralized `load_project_technical_details` and returns (`delay_years`, `construction_years`, `project_lifetime`) from the project technical details dataclass.

24.3.3 get_rate_base()

Returns rate base (AFUDC capital at COD) = `build_cost_afudc` + `row_cost_afudc` + `env_mitigation_afudc`. In JSON mode (when `CTCC_OUTPUT_MODE=json` and `CTCC_SCENARIO_ID` is set), reads `total_afudc` from `json_output_<scenario_id>_build_costs.json`, `row_costs.json`, `environmental_mitigation.json` in `OUTPUTS_DIR`. In CSV mode, reads from `batch_summary.csv` (row matching `scenario_id`, or last row if `scenario_id` not set). Returns 0 if files missing or scenario not found.

Key logic (Option A: real stream, real WACC):

```
1 rate_base = get_rate_base() # nominal at COD
2 cod_year = calculate_cod_year(delay_years, construction_years)
3 rate_base_real = rate_base / (1 + financing.inflation_rate) **
   cod_year
4 annual_revenue_real = rate_base_real * allowed_return_rate
5 revenue_undiscounted_real = annual_revenue_real * project_lifetime
6 revenue_pv = calculate_present_value(
7     annual_revenue_real,
8     financing.wacc_real,
9     project_lifetime,
10    start_year=cod_year,
11 )
```

24.3.4 main()

Checks `load_rate_based_revenue_parameters()`; if not enabled, writes zeros (including `rate_base_real`, `annual_revenue_real`) via `add_revenue` and `write_batch_summary` and returns. Otherwise loads details, gets `get_rate_base()` (nominal at COD), computes real rate base, real annual revenue, undiscounted real total, and PV at real WACC (Option A), then adds results and writes batch summary.

24.4 Example

With `financial.revenue.rate_based.enabled = true`, `allowed_return_rate = 0.10` (real), `inflation_rate = 0.03`, rate base nominal at COD = \$500M, `project_lifetime = 40`, $t_{\text{COD}} = 5$:

- Rate base (real) = $500 / (1.03)^5 \approx \$430.5\text{M}$ (base-year \$).
- Annual revenue (real) = $430.5 \times 0.10 \approx \43.05M per year (constant real \$/year).
- Undiscounted total (real) = $43.05 \times 40 \approx \1722M .

- Revenue PV = present value of \$43.05M per year (real) for 40 years starting at year 5, discounted at real WACC.

Standalone run (after capital scripts have run):

```
1 cd scripts
2 python revenue.py
```

Chapter 25

Benefit-Cost Ratio Calculator Documentation

25.1 Introduction

The `bcr_calculator.py` script calculates Benefit-Cost Ratios (BCR) and Net Benefits for transmission projects. It aggregates benefits and costs from all CTCC modules and computes multiple BCR metrics to evaluate project economics from different perspectives.

25.2 Method Overview

25.2.1 Purpose

This script calculates BCR metrics by:

1. Loading scenario data from `batch_summary.csv` (robust lookup: exact, partial, prefix, last-row fallback)
2. Aggregating benefits (congestion, curtailment; line losses are always costs, never benefits)
3. Aggregating costs by category (capital, operational, energy/emissions, risk, delay) using `safe_get_numeric` and `prefer_csv_subtotal`
4. Applying optional `BCRConfig` exclusions (`no_emissions`, `no_linelosses`, `capital_only`, `no_wildfire`, etc.) when loading from YAML
5. Calculating multiple BCR metrics (system, capital, excluding risk, excluding emissions)
6. Computing net benefits; displaying formatted summary and updating CSV

25.2.2 Calculation Approach

The BCR calculation follows this structure:

$$\text{Total Benefits (Conservative)} = \text{Congestion (Haircut)} + \text{Curtailment (Haircut)} \quad (25.1)$$

$$\text{Total Costs} = \text{Capital} + \text{Operational} + \text{Energy/Emissions} + \text{Risk} + \text{Delay} \quad (25.2)$$

$$\text{BCR}_{\text{system}} = \frac{\text{Conservative Benefits}}{\text{Total Costs}} \quad (25.3)$$

$$\text{BCR}_{\text{capital}} = \frac{\text{Conservative Benefits}}{\text{Capital Costs}} \quad (25.4)$$

$$\text{BCR}_{\text{excluding_risk}} = \frac{\text{Conservative Benefits}}{\text{Total Costs} - \text{Risk Costs}} \quad (25.5)$$

$$\text{BCR}_{\text{excluding_emissions}} = \frac{\text{Conservative Benefits}}{\text{Total Costs} - \text{Emissions Costs}} \quad (25.6)$$

$$\text{Net Benefit} = \text{Conservative Benefits} - \text{Total Costs} \quad (25.7)$$

25.2.3 Inputs and Outputs

Inputs:

- Scenario ID (to load from `batch_summary.csv`)
- All cost and benefit metrics from CTCC modules

Outputs:

- BCR metrics (system, capital, excluding risk, excluding emissions) - all use conservative benefits
- Net benefits (total and excluding risk)
- Benefit and cost breakdowns
- Formatted summary display

25.2.4 Integration with Other Scripts

This script integrates with:

- `path_config.py` — `OUTPUTS_DIR`, `YAMLS_DIR` (BCR config YAML optional)
- `batch_summary.csv` — Reads scenario row by `scenario_id`; writes BCR columns back
- All CTCC cost/benefit scripts — Aggregates their outputs from the batch summary row

25.3 Key Functions

25.3.1 `load_scenario_data(scenario_id, output_dir)`

Loads scenario data from `batch_summary.csv`. Uses robust lookup: (1) exact match on `scenario_id`, (2) partial match (substring), (3) prefix match (e.g. `sample_0_...`), (4) fallback to last row (for batch sensitivity runs). Converts numeric cells via `convert_row_to_numeric`.

Parameters:

- `scenario_id` (str): Scenario identifier
- `output_dir` (str): Directory containing `batch_summary.csv` (default `OUTPUTS_DIR`)

Returns:

- Dictionary with all data for the scenario, or `None` if file missing/empty

25.3.2 `calculate_benefits(data)`

Calculates total benefits from scenario data.

Parameters:

- `data` (dict): Dictionary with scenario data

Returns:

- Dictionary with benefit breakdown and totals (PV, nominal, haircut)

Key Logic:

```
1 congestion_benefit_pv = safe_get_numeric(data, "  
    congestion_benefit_pv")  
2 curtailment_benefit_pv = safe_get_numeric(data, "  
    curtailment_benefit_pv")  
3 # Line losses are always costs, never benefits (line_loss_benefit_pv  
    = 0)  
4 calculated_total_benefits = congestion_benefit_pv +  
    curtailment_benefit_pv  
5 total_benefits_pv = prefer_csv_subtotal(data, "total_benefits_pv",  
    calculated_total_benefits)  
6 congestion_benefit_haircut = safe_get_numeric(data, "  
    congestion_benefit_haircut_pv")  
7 curtailment_benefit_haircut = safe_get_numeric(data, "  
    curtailment_benefit_haircut_pv")  
8 total_benefits_haircut_pv = prefer_csv_subtotal(data, "  
    total_benefits_haircut_pv",  
9    congestion_benefit_haircut + curtailment_benefit_haircut)
```

25.3.3 calculate_costs(data)

Calculates total costs from scenario data, organized by category.

Parameters:

- **data** (dict): Dictionary with scenario data

Returns:

- Dictionary with cost breakdown by category and totals (PV and nominal)

Cost Categories:

- **Capital:** Build, ROW, environmental mitigation
- **Operational:** O&M, insurance, line losses (greenfield), emissions
- **Risk:** Wildfire, outage
- **Delay:** Construction delay, congestion/curtailment delay, residual congestion

Key Logic:

```
1 # Capital costs
2 capital_costs_pv = build_cost_pv + row_cost_pv + env_mitigation_pv
3
4 # Operational costs (line losses only if positive - greenfield)
5 line_loss_cost_pv = max(0, line_loss_pv)
6 operational_costs_pv = oandm_pv + insurance_pv + line_loss_cost_pv +
   emissions_pv
7
8 # Risk costs
9 risk_costs_pv = wildfire_pv + outage_pv
10
11 # Delay costs
12 delay_costs_pv = (delay_cost_pv + congestion_delay_pv
13                  + curtailment_delay_pv + residual_congestion_pv)
14
15 # Totals
16 total_costs_pv = (capital_costs_pv + operational_costs_pv
17                  + risk_costs_pv + delay_costs_pv)
```

25.3.4 calculate_bcr_metrics(benefits, costs)

Calculates benefit-cost ratios and net benefits.

Parameters:

- **benefits** (dict): Dictionary with benefit breakdown
- **costs** (dict): Dictionary with cost breakdown

Returns:

- Dictionary with BCR metrics and net benefits

BCR Metrics:

- **BCR_system:** Conservative benefits / Total costs (primary metric, uses saturation-adjusted benefits)
- **BCR_capital:** Conservative benefits / Capital costs (investor perspective, uses saturation-adjusted benefits)
- **BCR_excluding_risk:** Conservative benefits / (Total costs - Risk costs) (core economics, uses saturation-adjusted benefits)
- **BCR_excluding_emissions:** Conservative benefits / (Total costs - Emissions costs) (uses saturation-adjusted benefits)

Key Logic:

```
1 # Calculate costs excluding risk
2 total_costs_excluding_risk_pv = total_costs_pv - risk_costs_pv
3
4 # Use conservative (haircut) benefits for all BCR calculations
5 total_benefits_pv = total_benefits_haircut_pv
6
7 # Calculate BCRs (with division by zero protection)
8 bcr_system = total_benefits_pv / total_costs_pv if total_costs_pv >
   0 else 0
9 bcr_capital = total_benefits_pv / capital_costs_pv if
   capital_costs_pv > 0 else 0
10 bcr_excluding_risk = (total_benefits_pv /
   total_costs_excluding_risk_pv
11                       if total_costs_excluding_risk_pv > 0 else 0)
12 bcr_excluding_emissions = (total_benefits_pv /
   total_costs_excluding_emissions_pv
13                           if total_costs_excluding_emissions_pv > 0
   else 0)
14
15 # Net benefits (using conservative benefits)
16 net_benefit_pv = total_benefits_pv - total_costs_pv
17 net_benefit_excluding_risk_pv = total_benefits_pv -
   total_costs_excluding_risk_pv
```

25.3.5 calculate_and_display_bcr(scenario_id, output_dir)

Main function to calculate and display BCR analysis.

Parameters:

- `scenario_id` (str): Unique identifier for the scenario
- `output_dir` (str): Directory containing `batch_summary.csv`

Returns:

- Dictionary with all BCR results, or None if calculation fails

25.3.6 `print_bcr_summary(benefits, costs, bcr_metrics, data)`

Prints formatted BCR summary to terminal.

25.4 Example

The following example demonstrates a typical usage:

```

1 # Example: 500 MW Overhead AC line
2 # Scenario ID: "20250101_120000"
3 #
4 # Inputs (from batch_summary.csv):
5 #   - Congestion benefit PV: $500M
6 #   - Curtailment benefit PV: $200M
7 #   - Line loss cost PV: $10M (greenfield - cost)
8 #   - Build cost PV: $100M
9 #   - ROW cost PV: $20M
10 #   - Environmental PV: $15M
11 #   - O&M PV: $50M
12 #   - Insurance PV: $5M
13 #   - Emissions PV: $8M
14 #   - Wildfire PV: $1B
15 #   - Outage PV: $500M
16 #   - Delay costs PV: $2M
17 #   - Residual congestion PV: $50M
18 #
19 # Calculation:
20 #   Total Benefits = $500M + $200M + $0 = $700M
21 #   Capital Costs = $100M + $20M + $15M = $135M
22 #   Operational Costs = $50M + $5M + $10M + $8M = $73M
23 #   Risk Costs = $1B + $500M = $1.5B
24 #   Delay Costs = $2M + $50M = $52M
25 #   Total Costs = $135M + $73M + $1.5B + $52M = $1.76B
26 #
27 #   BCR_system = $700M / $1.76B = 0.40
28 #   BCR_capital = $700M / $135M = 5.19
29 #   BCR_excluding_risk = $700M / ($1.76B - $1.5B) = 2.69
30 #   Net Benefit = $700M - $1.76B = -$1.06B
31 #
32 # Output:

```

```

33 # - BCR_system: 0.40
34 # - BCR_capital: 5.19
35 # - BCR_excluding_risk: 2.69
36 # - Net benefit: -$1.06B

```

Integration Example:

```

1 # When called from ctcc.py:
2 from scripts.bcr_calculator import calculate_and_display_bcr
3
4 # After all other scripts complete
5 scenario_id = os.environ.get('CTCC_SCENARIO_ID')
6 results = calculate_and_display_bcr(scenario_id)
7
8 # Update batch_summary.csv with BCR metrics
9 csv_manager = CTCCOutputManager()
10 csv_manager.add_bcr_metrics(results)
11 csv_manager.write_batch_summary()

```

25.5 Key Implementation Details

25.5.1 Line Loss Treatment

The script handles line losses differently for greenfield vs. reconductoring:

- **Greenfield:** Line losses are positive (costs) - counted in operational costs
- **Reconductoring:** Line losses are negative (benefits) - counted in benefits

25.5.2 Multiple BCR Perspectives

The script calculates multiple BCR metrics to provide different perspectives. All BCR metrics use conservative (saturation-adjusted) benefits to account for diminishing marginal returns:

- **System BCR:** Overall project economics including all costs (uses conservative benefits)
- **Capital BCR:** Benefits relative to capital investment (investor view, uses conservative benefits)
- **Excluding Risk BCR:** Core economics without wildfire/outage risk costs (uses conservative benefits)
- **Excluding Emissions BCR:** Economics without emissions costs (uses conservative benefits)

25.5.3 Net Benefits

The script calculates net benefits in addition to BCRs:

- **Net Benefit:** Total benefits - Total costs
- **Net Benefit Excluding Risk:** Total benefits - (Total costs - Risk costs)

25.5.4 Division by Zero Protection

All BCR calculations include checks to prevent division by zero, returning 0 if the denominator is zero or negative.

25.5.5 Data Loading

The script loads data from `batch_summary.csv` using the scenario ID, automatically converting numeric strings to floats for calculations.

Chapter 26

BCR (Benefit-Cost Ratio) Cheat Sheet

Quick reference guide for all BCR perspectives calculated by CTCC.

Important Note on Revenue

Revenue (rate-based revenue requirement) is a **transfer** from ratepayers to the utility/TSP, not a net social benefit. System/societal BCRs exclude revenue from total benefits. Revenue appears only in stakeholder BCRs where it's a benefit to utilities and a cost to ratepayers.

Cost categories

Capital = build + ROW capital (acquisition + holding) + environmental mitigation.
Operational = O&M + insurance + ROW rent. **Energy/Emissions** = line losses + emissions + residual exceedance. Residual exceedance is a system/societal cost only. Rate base = AFUDC capital (build + row capital + env) at COD. Revenue is computed in real terms (Option A: real stream, real WACC).

26.1 BCR Metrics (order matches Benefit-Cost Ratio Metrics tables)

Order below matches the per-scenario and all-scenarios BCR tables in `results_tables.tex` (updated by `papers/update_results_tables.py`).

26.1.1 bcr_system

Equation: Total Benefits (haircut) / Total Costs

Perspective: Full societal/system-wide analysis

Includes:

- **Benefits:** Congestion reduction + Curtailment reduction (with conservative haircuts; excludes revenue transfer)

- **Costs:** Capital + Operational (O&M + insurance only) + Energy/Emissions (line losses + emissions + residual exceedance) + Risk + Delay (everything)

Use Case: Complete societal cost-benefit analysis including all externalities and risks.

26.1.2 `bcr_utility`

Equation: Revenue / (Capital Costs + Delay Costs + Operational Costs). Benefits: Revenue only. Costs: Capital + Delay + Operational (O&M + insurance only). Excludes: Line losses, Emissions, Residual exceedance, Risk.

Key Insight: Utilities don't receive congestion/curtailment benefits directly; they only get rate base recovery.

26.1.3 `bcr_ratepayer`

Equation: (Congestion Benefits + Curtailment Benefits) / (Revenue + Line Loss Costs). Benefits: Congestion + Curtailment. Costs: Revenue + Line losses. Excludes: Capital/O&M as separate line items (recovered via Revenue).

Key Insight: Ratepayers receive congestion/curtailment relief but pay revenue/rate base and line losses through rates.

26.1.4 `bcr_capital`

Equation: Total Benefits (haircut) / Capital Costs. Costs: Build + ROW + environmental only.

26.1.5 `bcr_capital_and_delay`

Equation: Total Benefits (haircut) / (Capital Costs + Delay Costs). Costs: Capital + Delay (construction delay + congestion delay + curtailment delay).

26.1.6 `bcr_primary`

Equation: Primary Benefits / Primary Costs (customizable via flags). Benefits: Congestion (optional) + Curtailment (optional). Costs: Capital + Delay (always) + O&M, Insurance, Wildfire, Outage, Line Losses, Emissions (optional). If no flags are set, equals `bcr_system`.

26.1.7 `bcr_excluding_wildfire_risk_and_outage_risk`

Equation: Total Benefits (haircut) / (Total Costs - Wildfire Risk - Outage Risk)

Perspective: Societal perspective excluding both wildfire and outage risk costs.

Includes: Benefits: Congestion + Curtailment (excludes revenue). Costs: Capital + Operational + Energy/Emissions + Delay (excludes wildfire + wildfire liability + outage).

Use Case: Analysis when both wildfire and outage risk costs may be insured, managed separately, or contextually inappropriate.

26.1.8 `bcr_excluding_emissions`

Equation: Total Benefits (haircut) / (Total Costs - Emissions Costs). Costs: Capital + Operational + Risk + Delay + Line Losses (excludes emissions only).

26.1.9 `bcr_excluding_linelosses`

Equation: Total Benefits (haircut) / (Total Costs - Line Loss Costs). Costs: Capital + Operational + Risk + Delay + Emissions (excludes line losses only).

26.1.10 `bcr_excluding_emissions_and_linelosses`

Equation: Total Benefits (haircut) / (Total Costs - Emissions Costs - Line Loss Costs). Costs: Capital + Operational + Risk + Delay (excludes emissions + line losses).

26.1.11 `bcr_excluding_emissions_and_wildfire_risk_and_outage_risk`

Equation: Total Benefits / (Total Costs - Emissions - Wildfire Risk - Outage Risk). Costs: Capital + Operational + Delay + Line Losses.

26.1.12 `bcr_excluding_linelosses_and_wildfire_risk_and_outage_risk`

Equation: Total Benefits / (Total Costs - Line Losses - Wildfire Risk - Outage Risk). Costs: Capital + Operational + Delay + Emissions.

26.1.13 `bcr_excluding_emissions_and_linelosses_and_wildfire_risk_and_outage_risk`

Equation: Total Benefits / (Total Costs - Emissions - Line Losses - Wildfire Risk - Outage Risk). Costs: Capital + Operational + Delay only.

26.1.14 `bcr_excluding_wildfire_risk`

Equation: Total Benefits (haircut) / (Total Costs - Wildfire Risk Costs). Costs: Capital + Operational + Energy/Emissions + Delay + Outage Risk (excludes wildfire + wildfire liability).

26.1.15 `bcr_excluding_emissions_and_wildfire_risk`

Equation: Total Benefits / (Total Costs - Emissions - Wildfire Risk). Costs: Capital + Operational + Delay + Line Losses + Outage Risk.

26.1.16 `bcr_excluding_linelosses_and_wildfire_risk`

Equation: Total Benefits / (Total Costs - Line Loss Costs - Wildfire Risk Costs). Costs: Capital + Operational + Delay + Emissions + Outage Risk.

26.1.17 `bcr_excluding_emissions_and_linelosses_and_wildfire_risk`

Equation: Total Benefits / (Total Costs - Emissions - Line Losses - Wildfire Risk). Costs: Capital + Operational + Delay + Outage Risk only.

26.1.18 `bcr_excluding_outage_risk`

Equation: Total Benefits (haircut) / (Total Costs - Outage Risk Costs). Costs: Capital + Operational + Energy/Emissions + Delay + Wildfire Risk (excludes outage).

26.1.19 `bcr_excluding_emissions_and_outage_risk`

Equation: Total Benefits / (Total Costs - Emissions - Outage Risk). Costs: Capital + Operational + Delay + Line Losses + Wildfire Risk.

26.1.20 `bcr_excluding_linelosses_and_outage_risk`

Equation: Total Benefits / (Total Costs - Line Loss Costs - Outage Risk Costs). Costs: Capital + Operational + Delay + Emissions + Wildfire Risk.

26.1.21 `bcr_excluding_emissions_and_linelosses_and_outage_risk`

Equation: Total Benefits / (Total Costs - Emissions - Line Losses - Outage Risk). Costs: Capital + Operational + Delay + Wildfire Risk only.

26.2 Net Benefits

All BCRs have corresponding net benefit calculations (Present Value unless noted).

26.2.1 System/Societal Net Benefits

- `net_benefit_pv` = Total Benefits (haircut) - Total Costs
- `net_benefit_nominal` = Total Benefits (nominal) - Total Costs (nominal), undiscounted
- `net_benefit_primary_pv` = Primary Benefits - Primary Costs (customizable)
- `net_benefit_excluding_wildfire_risk_and_outage_risk_pv` = Total Benefits - (Total Costs - Wildfire Risk - Outage Risk)
- `net_benefit_excluding_wildfire_risk_pv`, `net_benefit_excluding_outage_risk_pv`
- `net_benefit_excluding_emissions_and_linelosses_pv`
- `net_benefit_excluding_emissions_and_linelosses_and_wildfire_risk_and_outage_risk_pv`
- `net_benefit_capital_only_pv`, `net_benefit_capital_and_delay_pv`

26.2.2 Stakeholder Net Benefits

- $\text{net_benefit_utility_pv} = \text{Revenue} - (\text{Capital} + \text{Delay} + \text{Operational Costs})$
- $\text{net_benefit_ratepayer_pv} = (\text{Congestion} + \text{Curtailment Benefits}) - (\text{Revenue} + \text{Line Loss Costs})$

26.2.3 Partial Exclusion Net Benefits (16 Combinations)

Systematic exploration of all combinations excluding emissions, line losses, wildfire risk, and outage risk. All use societal perspective (benefits: congestion + curtailment; excludes revenue transfer). Positive = benefits exceed costs; Negative = costs exceed benefits.

WF	Out	Em	LL	Net Benefit Variable	Equation
N	N	N	N	<code>net.benefit.pv</code>	Benefits - Total Costs
Y	N	N	N	<code>...excl. wildfire_risk.pv</code>	Benefits - (Costs - WF)
N	Y	N	N	<code>...excl. outage_risk.pv</code>	Benefits - (Costs - Outage)
Y	Y	N	N	<code>...excl. wildfire_and_outage.pv</code>	Benefits - (Costs - WF - Out)
N	N	Y	N	<code>...excl. emissions.pv</code>	Benefits - (Costs - Em); keeps LL
Y	N	Y	N	<code>...excl. emissions_and_wildfire.pv</code>	Benefits - (Costs - Em - WF)
N	Y	Y	N	<code>...excl. emissions_and_outage.pv</code>	Benefits - (Costs - Em - Out)
Y	Y	Y	N	<code>...excl. em_wf_outage.pv</code>	Benefits - (Costs - Em - WF - Out)
N	N	N	Y	<code>...excl. linelosses.pv</code>	Benefits - (Costs - LL); keeps Em
Y	N	N	Y	<code>...excl. LL_wildfire.pv</code>	Benefits - (Costs - LL - WF)
N	Y	N	Y	<code>...excl. LL_outage.pv</code>	Benefits - (Costs - LL - Out)
Y	Y	N	Y	<code>...excl. LL_wf_outage.pv</code>	Benefits - (Costs - LL - WF - Out)
N	N	Y	Y	<code>...excl. emissions_and_linelosses.pv</code>	Benefits - (Costs - Em - LL)
Y	N	Y	Y	<code>...excl. em_LL_wildfire.pv</code>	Benefits - (Costs - Em - LL - WF)
N	Y	Y	Y	<code>...excl. em_LL_outage.pv</code>	Benefits - (Costs - Em - LL - Out)
Y	Y	Y	Y	<code>...excl. em_LL_wf_outage.pv</code>	Benefits - (Costs - Em - LL - WF - Out)

Column key: WF = wildfire risk, Out = outage risk, Em = emissions, LL = line losses. “...” = `net_benefit_excluding_..`

26.3 Quick Decision Guide

Question	Use This BCR
Should a utility/TSP build this?	<code>bcr_utility</code>
Do ratepayers benefit?	<code>bcr_ratepayer</code>
Is it good for society overall?	<code>bcr_system</code> or <code>bcr_excluding_wildfire_risk_and_outage_risk</code>
Is the capital investment attractive?	<code>bcr_capital</code> or <code>bcr_capital_and_delay</code>
Ignore wildfire risk (keep outage)?	<code>bcr_excluding_wildfire_risk</code>
Ignore outage risk (keep wildfire)?	<code>bcr_excluding_outage_risk</code>
Ignore both wildfire and outage risk?	<code>bcr_excluding_wildfire_risk_and_outage_risk</code>
Ignore emissions only?	<code>bcr_excluding_emissions</code>
Ignore line losses only?	<code>bcr_excluding_line losses</code>
Ignore both emissions and line losses?	<code>bcr_excluding_emissions_and_line losses</code>
Ignore emissions and both risks?	<code>bcr_excluding_emissions_and_wildfire_risk_and_outage_risk</code>
Ignore line losses and both risks?	<code>bcr_excluding_line losses_and_wildfire_risk_and_outage_risk</code>
Ignore emissions and wildfire risk (keep outage)?	<code>bcr_excluding_emissions_and_wildfire_risk</code>
Ignore emissions and outage risk (keep wildfire)?	<code>bcr_excluding_emissions_and_outage_risk</code>
Core project economics only (no externalities, no risks)?	<code>bcr_excluding_emissions_and_line losses_and_wildfire_risk_and_outage_risk</code>

26.4 Important Notes

1. **Haircut Benefits:** Most BCRs use conservative “haircut” benefits (saturation factors applied to congestion/curtailment).
2. **Present Value:** All values are in present value (PV) terms, discounted to base year.
3. **Context:** Wildfire costs assume California-style catastrophic events. For lower wildfire risk regions, `bcr_excluding_wildfire_risk` may be more appropriate. Outage risk is more universal. Line losses are socialized through rates; emissions are externalities.
4. **Separating Wildfire and Outage Risk:** Use `bcr_excluding_wildfire_risk` for Virginia-style scenarios; `bcr_excluding_wildfire_risk_and_outage_risk` when both should be excluded; `bcr_excluding_outage_risk` when only outage risk should be excluded.

26.5 Example: Scenario 1 (Rural Overhead AC 657MW)

- `bcr_utility` = 0.36 → Utility loses money.

- `bcr_ratepayer` = 0.22 $\rightarrow$ Ratepayers lose money.
- `bcr_system` = 0.02 $\rightarrow$ Very poor societal return (wildfire risk dominates).
- `bcr_excluding_wildfire_risk_and_outage_risk` = 0.24 $\rightarrow$ Better but still negative.

The separation of wildfire and outage risk allows for more nuanced regional analysis.

Chapter 27

Utility Modules Documentation

27.1 Introduction

This document describes the utility modules used throughout the CTCC: `yaml_loaders.py` (or `smart_loaders`, which dispatches to YAML/JSON by mode), `financial_utils.py`, `calculation_utils.py`, `constants.py`, and `weighted_miles.py`. They provide shared functionality for loading configuration data, financial calculations (PV, AFUDC, discount rate from config), transmission line parameters (phase current, line/converter losses), constants (e.g. `MIN_DISCOUNT_RATE`, `HOURS_PER_YEAR`), and terrain-adjusted distances.

27.2 `yaml_loaders.py`

27.2.1 Method Overview

The `yaml_loaders.py` module provides centralized functions for loading YAML configuration files. It eliminates code duplication by providing a single interface for accessing project parameters, financing details, physical details, and cost parameters.

27.2.2 Key Functions

`load_project_technical_details()`

Loads project technical details from `01_project_technical_details.yaml`.

Returns:

- Tuple: (`construction_type`, `ac_dc`, `capacity_mw`, `conductor_type`, `converter_type`, `line_utilization`, `reconductoring`, `delay_years`, `construction_years`, `project_lifetime`)

`load_physical_details()`

Loads physical project details and returns total miles only.

Returns:

- `total_miles` (float): Sum of all terrain miles

`load_physical_details_detailed()`

Loads physical project details with detailed terrain breakdown.

Returns:

- Tuple: (total_miles, forested_miles, scrubbed_flat_miles, wetland_miles, farmland_miles, desert_barren_miles, urban_miles, rolling_hills_miles, mountain_miles, subsea_miles)

`load_financing_details()`

Loads financing parameters and calculates real WACC using Fisher equation.

Returns:

- Tuple: (inflation_rate, base_year, wacc_nominal, wacc_real)

Key Logic:

```
1 wacc_real = (1 + wacc_nominal) / (1 + inflation_rate) - 1
```

`load_circuit_and_resistance_details(category)`

Loads circuit and resistance details for specified project category.

Parameters:

- `category` (str): Project category identifier

Returns:

- Tuple: (voltage_kv, conductors_per_phase, number_of_phases, number_of_circuits_poles, AC_75_resistance, DC_20_resistance)

Other Loader Functions

Additional loader functions include:

- `load_row_widths(category)` - ROW width for project category
- `load_row_details()` - ROW cost parameters
- `load_delay_costs()` - Delay cost components
- `load_emissions_details()` - Emissions parameters
- `load_congestion_reductions()` - Congestion reduction parameters
- `load_curtailment_reductions()` - Curtailment reduction parameters
- `load_contingencies()` - Contingency percentages
- `load_environmental_mitigation()` - Environmental mitigation parameters

- `load_cost_timing_patterns()` - AFUDC timing patterns
- `load_afudc_config()` - AFUDC configuration
- `load_insurance_details()` - Insurance parameters
- `load_wildfire_costs()` - Wildfire parameters
- `load_outage_costs()` - Outage parameters

27.2.3 Example

```

1 from yaml_loaders import (
2     load_project_technical_details,
3     load_financing_details,
4     load_physical_details
5 )
6
7 # Load project specs
8 construction_type, ac_dc, capacity_mw, ... =
9     load_project_technical_details()
10
11 # Load financing
12 inflation_rate, base_year, wacc_nominal, wacc_real =
13     load_financing_details()
14
15 # Load physical details
16 total_miles = load_physical_details()

```

27.3 financial_utils.py

27.3.1 Method Overview

The `financial_utils.py` module provides shared financial calculation functions: present value (level and growing annuity), COD year, discount rate from config, amortization, and AFUDC capitalization.

Nominal WACC (`r_wacc,nominal`): When capital structure is present and valid (equity_percent, debt_percent, cost_of_equity, cost_of_debt; percentages sum to 1.0; not both costs zero), $r_wacc,nominal = equity_percent \times cost_of_equity + debt_percent \times cost_of_debt$. Otherwise $r_wacc,nominal = financial.wacc_nominal$. This value is used for both discounting (via real WACC from the Fisher equation) and for the AFUDC rate. The helper `get_wacc_nominal(financing_yaml)` returns this value.

27.3.2 Key Functions

`get_discount_rate_from_config(config_yaml, financing_yaml,
rate_key="discount_rate_type")`

Returns discount rate and source description from config (e.g. wildfire, outage). Supports `discount_rate_type` or `discount_rate_source`; uses financing YAML for social or WACC real. Returns (`discount_rate`, `source_description`).

`calculate_cod_year(delay_years, construction_years)`

Returns Commercial Operation Date year: `delay_years + construction_years + 1.0` (can be fractional). Used as `start_year` in PV calculations so benefits/costs start at COD.

`calculate_growing_annuity_pv(annual_amount, growth_rate, discount_rate,
project_lifetime, delay_years=0, construction_years=0)`

Present value of a growing annuity starting at COD; used for wildfire, outage, and similar streams.

`calculate_nominal_growing_series(annual_amount, growth_rate,
project_lifetime)`

Nominal (undiscounted) sum of growing annual amounts over project lifetime.

`validate_discount_rate(discount_rate)`

Raises `ValueError` if discount rate $\leq$ `constants.MIN_DISCOUNT_RATE` (avoids division by zero in PV).

`calculate_present_value(annual_cost, wacc_real, total_years, start_year=1)`

Calculates the present value of annual payments over a given time period.

Parameters:

- `annual_cost` (float): Annual cost amount
- `wacc_real` (float): Real WACC (discount rate)
- `total_years` (float): Number of years
- `start_year` (int): Year when payments begin (default: 1)

Returns:

- `present_value` (float): Present value of the payment stream

Formula:

$$PV = \sum_{t=start}^{start+total-1} \frac{C}{(1+r)^t}$$

`calculate_amortized_cost(principal, wacc_real, project_lifetime)`

Calculates annual amortized cost using standard amortization formula.

Parameters:

- `principal` (float): Initial cost
- `wacc_real` (float): Real WACC (discount rate)
- `project_lifetime` (int): Number of years to amortize

Returns:

- `annual_payment` (float): Annual amortized payment

Formula:

$$A = P \times \frac{r(1+r)^n}{(1+r)^n - 1}$$

where P is principal, r is rate, and n is number of years.

`get_wacc_nominal(financing_yaml)`

Returns nominal WACC (`r_wacc,nominal`). Calls `calculate_afudc_rate(financing_yaml)` and returns the rate; same logic so discounting and AFUDC use the same nominal rate when capital structure is present.

Returns: float — Nominal WACC.

`calculate_afudc_rate(financing_yaml)`

Calculates AFUDC rate from capital structure or falls back to WACC.

Parameters:

- `financing_yaml` (dict): Loaded financing YAML data

Returns:

- Tuple: (`afudc_rate`, `source_description`)

Key Logic:

```
1 if capital_structure specified:
2     rate = equity_percent * cost_of_equity + debt_percent *
      cost_of_debt
3     source = "capital structure"
4 else:
5     rate = wacc_nominal
6     source = "WACC nominal"
```

`calculate_afudc_capitalized_cost(nominal_cost, timing_pattern, delay_years, construction_years, afudc_rate, delay_has_active_work)`

Capitalizes a cost using AFUDC (compounds forward to Commercial Operation Date). If `timing_pattern["afudc_eligible"]` is False, returns `(nominal_cost, 0.0)` with no capitalization.

Parameters:

- `nominal_cost` (float): Total nominal cost
- `timing_pattern` (dict): Dict with `during_delay`, `during_construction`, `afudc_eligible`. When `afudc_eligible` is true, `during_delay` and `during_construction` must sum to 1.0 (within tolerance 0.001) so that 100% of the cost is allocated to delay and/or construction.
- `delay_years` (float): Number of delay years
- `construction_years` (float): Number of construction years
- `afudc_rate` (float): AFUDC rate (annual)
- `delay_has_active_work` (bool): Whether active CWIP work continues during delay

Returns:

- Tuple: `(capitalized_cost, afudc_amount)`

Key Logic:

```
1 # Cost during delay period (only if afudc_eligible)
2 delay_cost = nominal_cost * timing_pattern["during_delay"]
3 if delay_cost > 0:
4     if delay_has_active_work and delay_years > 0:
5         # AFUDC during delay: uniform spending, compound from
6         # midpoint to COD
7         avg_years_to_cod = total_years_to_cod - (delay_years / 2)
8         capitalized_cost += delay_cost * (1 + afudc_rate) **
9         avg_years_to_cod
10    else:
11        if delay_years > 0:
12            # No active work: AFUDC suspended during delay, compound
13            # only during construction
14            capitalized_cost += delay_cost * (1 + afudc_rate) **
15            construction_years
16        else:
17            capitalized_cost += delay_cost # No delay period
18
19 # Cost during construction period
20 construction_cost = nominal_cost * timing_pattern["
    during_construction"]
```

```

17 if construction_cost > 0 and construction_years > 0:
18     # Uniform spending: compound from midpoint of construction to
        COD
19     avg_years_to_cod = construction_years / 2
20     capitalized_cost += construction_cost * (1 + afudc_rate) **
        avg_years_to_cod
21 else:
22     capitalized_cost += construction_cost
23
24 afudc_amount = capitalized_cost - nominal_cost

```

27.3.3 Example

```

1 from financial_utils import (
2     calculate_present_value,
3     calculate_amortized_cost,
4     calculate_afudc_capitalized_cost
5 )
6
7 # Present value of annual O&M costs
8 pv = calculate_present_value(
9     annual_cost=1000000,
10    wacc_real=0.03,
11    total_years=50,
12    start_year=3
13 )
14
15 # Amortize build cost
16 annual_payment = calculate_amortized_cost(
17     principal=100000000,
18     wacc_real=0.03,
19     project_lifetime=50
20 )
21
22 # AFUDC capitalization
23 capitalized, afudc = calculate_afudc_capitalized_cost(
24     nominal_cost=100000000,
25     timing_pattern={"during_delay": 0.1, "during_construction": 0.9,
        "afudc_eligible": True},
26     delay_years=2,
27     construction_years=2,
28     afudc_rate=0.05,
29     delay_has_active_work=False
30 )

```

27.4 calculation_utils.py

27.4.1 Method Overview

The `calculation_utils.py` module provides shared calculation functions for transmission line parameters, including phase current, full load adjustment, line losses, and converter losses.

27.4.2 Key Functions

`calculate_phase_current(capacity_mw, voltage_kv, number_of_phases, number_of_circuits_poles, ac_dc)`

Calculates phase current based on AC or DC transmission type.

Parameters:

- `capacity_mw` (float or str): Line capacity in MW
- `voltage_kv` (float): Voltage in kV
- `number_of_phases` (int): Number of phases
- `number_of_circuits_poles` (int): Number of circuits/poles
- `ac_dc` (str): "AC" or "DC"

Returns:

- `phase_current` (float): Phase current in Amps

Formulas:

$$I_{AC} = \frac{P \times 1000}{0.95 \times V \times \sqrt{N_{phases}} \times N_{circuits}} \quad (27.1)$$

$$I_{DC} = \frac{P \times 1000}{V \times \sqrt{N_{phases}} \times N_{poles}} \quad (27.2)$$

`full_load_adjusted(line_utilization_percent)`

Calculates full load adjustment based on line utilization.

Parameters:

- `line_utilization_percent` (float): Line utilization as decimal (0-1)

Returns:

- `adjustment_factor` (float): Full load adjustment factor

Formula:

$$\text{Adjustment} = \frac{u + u^2}{2}$$

where u is the utilization factor.

`calculate_line_losses(...)`

Calculates line losses for transmission lines.

Parameters:

- `phase_current` (float): Phase current in Amps
- `full_load_adj` (float): Full load adjustment factor
- `AC_75_resistance` (float): AC resistance at 75°C (ohms/mile)
- `DC_20_resistance` (float): DC resistance at 20°C (ohms/mile)
- `ac_dc` (str): "AC" or "DC"
- `number_of_circuits_poles` (int): Number of circuits/poles
- `conductors_per_phase` (int): Number of conductors per phase
- `number_of_phases` (int): Number of phases
- `line_length` (float): Line length in miles
- `project_lifetime` (int): Project lifetime in years
- `capacity_mw_numeric` (float): Capacity in MW
- `line_utilization_percent` (float): Line utilization as decimal

Returns:

- Tuple: (`losses_mwh_per_year`, `lifetime_losses_mwh`, `losses_mw_per_mile`, `resistance_per_mile`, `line_loss_per_mile_percent`, `total_line_loss_mw`, `total_line_loss_percent`)

Key Logic:

```
1 # Select resistance based on AC/DC
2 resistance_per_mile = AC_75_resistance if ac_dc == "AC" else
   DC_20_resistance
3
4 # Calculate losses per mile
5 number_of_conductors = conductors_per_phase * number_of_phases *
   number_of_circuits_poles
6 losses_mw_per_mile = ((phase_current / conductors_per_phase) ** 2
7                       * number_of_conductors
8                       * resistance_per_mile
9                       * full_load_adj / 1000000)
10
11 # Total losses
12 total_line_loss_mw = losses_mw_per_mile * line_length
13 losses_mwh_per_year = total_line_loss_mw * 8760
14 lifetime_losses_mwh = losses_mwh_per_year * project_lifetime
```

`calculate_converter_losses(number_of_converters, converter_type,
line_utilization_percent, capacity_mw_numeric, ac_dc, converter_loss_percentage=None)`

Calculates converter losses based on number of converters (DC only).
Uses `converter_loss_percentage` from project if provided; otherwise
`constants.LCC_CONVERTER_LOSS` or `VSC_CONVERTER_LOSS` by converter type.

Parameters:

- `number_of_converters` (int): Number of converters
- `converter_type` (str): Converter type (LCC or VSC)
- `line_utilization_percent` (float): Line utilization as decimal
- `capacity_mw_numeric` (float): Capacity in MW
- `ac_dc` (str): "AC" or "DC"
- `converter_loss_percentage` (float, optional): Override loss rate from project

Returns:

- Tuple: (total_converter_losses_mw, total_converter_losses_mwh, converter_loss_percent)

Key Logic:

```
1 if ac_dc == "DC":
2     converter_loss_percentage = 0.0075 if "LCC" in converter_type
3     else 0.01
4     converter_losses_mw = (converter_loss_percentage
5                             * line_utilization_percent
6                             * capacity_mw_numeric)
7     total_converter_losses_mw = converter_losses_mw *
8     number_of_converters
9     total_converter_losses_mwh = total_converter_losses_mw * 8760
10 else:
11     total_converter_losses_mw = 0
12     total_converter_losses_mwh = 0
```

Other calculation_utils functions

- `build_category_string(...)` — Build project category for YAML/JSON lookups (e.g. Overhead/AC/500MW/Advanced Aluminum/NA).
- `normalize_construction_type_for_yaml(construction_type)` — Map script construction type to YAML key (e.g. overhead → Overhead).
- `get_converter_type(ac_dc, converter_type)` — Return converter type for category (e.g. AC → NA).

- `miles_to_acres(miles, row_width_feet)` — Convert ROW miles to acres using `constants.FEET_PER_MILE` and `SQUARE_FEET_PER_ACRE`.
- `to_percent(decimal)`, `from_percent(percentage)` — Convert between decimal and percentage.

27.4.3 Example

```

1 from calculation_utils import (
2     calculate_phase_current,
3     full_load_adjusted,
4     calculate_line_losses,
5     build_category_string,
6 )
7
8 # Calculate phase current
9 phase_current = calculate_phase_current(
10     capacity_mw=500,
11     voltage_kv=345,
12     number_of_phases=3,
13     number_of_circuits_poles=1,
14     ac_dc="AC"
15 )
16
17 # Calculate full load adjustment
18 full_load_adj = full_load_adjusted(0.7) # 70% utilization
19
20 # Calculate line losses
21 losses_mwh_per_year, lifetime_losses_mwh, ... =
22     calculate_line_losses(
23         phase_current, full_load_adj, AC_75_resistance, DC_20_resistance
24         ,
25         "AC", 1, 2, 3, 100, 50, 500, 0.7
26     )

```

27.5 weighted_miles.py

27.5.1 Method Overview

The `weighted_miles.py` module calculates weighted miles and average terrain multiplier for transmission lines. Weighted miles account for terrain complexity by multiplying miles in each terrain type by a terrain-specific cost multiplier.

27.5.2 Key Functions

`calculate_weighted_miles()`

Calculates weighted miles and average terrain multiplier from project physical details.

Returns:

- Tuple: (weighted_miles, average_terrain_multiplier)

Calculation:

$$\text{Weighted Miles} = \sum_{\text{terrains}} \text{Miles}_{\text{terrain}} \times \text{Multiplier}_{\text{terrain}} \quad (27.3)$$

$$\text{Average Multiplier} = \frac{\text{Weighted Miles}}{\text{Total Miles}} \quad (27.4)$$

Key Logic:

```
1 weighted_miles = 0
2 for terrain_type, terrain_miles in terrain_miles_dict.items():
3     multiplier = terrain_multipliers[terrain_type]
4     weighted_miles += terrain_miles * multiplier
5
6 average_terrain_multiplier = weighted_miles / total_miles
```

27.5.3 Example

```
1 from weighted_miles import calculate_weighted_miles
2
3 # Calculate weighted miles
4 weighted_miles, avg_multiplier = calculate_weighted_miles()
5
6 # Example calculation:
7 # Terrain distribution:
8 #   - Forested: 40 miles, multiplier: 2.25
9 #   - Farmland: 60 miles, multiplier: 1.0
10 #   - Total: 100 miles
11 #
12 # Weighted miles = (40 * 2.25) + (60 * 1.0) = 150 miles
13 # Average multiplier = 150 / 100 = 1.5
```

27.6 Integration Examples

27.6.1 Combined Usage

```

1 # Typical usage in a cost calculation script:
2 from yaml_loaders import (
3     load_project_technical_details,
4     load_financing_details,
5     load_physical_details
6 )
7 from financial_utils import calculate_present_value
8 from weighted_miles import calculate_weighted_miles
9
10 # Load configuration
11 construction_type, ac_dc, capacity_mw, ... =
12     load_project_technical_details()
13 inflation_rate, base_year, wacc_nominal, wacc_real =
14     load_financing_details()
15 total_miles = load_physical_details()
16
17 # Calculate weighted miles
18 weighted_miles, avg_multiplier = calculate_weighted_miles()
19
20 # Calculate present value
21 pv = calculate_present_value(
22     annual_cost=1000000,
23     wacc_real=wacc_real,
24     total_years=50,
25     start_year=3
26 )

```

27.7 Key Implementation Details

27.7.1 Path Resolution

All YAML loaders use relative paths ("../yaml/") assuming scripts are run from the project root directory. The `csv_output_manager.py` uses dynamic path resolution to handle different execution contexts.

27.7.2 Fisher Equation

The real WACC calculation uses the Fisher equation to convert nominal WACC to real WACC, accounting for inflation:

$$r_{real} = \frac{1 + r_{nominal}}{1 + \pi} - 1$$

27.7.3 Full Load Adjustment

The full load adjustment accounts for the fact that losses are proportional to current squared, and average current is less than peak current. The formula $(u + u^2)/2$ provides a better approximation than simple u^2 for typical utilization patterns.

27.7.4 AFUDC Timing

AFUDC capitalization compounds costs forward to Commercial Operation Date (COD). The calculation accounts for:

- Costs during delay period (may or may not have AFUDC depending on active work)
- Costs during construction period (always has AFUDC)
- Uniform spending assumption (uses midpoint of each period)

27.7.5 Terrain Multipliers

Terrain multipliers reflect the relative cost/complexity of construction in different terrain types. For example, forested terrain may have a multiplier of 2.25, meaning construction costs 2.25x more per mile than baseline terrain.

27.8 constants.py

27.8.1 Method Overview

The `constants.py` module centralizes magic numbers used across CTCC scripts.

27.8.2 Key Constants

- `FEET_PER_MILE`, `SQUARE_FEET_PER_ACRE`, `HOURS_PER_YEAR`
- `MIN_DISCOUNT_RATE` — Minimum allowed discount rate (e.g. -0.99) to avoid division by zero in PV
- `GROWTH_RATE_TOLERANCE`, `DISCOUNT_GROWTH_EQUALITY_TOLERANCE`
- `LCC_CONVERTER_LOSS`, `VSC_CONVERTER_LOSS` — Converter loss fractions
- `AC_POWER_FACTOR` — Power factor for AC phase current
- `CONSTRUCTION_TYPE_OVERHEAD`, `CONSTRUCTION_TYPE_UNDERGROUND`, `CONSTRUCTION_TYPE_SUBSEA`, etc.
- `CONVERTER_TYPE_NA` — Used when AC (no converter)

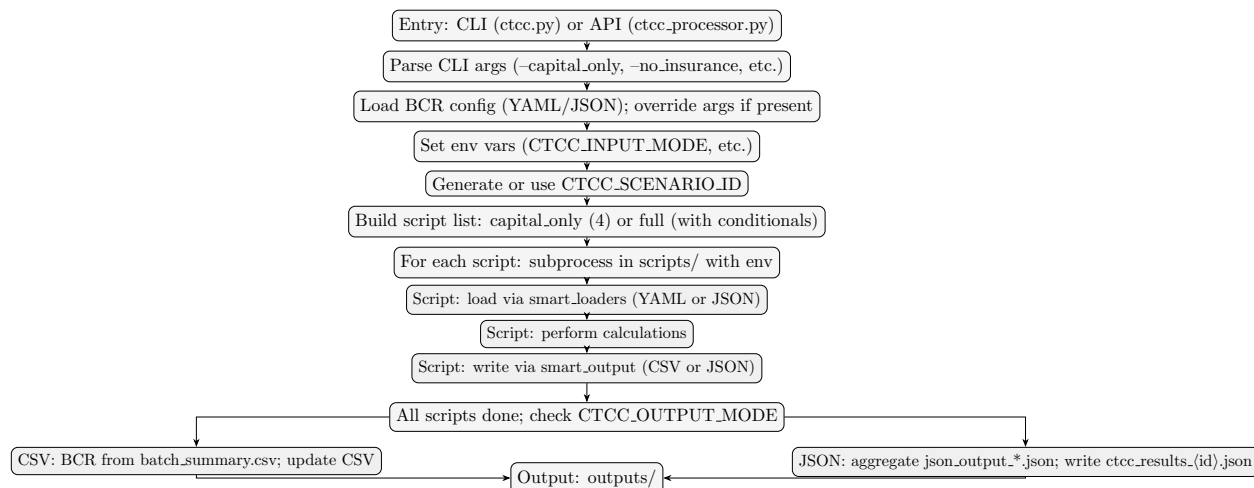
Chapter 28

Master Code Map

28.1 Introduction

This chapter maps every step of CTCC's execution: from entry (CLI or API) through argument parsing, environment setup, script list construction, sequential run of calculation scripts, and final BCR and output (CSV or JSON). A full workflow diagram in Mermaid syntax is available in `docs/ctcc_execution_workflow.txt`.

28.2 Execution Flow Diagram



28.3 Every Step (Numbered)

1. Entry: user runs `python ctcc.py` (CLI) or POSTs to API `/api/ctcc/calculate` (server writes combined JSON to temp file and calls `ctcc.py` via subprocess).
2. Parse command-line arguments: `--norisk`, `--simple`, `--no_wf_liability`, `--no_emissions`, `--no_line losses`, `--capital_only`, `--no_oandm`, `--no_insurance`,

`--no_delay_costs,` `--no_wildfire,` `--no_outages,` `--no_congestion,`
`--no_curtailement,` `-j,` `-o,` `--id,` etc.

3. Load primary BCR config from YAML or JSON (based on `CTCC_INPUT_MODE`); if present, merge with CLI flags (config overrides).
4. Build `BCRConfig` from args and set environment variables (`CTCC_NO_CONGESTION`, `CTCC_NO_WF_LIABILITY`, etc.).
5. Generate `CTCC_SCENARIO_ID` if not set (e.g. `YYYYMMDD_HHMMSS_microseconds`), or use existing (e.g. from `--id` or API).
6. If JSON input mode: set `CTCC_JSON_DATA_FILE` (and `os.environ`) to the combined data file path.
7. Build script list: if `--capital_only`, list is `weighted_miles.py`, `build_costs.py`, `row_costs.py`, `environmental_mitigation.py`. Otherwise, start with `weighted_miles.py`, `build_costs.py`, `row_costs.py`, `environmental_mitigation.py`, `revenue.py`; then append `insurance_costs.py` unless `--no_insurance`, `delay_costs.py` unless `--no_delay_costs`, `wildfire_costs.py` unless `--no_wildfire`, `outage_costs.py` unless `--no_outages`; then `congestion_curtailement_reduction.py`, `energy_losses.py`; then `oandm.py` unless `--no_oandm`; then `emissions.py` unless `--no_emissions`, `line_loss_costs.py` unless `--no_line_losses`.
8. For each script in order: run `sys.executable script_name` as subprocess with `cwd=scripts`, `env=os.environ.copy()`; capture stdout/stderr; track success/failure.
9. Each script: (a) loads data via `smart_loaders` (dispatches to YAML or JSON loaders per `CTCC_INPUT_MODE`); (b) performs its calculations; (c) writes results via `smart_output` (CSV or JSON per `CTCC_OUTPUT_MODE`).
10. After all scripts: if no successful runs, exit with error. Otherwise, if `CTCC_OUTPUT_MODE=csv`: run `calculate_and_display_bcr` on `batch_summary.csv`, update `batch_summary.csv` with BCR metrics via `CSVOutputManager`. If `CTCC_OUTPUT_MODE=json`: `aggregate_json_outputs` loads and merges `json_output_{id}*.json`, `build_bcr_data_from_json` extracts flat dict, BCR computed via `calculate_benefits/calculate_costs/calculate_bcr_metrics`, `write_final_json_output` writes `ctcc_results_{id}.json`, then intermediate JSON files are removed.
11. Exit: code 0 if all scripts succeeded; 1 if any failed.

28.4 Script Inclusion Table

Table 28.1 shows when each script is included in the run list and its purpose in one line.

With `--capital_only`, only the first four scripts (`weighted_miles`, `build_costs`, `row_costs`, `environmental_mitigation`) are run.

Script	When included	Purpose
weighted_miles.py	Always	Terrain-adjusted miles; prerequisite for build_costs, insurance
build_costs.py	Always	Construction costs (conductor, structure, converter; AFUDC/PV)
row_costs.py	Always	Right-of-way costs
environmental_mitigation.py	Always	Environmental mitigation costs
revenue.py	Always (full run)	Revenue requirement
insurance_costs.py	If not <code>--no_insurance</code>	Insurance costs
delay_costs.py	If not <code>--no_delay_costs</code>	Construction/delay costs
wildfire_costs.py	If not <code>--no_wildfire</code>	Wildfire risk costs
outage_costs.py	If not <code>--no_outages</code>	Outage risk costs
congestion_curtailment_reduction.py	Always (full run)	Congestion and curtailment benefits
energy_losses.py	Always (full run)	Loss data; prerequisite for line_loss_costs
oandm.py	If not <code>--no_oandm</code>	O&M costs
emissions.py	If not <code>--no_emissions</code>	Emissions costs
line_loss_costs.py	If not <code>--no_linelosses</code>	Line loss costs/benefits

Table 28.1: Scripts: inclusion condition and one-line purpose.